

Sicurezza dei Sistemi

Appunti del Corso

Università degli Studi

Indice

1	Fondamenti di Sicurezza	10
1.1	Introduzione alla Sicurezza Informatica	10
1.1.1	La Triade CIA	10
1.1.2	Livelli di Impatto delle Violazioni	11
1.2	Terminologia della Sicurezza	11
1.2.1	Modelli di Controllo degli Accessi	11
1.3	Tipologie di Attacco e di Minacce	12
1.3.1	Classificazione degli Attacchi	12
1.3.2	Le Minacce Specifiche	12
1.4	Valutazione delle Minacce	13
1.4.1	Superfici di Attacco	13
1.5	Minacce alle Risorse del Sistema	14
2	Crittografia e Funzioni Hash	15
2.1	Cifratura Simmetrica	15
2.1.1	I Cinque Componenti di uno Schema Simmetrico	15
2.1.2	Requisiti per un Utilizzo Sicuro	15
2.1.3	Metodi di Attacco alla Cifratura Simmetrica	16
2.2	Analisi degli Algoritmi: DES, Triple DES e AES	16
2.3	Modalità Operative e Cifrari	16
2.3.1	Il Problema della Modalità ECB (Electronic Codebook)	16
2.3.2	Cifrari a Blocchi vs Cifrari a Flusso	17
2.4	Funzioni Hash e Digest	17
2.4.1	Differenza Fondamentale: Hash vs Crittografia	17
2.4.2	Requisiti delle Funzioni Hash	17
2.5	Autenticazione dei Messaggi	18
2.5.1	Il Limite della Sola Cifratura	18
2.5.2	Message Authentication Code (MAC)	18

2.6	Crittografia a Chiave Pubblica (Asimmetrica)	19
2.6.1	Miti Comuni sulla Crittografia Asimmetrica	19
2.6.2	I Sei Componenti	19
2.6.3	Processi Fondamentali	19
2.6.4	La Busta Digitale	20
2.6.5	Requisiti degli Algoritmi Asimmetrici	20
2.6.6	Panoramica degli Algoritmi Asimmetrici	20
2.7	Firme Digitali	21
2.8	Certificati a Chiave Pubblica	21
3	Autenticazione Digitale degli Utenti	22
3.1	Introduzione	22
3.2	Modello di Autenticazione Digitale	22
3.3	Mezzi di Autenticazione	23
3.4	Rischi nell'Autenticazione degli Utenti	23
3.4.1	Livello di Garanzia (Level of Assurance - LoA)	23
3.4.2	Impatto Potenziale e Aree di Rischio	24
3.5	Autenticazione Basata su Password	24
3.5.1	Vulnerabilità e Contromisure	24
3.6	Hash delle Password e Uso del Salt	24
3.6.1	Perché il Salt è indispensabile?	25
3.7	Cracking delle Password e Approcci Moderni	25
3.7.1	Migliorare la Selezione della Password	25
3.8	Autenticazione Basata su Token	25
3.8.1	Memory Card	25
3.8.2	Smart Card e Token Crittografici	26
3.9	Autenticazione Biometrica	26
3.9.1	Tipologie di Caratteristiche Biometriche	26
4	Controllo degli Accessi	27
4.1	Introduzione	27
4.1.1	Obiettivi Principali	27
4.1.2	Contesto Operativo	27
4.2	Modelli e Politiche di Controllo degli Accessi	28
4.3	Gestione delle Identità, Credenziali e Accessi (ICAM)	29
4.3.1	1. Gestione delle Identità	29
4.3.2	2. Gestione delle Credenziali	29

4.3.3	3. Gestione degli Accessi	29
4.4	Trust Framework (OITF)	30
4.4.1	Componenti e Ruoli del Framework OITF	30
5	SQL Injection e Sicurezza dei Database	31
5.1	SQL Injection	31
5.1.1	Come Funzionano le SQLi e Fonti di Attacco	31
5.2	Tipi di SQL Injection	32
5.2.1	1. Attacchi In Banda (In-Band SQLi)	32
5.2.2	2. Attacchi Inferenziali (Inferential SQLi)	33
5.2.3	3. Attacchi Fuori Banda (Out-of-Band SQLi)	33
5.3	Contromisure per le SQL Injection	33
5.3.1	1. Programmazione Difensiva	33
5.3.2	2. Rilevamento (Detection)	34
5.3.3	3. Prevenzione a Tempo di Esecuzione (Runtime)	34
5.4	Controllo degli Accessi ai Database (Amministrazione)	34
5.4.1	Ruoli nel Database (RBAC)	34
5.5	Inferenza nei Database e Cifratura	35
5.5.1	Contromisure contro l'Inferenza	35
5.5.2	Cifratura dei Database	35
6	Malware	36
6.1	Introduzione	36
6.1.1	Classificazione e Propagazione	36
6.2	Tipi di Malware	36
6.3	Virus	37
6.3.1	Struttura Generale e Fasi	38
6.3.2	Classificazione dei Virus	38
6.3.3	Macro Virus e di Scripting	39
6.4	I Worm	39
6.4.1	Meccanismi di Diffusione	39
6.4.2	Ricerca del Target (Scanning)	40
6.5	Vulnerabilità Lato Client	40
6.6	Trojan	40
6.7	Payload di un Malware	41
6.7.1	Distruzione dei Dati	41
6.7.2	Bombe Logiche	41

6.8	Agenti di Attacco: Bot e Botnet	41
6.8.1	Uso delle Botnet	41
6.8.2	Controllo Remoto (Command and Control)	42
6.9	Furto di Credenziali	42
6.9.1	Keylogger e Spyware	42
6.9.2	Phishing	42
6.10	Stealth (Backdoor e Rootkit)	42
6.10.1	Backdoor	42
6.10.2	Rootkit	43
7	Attacchi Denial-of-Service	44
7.1	Introduzione	44
7.2	TCP SYN Flood	44
7.2.1	Funzionamento del TCP SYN Flood	44
7.2.2	Come si Verifica l'Attacco	45
7.2.3	Tipologie di Attacco SYN Flood	45
7.2.4	Mitigazione degli Attacchi SYN Flood	45
7.3	ICMP Flood	45
7.3.1	Funzionamento dell'ICMP Flood	46
7.3.2	Mitigare l'ICMP Flood	46
7.4	UDP Flood	46
7.4.1	Funzionamento dell'UDP Flood	46
7.4.2	Mitigare l'UDP Flood	46
7.5	HTTP Flood	47
7.5.1	Tipologie di Attacco HTTP Flood	47
7.5.2	Mitigare un HTTP Flood	47
7.6	DNS Amplification Attack	47
7.6.1	Concetti Chiave	47
7.6.2	Funzionamento del DNS Amplification Attack	47
7.6.3	Mitigare il DNS Amplification Attack	48
8	Intrusion Detection System	49
8.1	Introduzione	49
8.2	Struttura di un IDS	49
8.3	Configurazioni di IDS	49
8.4	Principi di Base e Requisiti	50
8.4.1	Requisiti di un IDS Efficace	50

8.5	Analisi di un IDS (Metodi di Rilevamento)	50
8.5.1	Rilevazione delle Anomalie (Anomaly Detection)	50
8.5.2	Rilevazione tramite Firma o Euristica (Misuse Detection)	51
8.6	Host-Based e Network-Based IDS	51
8.6.1	HIDS Distribuito	51
8.6.2	Network-Based Intrusion Detection System (NIDS)	51
8.7	IDS Distribuito e Cooperativo (Ibrido)	51
8.7.1	Sfide Moderne	52
8.7.2	Rilevamento Cooperativo (Gossip)	52
8.8	Honeypot	52
8.8.1	Classificazione degli Honeypot	52
8.8.2	Posizionamento degli Honeypot nella Rete	53
9	Firewall	54
9.1	Introduzione	54
9.2	Requisiti Minimi e Politiche d'Accesso	54
9.3	Filtri del Firewall	54
9.3.1	Firewall a Filtraggio di Pacchetto (Stateless)	55
9.3.2	Firewall a Filtraggio di Pacchetto con Stato (Stateful)	55
9.4	Tipi di Firewall Avanzati	55
9.4.1	Gateway a Livello Applicativo (Proxy)	55
9.4.2	Proxy a Livello di Circuito	56
9.5	Classificazione per Locazione	56
9.6	Architetture di Sicurezza di Rete (Topologie)	56
9.6.1	La Rete DMZ (Demilitarized Zone)	56
9.6.2	Topologie Comuni	57
9.6.3	VPN e Firewall	57
9.7	Intrusion Prevention System (IPS)	57
9.7.1	Host-Based IPS (HIPS)	57
9.7.2	Network-Based IPS (NIPS)	58
9.7.3	Sistema Immunitario Digitale	58
10	Buffer Overflow	59
10.1	Principi del Buffer Overflow	59
10.1.1	Cause e Conseguenze	59
10.1.2	Requisiti per Sfruttare un Overflow	59
10.2	Shellcode	60

10.2.1	Caratteristiche e Restrizioni	60
10.3	Difese Contro i Buffer Overflow	60
10.3.1	Difese a Tempo di Compilazione	60
10.3.2	Difese a Tempo di Esecuzione	61
10.4	Altre Forme di Attacchi Overflow	61
11	Sicurezza del Software	62
11.1	Problemi di Sicurezza del Software	62
11.2	Sicurezza vs. Qualità del Software	62
11.3	Programmazione Difensiva	62
11.4	Gestione Sicura dell'Input	63
11.4.1	Dimensione dell'Input e Buffer Overflow	63
11.4.2	Interpretazione dell'Input e Attacchi di Injection	63
11.4.3	Tecniche di Validazione dell'Input	64
11.5	Scrittura di Codice Sicuro e Gestione della Memoria	64
11.5.1	Tipizzazione, Memory Leak e Race Condition	64
11.6	Interazione con il Sistema Operativo e Privilegi	64
11.7	Gestione Sicura dell'Output e Cross-Site Scripting (XSS)	65
12	Sicurezza nei OS e Virtualizzazione	66
12.1	Introduzione alla Sicurezza dei Sistemi Operativi	66
12.2	Il Processo di Messa in Sicurezza (NIST SP 800-123)	66
12.3	Hardening del Sistema Operativo	66
12.3.1	Manutenzione, Logging e Backup	67
12.4	Sicurezza in Ambiente Linux/Unix	67
12.5	Sicurezza in Ambiente Windows	68
12.6	Sicurezza della Virtualizzazione	68
12.6.1	Tipologie di Hypervisor	68
12.6.2	Minacce e Isolamento negli Ambienti Virtualizzati	69
12.7	Metasploit Framework: Penetration Testing e Assessment	69
12.7.1	Architettura e Moduli	69
13	Cloud Computing e IoT	70
13.1	Cloud Computing: Definizioni e Architettura	70
13.1.1	Modelli di Servizio Cloud	70
13.1.2	Modelli di Deployment	71
13.2	Sicurezza nel Cloud Computing	71

13.2.1 Multi-Tenancy e Isolamento	71
13.2.2 Sicurezza dei Dati e Database nel Cloud	71
13.2.3 Security as a Service (SecaaS) e OpenStack Keystone	72
13.3 Internet of Things (IoT) e Livelli di Rete	72
13.3.1 Architettura di Rete IoT	72
13.4 Sicurezza nell'Ambiente IoT	73
13.4.1 Requisiti e Framework Operativi	73
13.4.2 Sicurezza Open-Source: Il modulo MiniSec	73
14 Sicurezza IT e Prevenzione dei Rischi	74
14.1 Definizione e Ciclo di Gestione	74
14.1.1 Il Modello Plan-Do-Check-Act (PDCA)	74
14.2 Contesto e Policy di Sicurezza	74
14.3 Il Processo di Risk Assessment	75
14.3.1 Approcci alla Valutazione del Rischio	75
14.4 Analisi Dettagliata del Rischio	75
14.4.1 1. Contesto e Identificazione degli Asset	75
14.4.2 2. Identificazione di Minacce e Vulnerabilità	75
14.4.3 3. Analisi e Calcolo del Rischio	76
15 Controlli di Sicurezza	77
15.1 Classificazione dei Controlli di Sicurezza	77
15.1.1 Classificazione Funzionale	77
15.2 Selezione e Analisi Costi-Benefici	78
15.3 Monitoraggio e Gestione dei Cambiamenti	78
16 Sicurezza Fisica e delle Infrastrutture	79
16.1 Panoramica	79
16.2 Minacce Principali	79
16.3 Misure di Mitigazione e Prevenzione	79
17 Algoritmi di Crittografia	80
17.1 Principi Fondamentali e Strutture	80
17.2 Interno dell'AES (Advanced Encryption Standard)	80
17.3 Modalità Operative dei Cifrari a Blocchi	81
17.4 Distribuzione delle Chiavi (KDC)	81
18 Crittografia a Chiave Pubblica e Autenticazione del Messaggio	82

18.1	Funzioni di Hash Sicure	82
18.1.1	Funzioni di Hash Semplici	82
18.1.2	La Funzione di Hash Sicura SHA	83
18.1.3	Elaborazione di SHA-512	83
18.2	HMAC	83
18.3	Cifratura Autenticata (AE)	84
18.4	L'Algoritmo di Cifratura a Chiave Pubblica RSA	85
18.4.1	Descrizione dell'Algoritmo	85
18.4.2	Generazione delle Chiavi	85
18.4.3	La Sicurezza di RSA	86
18.5	Diffie-Hellman e Altri Algoritmi Asimmetrici	86
18.5.1	L'Algoritmo Diffie-Hellman	86
19	Sicurezza su Internet	87
19.1	Secure E-mail e S/MIME	87
19.1.1	S/MIME e Funzionalità	87
19.1.2	Funzionamento Generale	88
19.2	DomainKeys Identified Mail (DKIM)	88
19.2.1	Architettura della Posta su Internet	88
19.2.2	Strategia di DKIM	89
19.3	Secure Sockets Layer e Transport Layer Security	89
19.3.1	Architettura TLS	89
19.3.2	Protocolli TLS in Dettaglio	89
19.4	HTTPS	90
19.5	Sicurezza di IPv4 e IPv6 (IPsec)	90
19.5.1	Associazioni di Sicurezza (SA)	90
19.5.2	Encapsulating Security Payload (ESP)	90
20	Autenticazione su Internet	91
20.1	Kerberos	91
20.1.1	Il Protocollo Kerberos	91
20.1.2	Realm Kerberos e Kerberi Multipli	93
20.1.3	Versioni di Kerberos	93
20.2	X.509	93
20.2.1	Formato del Certificato	93
20.2.2	Revoca dei Certificati	94
20.3	Public-Key Infrastructure (PKI)	94

20.3.1	Modello di Trust e Problematiche Attuali	94
20.3.2	Public Key Infrastructure X.509 (PKIX)	95

Capitolo 1

Fondamenti di Sicurezza

1.1 Introduzione alla Sicurezza Informatica

Per sicurezza informatica si intendono una serie di misure e controlli che assicurano la confidenzialità, l'integrità e la disponibilità delle risorse del sistema informativo, compresi l'hardware, il software, il firmware e le informazioni elaborate, memorizzate e trasmesse.

1.1.1 La Triade CIA

I tre obiettivi chiave al centro della sicurezza informatica, noti anche come **triade CIA**, sono fondamentali per qualsiasi architettura sicura:

Confidentiality (Confidenzialità)

Ossia preservare le autorizzazioni riguardo l'accesso e la divulgazione delle informazioni. Comprende:

- **Confidenzialità dei dati:** assicura che le informazioni sensibili non siano rese disponibili a individui o processi non autorizzati.
- **Privacy:** assicura che gli individui controllino quali informazioni che li riguardano possano essere raccolte e conservate, nonché a chi possano essere divulgate.

Integrity (Integrità)

Ossia la protezione contro la modifica impropria o accidentale delle informazioni. Racchiude:

- **Integrità dei dati:** assicura che le informazioni e i programmi siano modificati solo in un modo specificato e autorizzato.
- **Integrità del sistema:** assicura che un sistema esegua il suo compito prestabilito in modo ineccepibile, privo di manipolazioni esterne.

Availability (Disponibilità)

Assicura che i sistemi lavorino prontamente e che il servizio non sia negato agli utenti autorizzati quando ne hanno bisogno.

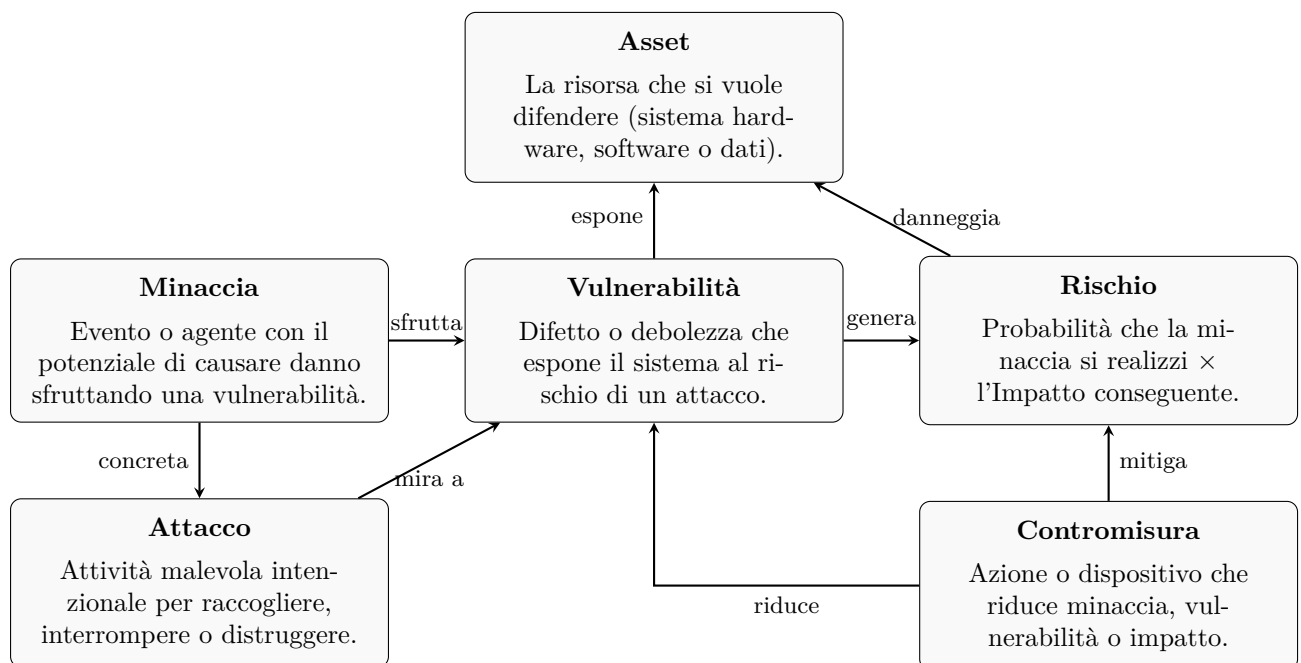
1.1.2 Livelli di Impatto delle Violazioni

La sicurezza informatica definisce tre livelli di impatto per valutare le conseguenze potenziali di una violazione, aiutando le organizzazioni a classificare il rischio:

- **Basso:** La perdita potrebbe avere un effetto limitato (es. efficacia ridotta, piccoli disagi).
- **Moderato:** La perdita potrebbe avere un serio effetto negativo (es. operatività svolta con grandi difficoltà, danni finanziari misurabili).
- **Alto:** La perdita potrebbe avere un effetto negativo grave o catastrofico (es. impossibilità di svolgere le funzioni primarie, danni d'immagine irreparabili o pesanti sanzioni legali).

1.2 Terminologia della Sicurezza

Comprendere le relazioni tra gli elementi chiave della sicurezza è fondamentale. Le vulnerabilità rappresentano delle falle, mentre le minacce sono gli agenti che cercano di sfruttarle.



1.2.1 Modelli di Controllo degli Accessi

Modelli di Accesso: RBAC e ABAC

- **RBAC (Role-Based Access Control):** Il controllo è basato sui *ruoli* (es. amministratore, utente base, revisore) a cui gli utenti vengono assegnati in base alle loro mansioni. Semplifica enormemente la gestione dei permessi nelle grandi organizzazioni, poiché i diritti sono associati al ruolo e non al singolo utente.
- **ABAC (Attribute-Based Access Control):** Modello molto più granulare e dinamico. I permessi vengono valutati in tempo reale in base agli *attributi* del soggetto (es. dipartimento), dell'oggetto (es. livello di classificazione) e dell'ambiente (es. orario di accesso, posizione geografica, tipo di rete).

1.3 Tipologie di Attacco e di Minacce

1.3.1 Classificazione degli Attacchi

Gli attacchi possono essere classificati secondo diverse dimensioni, principalmente in base all'interazione con il sistema:

Attacchi: Passivi vs Attivi

Attacco passivo Consiste nel tentativo di apprendere o utilizzare informazioni dal sistema senza alterarne le risorse o il funzionamento (es. intercettazione del traffico di rete, sniffing). L'obiettivo principale della difesa contro di essi è la **prevenzione**, tipicamente tramite la cifratura, in quanto sono difficili da rilevare.

Attacco attivo Comporta il tentativo esplicito di alterare le risorse del sistema o di influenzarne pesantemente il funzionamento (es. iniezione di codice, modifica di file, denial of service). L'obiettivo principale della difesa in questo caso è la **rilevazione** tempestiva e la pronta risposta.

Un'altra classificazione importante si basa sull'origine dell'attacco:

- **Attacco interno (Insider Threat):** Effettuato da un utente autorizzato ad accedere al sistema, ma che abusa dei propri privilegi per fini illeciti. Spesso è il più difficile da fermare poiché l'attaccante è già all'interno del perimetro difensivo.
- **Attacco esterno:** Effettuato da un *outsider*, ossia un soggetto non autorizzato che tenta di forzare le difese perimetrali dall'esterno.

1.3.2 Le Minacce Specifiche

Le minacce informatiche possono essere raggruppate in quattro categorie fondamentali a seconda dell'obiettivo e dell'effetto che producono sulle risorse del sistema.

1. Divulgazione Non Autorizzata

La divulgazione non autorizzata (*unauthorized disclosure*) è una minaccia diretta alla confidenzialità. I seguenti attacchi possono causarla:

- **Esposizione (exposure):** Può essere intenzionale o risultato di un errore umano/software, traducendosi nella conoscenza non autorizzata di dati sensibili.
- **Intercettazione (interception):** Ricezione di una copia di informazioni in transito destinate originariamente a un altro dispositivo (es. man-in-the-middle).
- **Inferenza (inference):** Nota anche come analisi del traffico; l'avversario ottiene informazioni preziose semplicemente osservando il pattern del traffico o incrociando dati apparentemente innocui.
- **Intrusione (intrusion):** Ottenimento di un accesso illegittimo a dati sensibili eludendo le protezioni del sistema.

2. Inganno

L'inganno (*deception*) è una minaccia che compromette l'integrità del sistema o dei dati:

- **Mascheramento (masquerade):** Tentativo di un utente non autorizzato di accedere al sistema fingendosi un utente autorizzato o un'entità legittima.
- **Falsificazione (falsification):** Alterazione fraudolenta o sostituzione di dati validi con dati falsi o manipolati.
- **Ripudio (repudiation):** Situazione in cui un utente nega falsamente di aver compiuto un'azione, come l'invio o la ricezione di un messaggio.

3. Interruzione

L'interruzione (*disruption*) mira ad abbattere la disponibilità o l'integrità del sistema:

- **Interdizione (incapacitation):** Attacco brutale alla disponibilità (es. danneggiamento fisico dell'hardware o disattivazione dei servizi tramite malware distruttivo).
- **Corruzione (corruption):** Attacco all'integrità in cui le risorse o i servizi vengono manipolati per funzionare in modo non previsto o errato.
- **Ostruzione (obstruction):** Interferenza con le comunicazioni, che avviene disabilitando i collegamenti di rete o sommergendo il sistema di traffico anomalo.

4. Usurpazione

L'usurpazione (*usurpation*) è una grave minaccia all'integrità del sistema stesso:

- **Appropriazione indebita (misappropriation):** Furto o dirottamento di un servizio, come ad esempio l'uso non autorizzato di un server aziendale come piattaforma per lanciare attacchi DDoS (Distributed Denial of Service) verso terzi o per minare criptovalute.
- **Uso improprio (misuse):** Disabilitazione o aggiramento attivo delle funzioni di sicurezza da parte di chi sfrutta falle o privilegi non dovuti, assumendo il controllo delle operazioni protette.

1.4 Valutazione delle Minacce

1.4.1 Superfici di Attacco

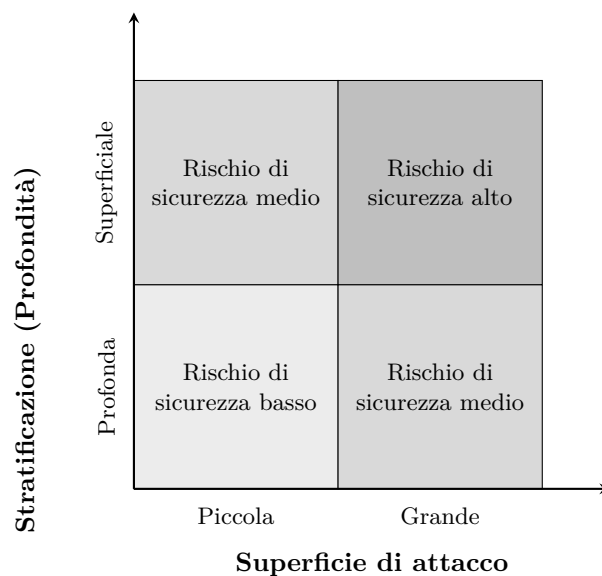
Una **superficie di attacco** rappresenta l'insieme totale di tutte le vulnerabilità raggiungibili e teoricamente sfruttabili all'interno di un sistema. Si divide comunemente in tre macro-aree:

- **Superficie di Rete:** Include porte aperte, protocolli di rete deboli, interfacce web, VPN e punti di accesso wireless.
- **Superficie Software:** Comprende le vulnerabilità insite nel codice delle applicazioni, nel sistema operativo e nelle librerie di terze parti utilizzate.
- **Superficie Umana:** Rappresenta le vulnerabilità legate alle persone, tra cui la suscettibilità ad attacchi di ingegneria sociale (es. phishing) o la propensione a commettere errori operativi.

Analisi della Superficie

L'**analisi della superficie di attacco** è una tecnica cruciale per valutare concretamente l'esposizione alle minacce. L'obiettivo ultimo di un'architettura sicura è quello di *ridurre tale superficie* il più possibile, chiudendo le porte inutilizzate e minimizzando i servizi in esecuzione, per rendere esponenzialmente più difficile il compito dell'avversario.

Il seguente grafico mostra il rapporto tra la dimensione e la profondità della superficie di attacco e il relativo rischio di sicurezza.



1.5 Minacce alle Risorse del Sistema

Le minacce possono colpire le diverse categorie di risorse di un sistema informatico, violando uno o più principi della triade CIA in modi specifici:

	Disponibilità	Confidenzialità	Integrità
Hardware	L'attrezzatura (server, router) viene rubata o disabilitata, impedendo l'erogazione del servizio.	Sottrazione di dispositivi (es. USB) non cifrati, esponendo i dati contenuti.	Dispositivi alterati fisicamente (es. keylogger hardware) o danneggiati.
Software	Cancellati o corrotti file eseguibili essenziali, bloccando l'avvio delle applicazioni.	Copia non autorizzata di software proprietario (furto di proprietà intellettuale).	Programma modificato per fallire o eseguire operazioni malevole nascoste (es. Trojan).
Dati	File essenziali cancellati o cifrati da ransomware, bloccando l'operatività.	Lettura, esfiltrazione non autorizzata o inferenza statistica sui dati esposti.	Modifica fraudolenta dei record in un database o iniezione di nuovi dati illeciti.

Tabella 1.1: Minacce alle risorse del sistema raggruppate in base alla triade CIA.

Capitolo 2

Crittografia e Funzioni Hash

2.1 Cifratura Simmetrica

2.1.1 I Cinque Componenti di uno Schema Simmetrico

Un sistema di cifratura simmetrica, nota anche come **cifratura convenzionale** o **a chiave singola**, si basa su cinque elementi fondamentali che interagiscono per proteggere l'informazione:

- **Testo in Chiaro (Plaintext):** Il messaggio o i dati originali, leggibili, che vengono forniti come input all'algoritmo.
- **Algoritmo di Cifratura:** L'algoritmo che esegue le sostituzioni e le trasformazioni matematiche sul testo in chiaro per renderlo incomprensibile.
- **Chiave Segreta (Secret Key):** Un valore segreto condiviso tra le parti, fornito anch'esso come input all'algoritmo. Le trasformazioni esatte eseguite dipendono interamente da questa chiave. Due chiavi diverse produrranno testi cifrati diversi.
- **Testo Cifrato (Ciphertext):** Il messaggio incomprensibile prodotto come output. Il suo valore dipende sia dal testo in chiaro sia dalla chiave segreta.
- **Algoritmo di Decifratura:** È essenzialmente l'algoritmo di cifratura eseguito in senso inverso. Prende in input il testo cifrato e la stessa chiave segreta per riprodurre il testo in chiaro originale.

2.1.2 Requisiti per un Utilizzo Sicuro

Per garantire che il sistema sia effettivamente impenetrabile, sono necessari due requisiti fondamentali:

1. **Un algoritmo di cifratura robusto:** L'algoritmo deve essere tale che un avversario, anche conoscendo l'algoritmo stesso e avendo accesso a testi cifrati (e persino a coppie plaintext-ciphertext), non possa decifrare i messaggi o risalire alla chiave.
2. **Gestione sicura della chiave:** Mittente e destinatario devono aver ottenuto la chiave in modo sicuro (tramite un canale fuori banda o altri metodi sicuri) e mantenerla segreta. Se un attaccante riesce a scoprire la chiave e conosce l'algoritmo, tutte le comunicazioni che la utilizzano sono compromesse.

2.1.3 Metodi di Attacco alla Cifratura Simmetrica

Attacchi alla Crittografia

- **Criptanalisi (Cryptanalysis):** Sfrutta le debolezze logiche e matematiche dell'algoritmo, oltre alle caratteristiche statistiche del testo in chiaro, per dedurre il messaggio o la chiave utilizzata.
- **Attacco a Forza Bruta (Brute-force):** Consiste nel provare sistematicamente ogni possibile chiave fino a ottenere un testo in chiaro di senso compiuto. La sua fattibilità dipende dalla lunghezza della chiave. In media, è necessario testare la metà di tutte le possibili chiavi per avere successo.

2.2 Analisi degli Algoritmi: DES, Triple DES e AES

Gli algoritmi di crittografia simmetrica più utilizzati sono i **cifrari a blocchi**, che elaborano il plaintext in blocchi di dimensione fissa, producendo blocchi di ciphertext di pari dimensione.

Algoritmi Simmetrici Storici e Moderni

- **DES (Data Encryption Standard):** È un cifrario a blocchi (64 bit) che usa una **chiave effettiva di 56 bit** tramite una Rete di Feistel (16 round). La sua chiave è vulnerabile agli attacchi a forza bruta, rendendolo obsoleto.
- **Triple DES (3DES):** Applica il DES tre volte in sequenza (**EDE - Encrypt-Decrypt-Encrypt**) usando due o tre chiavi distinte, portando la lunghezza effettiva a 112 o 168 bit. Questo rende impraticabile la forza bruta, ma essendo molto lento e computazionalmente pesante, è oggi deprecato in favore di algoritmi più moderni.
- **AES (Advanced Encryption Standard):** A differenza del DES, usa una **rete a sostituzione e permutazione (SPN)**. Opera su blocchi di **128 bit** e supporta chiavi di **128, 192 e 256 bit**, utilizzando rispettivamente 10, 12 o 14 cicli (round) di trasformazioni. Offre un livello di sicurezza estremamente alto combinato a grande efficienza sia hardware che software.

2.3 Modalità Operative e Cifrari

2.3.1 Il Problema della Modalità ECB (Electronic Codebook)

Nella cifratura di grandi quantità di dati si possono usare varie modalità operative. La modalità **ECB** cifra ogni blocco in modo indipendente.

Vulnerabilità della Modalità ECB

La semplicità di ECB causa una grave debolezza: blocchi di plaintext identici producono blocchi di ciphertext identici. Ciò permette a un attaccante di **sfruttare le regolarità** o i pattern nei dati crittografati (ad esempio, le sagome e i contorni in un'immagine cifrata rimangono perfettamente visibili). Per ovviare a ciò si utilizzano modi operativi più avanzati (es. CBC, GCM) che concatenano i blocchi, garantendo che lo stesso blocco di plaintext produca ciphertext diversi in posizioni differenti.

2.3.2 Cifrari a Blocchi vs Cifrari a Flusso

Caratteristica	Cifrario a Blocchi	Cifrario a Flusso
Elaborazione	Un intero blocco alla volta (es. 128 bit).	Flusso continuo (tipicamente un bit o un byte alla volta).
Meccanismo	Trasforma blocchi interi tramite permutazioni e sostituzioni.	Genera un flusso pseudocasuale (<i>keystream</i>) e ne fa lo XOR con il plaintext.
Velocità e Uso	Standard, usato per la gran parte dei dati a riposo.	Quasi sempre più veloci, ideali per stream in tempo reale.
Esempi	AES, 3DES.	RC4, ChaCha20.

2.4 Funzioni Hash e Digest

Una **funzione hash unidirezionale** è un algoritmo matematico che prende un input di dimensione variabile e restituisce una stringa di bit di lunghezza fissa, chiamata **digest** o impronta digitale del messaggio. Il digest svolge una funzione di controllo di integrità: se un singolo bit del messaggio originale cambia, il valore di hash cambia drasticamente.

2.4.1 Differenza Fondamentale: Hash vs Crittografia

Reversibilità vs Irreversibilità

- **Crittografia:** Processo a due vie, **reversibile**. Scopo: **Confidenzialità**. Il ciphertext può e deve essere riportato allo stato originale (decifatura).
- **Hashing:** Processo a una via, **irreversibile**. Scopo: **Integrità**. È computazionalmente impossibile ricostruire l'input originale dal digest. Assicura che un dato non sia stato manomesso durante il trasporto o l'archiviazione.

2.4.2 Requisiti delle Funzioni Hash

Per essere ritenuta crittograficamente sicura e utile all'autenticazione, una funzione hash H deve soddisfare proprietà rigorose:

1. **Dimensione Variabile e Fissa:** Accetta input di qualsiasi dimensione e produce output di lunghezza fissa.
2. **Facilità di Calcolo:** Il calcolo di $H(x)$ deve essere rapido e poco oneroso.
3. **Resistenza alla Preimmagine (One-Way):** Dato un hash h , deve essere computazionalmente impossibile trovare x tale che $H(x) = h$.
4. **Resistenza alla Seconda Preimmagine (Collisione Debole):** Dato uno specifico input x , deve essere impossibile trovare un $y \neq x$ che produca lo stesso hash ($H(x) = H(y)$).
5. **Resistenza alle Collisioni (Collisione Forte):** Deve essere computazionalmente impossibile trovare una *qualsiasi* coppia arbitraria (x, y) con $H(x) = H(y)$.

2.5 Autenticazione dei Messaggi

Mentre la crittografia protegge dagli attacchi passivi (intercettazione), l'**autenticazione dei messaggi** protegge dagli **attacchi attivi** (modifica, falsificazione). Un messaggio è considerato autentico se è *genuino*, se proviene dalla sua presunta fonte senza che l'identità del mittente sia falsificata, e se il contenuto non è stato alterato (integrità dei dati).

2.5.1 Il Limite della Sola Cifratura

Affidarsi unicamente alla cifratura simmetrica non garantisce in automatico l'integrità. Un attaccante potrebbe riordinare blocchi cifrati o ribaltare bit: il destinatario, pur decifrando correttamente i dati, otterrebbe un messaggio dal significato inaspettato (e potenzialmente malevolo) senza che il sistema segnali alcun errore.

2.5.2 Message Authentication Code (MAC)

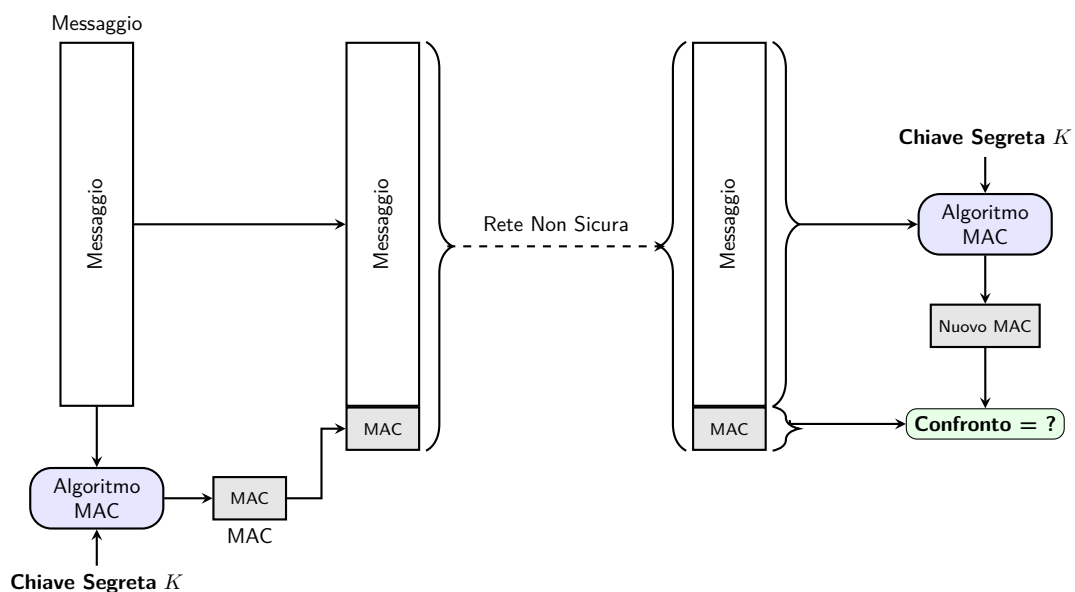
Un approccio diffuso consiste nell'autenticare senza necessariamente cifrare il messaggio. Viene generato e apposto un **tag di autenticazione**.

MAC - Message Authentication Code

Il MAC è un piccolo blocco di dati generato da un algoritmo crittografico utilizzando il messaggio e una **chiave segreta condivisa** (K_{AB}).

- Il mittente calcola il MAC per il messaggio e lo invia insieme ai dati in chiaro (garantendo autenticità e integrità, ma non confidenzialità).
- Il destinatario riceve il messaggio, ricalcola il MAC in modo indipendente usando la stessa chiave segreta e lo confronta con quello ricevuto.
- Se corrispondono, si è certi che i dati non sono stati manomessi e che solo chi possiede la chiave (il mittente legittimo) poteva generare quel MAC valido.

Se si desidera anche la **riservatezza**, è possibile combinare i due processi cifrando il messaggio insieme al suo tag di autenticazione (*Encrypt-then-MAC* o viceversa).



2.6 Crittografia a Chiave Pubblica (Asimmetrica)

Proposta nel 1976 da Diffie e Hellman. A differenza degli algoritmi simmetrici basati su veloci operazioni a livello di bit, gli algoritmi asimmetrici si basano su funzioni matematiche complesse.

La sua caratteristica fondante è l'uso di una **coppia di chiavi**: una pubblica e una privata.

2.6.1 Miti Comuni sulla Crittografia Asimmetrica

Miti da Sfatare

1. **"È più sicura della crittografia simmetrica"**: Falso. La sicurezza dipende dalla lunghezza della chiave e dal tipo di algoritmo.
2. **"Ha reso obsoleta la crittografia simmetrica"**: Falso. Gli algoritmi asimmetrici impongono un enorme carico computazionale e sono ordini di grandezza più lenti. Si usano comunemente in modo combinato: l'asimmetrica per scambiare in modo sicuro le chiavi, la simmetrica per cifrare grandi moli di dati.
3. **"La distribuzione delle chiavi è banale"**: Falso. La necessità di garantire che una chiave pubblica appartenga effettivamente a chi la dichiara richiede complesse infrastrutture basate su entità terze (PKI, Certification Authorities).

2.6.2 I Sei Componenti

- **Testo in Chiaro (Plaintext)**: Il messaggio originale.
- **Algoritmo di Cifratura**: Effettua le trasformazioni.
- **Chiave Pubblica (Public Key)**: Resa nota a tutti, usata generalmente per cifrare un messaggio destinato al proprietario o per verificare la sua firma.
- **Chiave Privata (Private Key)**: Conosciuta unicamente al proprietario, usata per decifrare i messaggi ricevuti o per firmare digitalmente.
- **Testo Cifrato (Ciphertext)**: Output prodotto. Se cifrato con la pubblica, si può decifrare solo con la corrispondente privata, e viceversa.
- **Algoritmo di Decifratura**: L'inverso del processo di cifratura.

2.6.3 Processi Fondamentali

La versatilità delle chiavi asimmetriche permette due grandi utilizzi operativi, a seconda di quale chiave venga usata per prima:

1. Confidenzialità (Cifratura)

Se Bob vuole inviare un messaggio **segreto** ad Alice:

- Bob usa la **chiave pubblica di Alice** per cifrare il messaggio.
- Solo Alice può leggerlo decifrandolo con la propria **chiave privata**.

Questo assicura la **riservatezza**, ma non l'autenticità: chiunque possiede la pubblica di Alice e avrebbe potuto inviarle quel messaggio.

2. Autenticazione (Firma Digitale)

Se Bob vuole inviare un messaggio dimostrandone la **provenienza**:

- Bob usa la propria **chiave privata** per cifrare (firmare) il messaggio o il suo hash.
- Chiunque riceva i dati può usare la **chiave pubblica di Bob** come verifica.

Questo garantisce **autenticità e non ripudio**, ma chiunque può leggere il contenuto, perdendo la confidenzialità.

2.6.4 La Busta Digitale

La **busta digitale** (Digital Envelope) è una tecnica che combina crittografia simmetrica e asimmetrica per migliorare sia le velocità che la sicurezza della distribuzione delle chiavi.

1. Bob prepara un messaggio lungo.
2. Bob genera una chiave simmetrica *random* usa e getta (chiave di sessione).
3. Bob cifra l'intero messaggio usando la chiave simmetrica (molto veloce).
4. Bob cifra la chiave simmetrica usando la **chiave pubblica di Alice**.
5. Bob invia ad Alice il messaggio cifrato insieme alla chiave cifrata (la "busta").
6. Solo Alice, usando la propria **chiave privata**, può aprire la busta, recuperare la chiave di sessione e decifrare istantaneamente il lungo messaggio.

2.6.5 Requisiti degli Algoritmi Asimmetrici

1. Facilità di generazione della coppia di chiavi.
2. Facilità nel calcolo di cifratura e decifratura.
3. Deve essere **computazionalmente impossibile** risalire alla chiave privata conoscendo quella pubblica.
4. Deve essere **computazionalmente impossibile** recuperare il messaggio originale conoscendo solo la chiave pubblica e il ciphertext.

2.6.6 Panoramica degli Algoritmi Asimmetrici

- **RSA (Rivest-Shamir-Adleman)**: L'algoritmo più diffuso. La sua robustezza si basa sulla difficoltà matematica di *fattorizzare grandi numeri primi*. È versatile: può essere usato sia per la cifratura dei dati sia per le firme digitali.
- **Diffie-Hellman Key Agreement**: Non cifra direttamente. È un protocollo matematico geniale che permette a due entità di negoziare in modo sicuro una chiave segreta condivisa attraverso un canale pubblico esposto, intercettato da terzi.
- **DSS / DSA (Digital Signature Standard)**: Progettato esclusivamente per la **firma digitale**, non fornisce capacità di cifratura dei messaggi.
- **ECC (Elliptic Curve Cryptography)**: Algoritmo avanzato basato sulla matematica delle curve ellittiche. Offre una sicurezza pari a RSA ma richiede **chiavi molto più corte** (es. 256 bit ECC offrono la stessa protezione di 3072 bit RSA), risultando incredibilmente rapido e ideale per IoT e smartphone.

2.7 Firme Digitali

La firma digitale garantisce tre requisiti vitali nelle comunicazioni formali:

- **Autenticità:** Conferma incontrovertibilmente l'identità del firmatario.
- **Integrità:** Assicura che nessun bit dei dati sia stato alterato post-firma.
- **Non Ripudio:** L'autore non può legalmente o tecnicamente negare di aver firmato quel documento, poiché è l'unico possessore della chiave privata utilizzata.

Processo di Firma Digitale e Verifica

Fase 1: La Firma (Mittente)

1. Il mittente passa il documento attraverso una funzione Hash, ottenendo il *Digest*.
2. Il mittente cifra il *Digest* usando la sua **Chiave Privata**. Questo digest cifrato costituisce la **Firma Digitale**.
3. Il documento e la firma vengono trasmessi al destinatario.

Fase 2: La Verifica (Destinatario)

1. Il destinatario usa lo stesso algoritmo Hash per calcolare in modo indipendente un nuovo Digest sul documento appena ricevuto.
2. Il destinatario utilizza la **Chiave Pubblica del Mittente** per decifrare la Firma, rivelando il Digest originale creato dal mittente.
3. Se i due Digest **combaciano perfettamente**, la firma è ritenuta valida.

2.8 Certificati a Chiave Pubblica

Se ricevo una chiave pubblica per posta elettronica da "Bob", come faccio a essere certo che appartenga davvero a lui e non a un attaccante che si finge Bob? La soluzione è affidarsi a un **Certificato Digitale**.

Un certificato lega crittograficamente un'identità (nome, email, organizzazione) alla sua chiave pubblica. È emesso e **firmato digitalmente** da un ente terzo fidato e autorevole chiamato **Certificate Authority (CA)**.

1. L'utente genera la sua coppia di chiavi e invia la chiave pubblica alla CA con i propri documenti d'identità.
2. La CA effettua le verifiche, crea il certificato e lo **firma con la propria chiave privata**.
3. Chiunque riceva il certificato può usare la chiave pubblica della CA (generalmente preinstallata nei sistemi operativi o browser) per verificarne la validità.

Lo standard universale che definisce la struttura di questi documenti è l'**X.509**, utilizzato in TLS, SSH e IPsec.

Capitolo 3

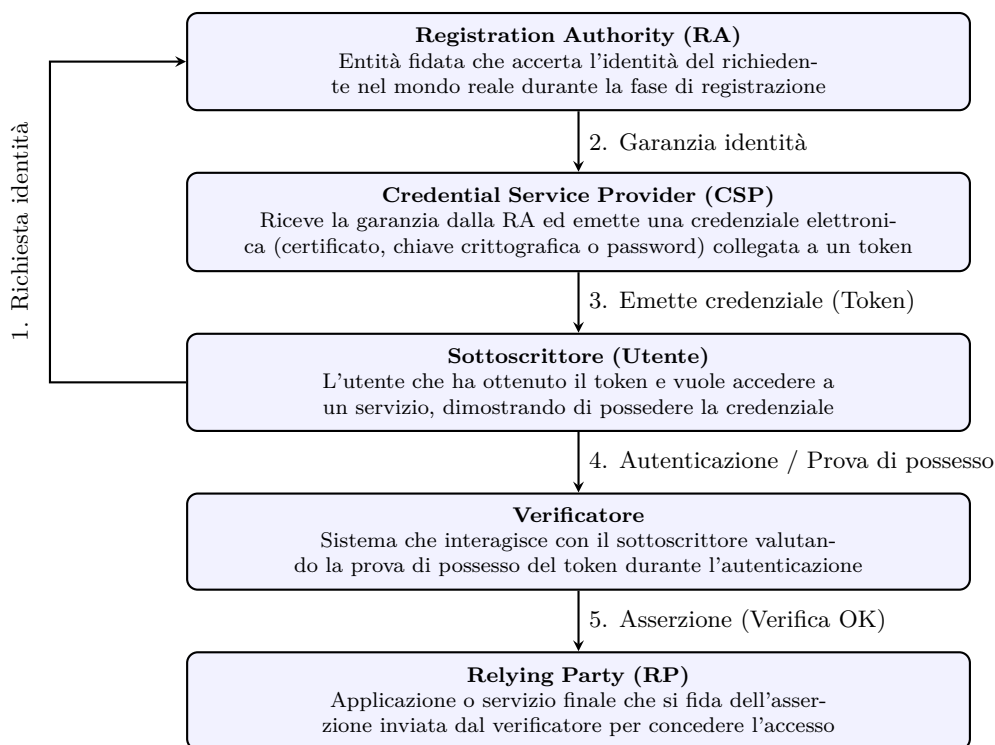
Autenticazione Digitale degli Utenti

3.1 Introduzione

L'autenticazione digitale rappresenta il processo fondamentale con cui si stabilisce la fiducia nelle identità presentate elettronicamente a un sistema informatico. Questa fiducia è cruciale per consentire a un sistema di determinare, in fase successiva, se l'individuo autenticato sia effettivamente autorizzato a compiere specifiche operazioni, come accedere a dati sensibili o modificare configurazioni del sistema.

3.2 Modello di Autenticazione Digitale

Il processo di gestione e verifica di un'identità digitale coinvolge diverse entità organizzative.



3.3 Mezzi di Autenticazione

L'identità di un utente può essere verificata e autenticata tramite quattro fattori o modalità principali:

1. **Qualcosa che l'utente conosce (Knowledge):** Come una password, un PIN o le risposte a domande di sicurezza specifiche.
2. **Qualcosa che l'utente possiede (Possession):** Come un dispositivo smartphone, una keycard elettronica, una smart card, un token hardware o una chiave fisica.
3. **Qualcosa che l'utente è (Inherence / Biometria statica):** Basato su caratteristiche fisiche uniche, come il riconoscimento delle impronte digitali, della geometria facciale, dell'iride o della retina.
4. **Qualcosa che l'utente fa (Action / Biometria dinamica):** Basato su caratteristiche comportamentali, come il riconoscimento della voce, le dinamiche della firma autografa o il ritmo di battitura sulla tastiera (*keystroke dynamics*).

Ciascun metodo presenta punti di forza e vulnerabilità (es. furto, falsificazione, falsi positivi biometrici). Inoltre, la gestione su larga scala di token hardware o dispositivi biometrici può risultare onerosa amministrativamente ed economicamente.

Autenticazione Multifattore (MFA)

L'**Autenticazione Multifattore (MFA)** richiede l'utilizzo combinato di *più di uno* dei metodi sopra elencati per concedere l'accesso (es. Password + Codice via SMS, ovvero *Conoscenza + Possesso*). La robustezza del sistema aumenta esponenzialmente: anche se un attaccante ruba la password, non potrà accedere senza possedere anche lo smartphone dell'utente.

3.4 Rischi nell'Autenticazione degli Utenti

La valutazione dei rischi di sicurezza per l'autenticazione considera tre elementi interconnessi per adattare dinamicamente le misure di sicurezza.

3.4.1 Livello di Garanzia (Level of Assurance - LoA)

Rappresenta il grado di affidabilità e certezza che il sistema ripone nell'identità dell'utente autenticato:

- **Livello 1:** Bassa fiducia. Usato dove i rischi sono minimi (es. registrazione in un forum o lettura di articoli gratuiti).
- **Livello 2:** Fiducia discreta. Adatto a interazioni generiche che richiedono un account verificato (es. e-commerce standard).
- **Livello 3:** Alta fiducia. Richiesto per l'accesso a dati sensibili o operazioni finanziarie. Impone l'autenticazione multifattoriale (MFA).
- **Livello 4:** Massima fiducia. Necessario per infrastrutture critiche o transazioni di altissimo valore. Richiede MFA con token hardware crittografici e verifica in presenza dell'identità.

3.4.2 Impatto Potenziale e Aree di Rischio

Il livello di impatto valuta le conseguenze di una compromissione dell'account. Es, se l'impatto finanziario o operativo è **Basso**, un LoA 1 è sufficiente. Se è **Moderato** (es. frode finanziaria limitata, danni all'immagine), si richiede un LoA 2 o 3. Se l'impatto è **Alto** (es. catastrofi finanziarie, blocco di ospedali o reti elettriche, lesioni letali), è tassativo operare a LoA 4.

3.5 Autenticazione Basata su Password

3.5.1 Vulnerabilità e Contromisure

Vulnerabilità dei sistemi a password

1. **Attacco offline (es. furto del database):** L'attaccante esfiltra il file degli hash e usa potenza di calcolo per craccarli offline. *Contromisure:* Hashing forte con Salt (es. Argon2, bcrypt), protezione rigorosa del file *shadow*.
2. **Brute-force o Dictionary Attack online:** Tentativi ripetuti di login su un account. *Contromisure:* Account lockout (blocco dopo N tentativi), ritardi progressivi, CAPTCHA.
3. **Password Spraying / Credential Stuffing:** Provare poche password molto comuni (es. "123456") su migliaia di account, o usare credenziali trapelate da altri siti. *Contromisure:* MFA, blacklist di password note o compromesse, monitoraggio degli IP anomali.
4. **Ingegneria sociale e Phishing:** L'utente viene ingannato a rivelare la password. *Contromisure:* MFA (soprattutto token hardware FIDO2 immuni al phishing), formazione aziendale.
5. **Riutilizzo delle password:** L'utente usa la stessa password ovunque. *Contromisure:* Integrazione con Password Manager, single sign-on (SSO) protetto da MFA.

3.6 Hash delle Password e Uso del Salt

Un sistema sicuro **non memorizza mai le password in chiaro**. Utilizza invece funzioni crittografiche per salvarne solo un derivato.

Hashing con Salt

Il **Salt** è una stringa o un numero casuale generato dal sistema nel momento in cui l'utente crea o cambia la propria password. Il processo funziona così:

1. La password scelta (P) viene concatenata al Salt univoco (S).
2. La stringa combinata ($P + S$) viene passata in una funzione di Hashing lenta e robusta.
3. Il risultato (l'*Hash*) e il *Salt* (che è pubblico) vengono salvati nel database.

Al login, il sistema recupera il Salt dell'utente, lo unisce alla password digitata, ne ricalcola l'Hash e verifica se combacia con quello salvato.

3.6.1 Perché il Salt è indispensabile?

- **Prevenzione dei duplicati:** Se Alice e Bob scelgono entrambi "Juventus123", i loro Salt saranno diversi, generando Hash completamente differenti nel database. Un amministratore malintenzionato non può notare l'uguaglianza.
- **Difesa contro le Rainbow Table:** Rende inefficaci le tabelle pre-calcolate di hash. Un attaccante dovrebbe pre-calcolare una tabella separata per ogni singolo possibile valore di Salt (miliardi di tabelle), il che è computazionalmente e spazialmente impossibile.
- **Aumento del costo di cracking offline:** L'attaccante che ruba il database è costretto a craccare le password *una alla volta*, rallentando drasticamente il processo.

3.7 Cracking delle Password e Approcci Moderni

L'evoluzione tecnologica ha cambiato le regole del gioco nel password cracking:

- **Aumento della potenza di calcolo (GPU e ASIC):** A differenza delle normali CPU, le moderne schede video (GPU) possono parallelizzare le operazioni di hashing, permettendo di testare decine di miliardi di combinazioni al secondo.
- **Modelli Probabilistici e IA:** Strumenti moderni (es. Hashcat, John the Ripper) non fanno solo forza bruta stupida, ma usano regole di mascheramento e apprendimento automatico per dedurre come gli utenti creano le password (es. maiuscola iniziale, parola, anno finale "Napoli2023!"), testando prima le combinazioni statisticamente più probabili.

3.7.1 Migliorare la Selezione della Password

Per proteggersi da hardware sempre più potente, le contromisure lato utente e sistema includono:

- **Passphrase:** Sostituire le password complesse ma corte (es. "k9#X1z!") con frasi lunghe ma facili da ricordare (es. "cavallo-batteria-graffetta-corretta"). La lunghezza estrema rende il cracking matematicamente infattibile, pur migliorando l'usabilità.
- **Controllo reattivo e proattivo:** Impedire l'uso di password presenti in enormi dizionari di violazioni note (es. servizio *Have I Been Pwned*).

3.8 Autenticazione Basata su Token

I token trasferiscono la sicurezza dal dominio mentale (memoria) al dominio fisico.

3.8.1 Memory Card

Le memory card (es. carte magnetiche, vecchi badge aziendali) immagazzinano dati statici, come un codice identificativo, ma non hanno capacità di calcolo.

- **Vantaggi:** Combinate a un PIN, forniscono una base di sicurezza a due fattori economica.
- **Svantaggi:** Sono facilmente clonabili. Se un attaccante copia la banda magnetica e ruba il PIN (es. tramite uno *skimmer* o una telecamera), il sistema è compromesso.

3.8.2 Smart Card e Token Crittografici

Le smart card e i token USB crittografici rappresentano un salto qualitativo:

- **Capacità computazionale:** Includono un vero e proprio microprocessore interno in grado di eseguire calcoli crittografici complessi in isolamento.
- **Sicurezza delle chiavi:** La chiave privata inserita nel token non è esportabile. Il chip firma le richieste internamente. Nessun malware sul computer può rubare la chiave.
- **Funzionalità dinamiche:** Usano protocolli *Challenge-Response*. Il server invia una sfida casuale (challenge), la smart card la cifra con la sua chiave e invia la risposta. Se l'attaccante intercetta la comunicazione, non potrà riutilizzarla (*replay attack*) perché la sfida successiva sarà diversa.
- **Applicazioni avanzate:** Supportano la Firma Digitale qualificata (eSign), l'identità nazionale (CIE, ePass) e l'autenticazione forte aziendale.

3.9 Autenticazione Biometrica

L'autenticazione biometrica eleva la sicurezza legando l'accesso direttamente alla fisiologia o al comportamento inscindibile dell'utente.

3.9.1 Tipologie di Caratteristiche Biometriche

Riconoscimento Facciale Identifica la geometria e le proporzioni del volto. Sistemi avanzati usano proiettori di punti 3D (es. FaceID) o termocamere a infrarossi per garantire il rilevamento della "vitalità" (*liveness detection*), impedendo l'uso di foto, video o maschere in silicone.

Impronte Digitali Analizza i pattern di creste, solchi e minuzie del polpastrello, estraendo un *template* numerico univoco (non viene salvata l'immagine dell'impronta vera e propria).

Geometria della Mano Misura forma, lunghezza e larghezza delle dita. È meno accurata ma veloce e storicamente diffusa per il controllo degli accessi fisici in ambito industriale.

Schema della Retina Analizza il pattern univoco dei vasi sanguigni sul fondo dell'occhio. Offre una sicurezza militare ma è altamente intrusiva e richiede l'avvicinamento dell'utente a uno scanner specifico.

Scansione dell'Iride Analizza i complessi intrecci visibili nella parte colorata dell'occhio. Ha tassi di accuratezza eccellenti ed è meno invasiva della retina, permettendo letture anche a distanza.

Dinamica della Firma Non controlla *come appare* la firma, ma *come viene eseguita*. Analizza accelerazione, velocità e pressione della penna su un tablet grafico. È un tratto comportamentale quasi impossibile da falsificare per un falsario tradizionale.

Riconoscimento Vocale Analizza il timbro e le risonanze delle corde vocali. Tuttavia, è suscettibile a variazioni nel tempo (malattie, stress) e ai recenti e sofisticati attacchi basati su *Deepfake* vocali e intelligenza artificiale.

Capitolo 4

Controllo degli Accessi

4.1 Introduzione

L'obiettivo principale del controllo degli accessi è quello di regolamentare rigorosamente **chi** può accedere alle risorse, **in che modo** e in **quali circostanze**.

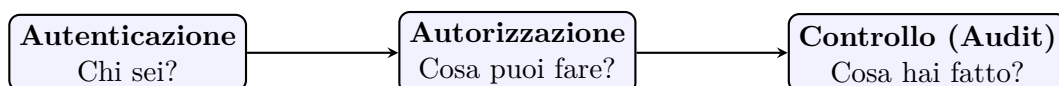
4.1.1 Obiettivi Principali

Gli obiettivi della sicurezza informatica nel controllo degli accessi si concentrano su tre aspetti cardine:

1. **Impedire l'accesso non autorizzato:** proteggere i dati sensibili da utenti o processi esterni senza permessi.
2. **Prevenire accessi impropri da utenti legittimi:** evitare che utenti autorizzati eseguano operazioni non permesse dal loro ruolo (es. un dipendente che legge lo stipendio dei colleghi).
3. **Consentire l'accesso autorizzato:** garantire che gli utenti legittimi possano operare sulle risorse in modo sicuro e senza inutili ostacoli.

4.1.2 Contesto Operativo

Il controllo degli accessi non è un blocco isolato, ma opera all'interno di un flusso logico sequenziale:



Autenticazione Processo di verifica delle credenziali di un utente per confermarne l'identità.

Autorizzazione (Controllo degli Accessi) Concessione effettiva dei diritti e dei permessi a un'entità per l'uso delle risorse di sistema.

Controllo (Audit) Registrazione e revisione a posteriori dei file di log per assicurare la conformità alle policy e individuare le violazioni.

4.2 Modelli e Politiche di Controllo degli Accessi

Le politiche definiscono architetture e logiche con cui i permessi vengono assegnati. Si dividono storicamente in quattro categorie principali, non esclusive, ma che possono essere combinate.

1. Controllo degli Accessi Discrezionale (DAC)

Il DAC (*Discretionary Access Control*) basa i permessi sull'identità del richiedente. È definito "discrezionale" perché il **proprietario** della risorsa ha la discrezione assoluta di trasferire i propri privilegi o concedere l'accesso ad altri utenti. È il modello classico usato dalla maggior parte dei sistemi operativi (es. permessi file in Linux/Windows).

Per implementare il DAC si utilizza la **Matrice degli Accessi**, una struttura dati bidimensionale in cui le righe sono i *Soggetti* (utenti) e le colonne gli *Oggetti* (file). L'intersezione contiene i permessi (Lettura, Scrittura, Esecuzione). Poiché la matrice è tipicamente enorme e molto sparsa (piena di celle vuote), viene implementata in due modi:

- **Access Control List (ACL):** Si scompone la matrice *per colonna*. Ogni oggetto ha una lista degli utenti e dei relativi permessi. Molto efficiente per sapere "chi può accedere a questo file?", ma lento per sapere "a quali file ha accesso questo utente?".
- **Capability Ticket:** Si scompone la matrice *per riga*. Ogni utente ha un proprio "biglietto" o portachiavi che elenca tutte le risorse a cui ha accesso.

2. Controllo degli Accessi Obbligatorio (MAC)

Nel MAC (*Mandatory Access Control*), la politica di accesso è determinata rigidamente dal sistema e l'utente non ha alcuna discrezionalità. Si basa su:

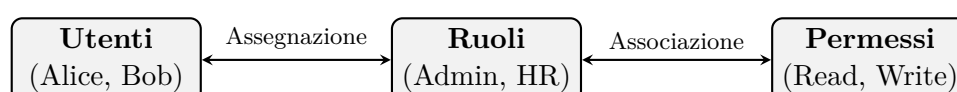
- **Etichette di sensibilità** associate agli oggetti (es. Non Classificato, Segreto, Top Secret).
- **Livelli di autorizzazione** (Clearance) assegnati ai soggetti.

Il sistema confronta i due valori e decide se consentire l'accesso. È tipico di ambienti militari e governativi ad alta sicurezza.

3. Controllo degli Accessi Basato sui Ruoli (RBAC)

L'RBAC (*Role-Based Access Control*) disaccoppia utenti e permessi introducendo l'entità del **Ruolo**. Un ruolo rappresenta una specifica funzione lavorativa (es. Medico, Infermiere, Amministratore). I permessi vengono assegnati *ai ruoli*, e gli utenti vengono assegnati *ai ruoli*. Questo modello:

- Semplifica drasticamente la gestione del turnover (se un dipendente cambia mansione, basta cambiare il suo ruolo, non mille permessi singoli).
- Facilita l'applicazione rigorosa del principio del **Minimo Privilegio**.



4. Controllo degli Accessi Basato sugli Attributi (ABAC)

L'ABAC (*Attribute-Based Access Control*) è il modello moderno più dinamico e granulare. Autorizza l'accesso valutando in tempo reale un set di attributi booleani legati a:

- **Soggetto:** es. Età, qualifica, dipartimento.
- **Risorsa:** es. Data di creazione, classificazione del documento.
- **Ambiente:** es. Orario d'ufficio, geolocalizzazione, tipo di rete (VPN vs Wi-Fi pubblico).

Permette policy del tipo: *"I Medici possono leggere le cartelle cliniche (Soggetto/Oggetto) solo dall'IP dell'ospedale (Ambiente) e solo durante i turni lavorativi (Ambiente)".*

4.3 Gestione delle Identità, Credenziali e Accessi (ICAM)

L'ICAM (*Identity, Credential, and Access Management*) rappresenta l'infrastruttura completa e standardizzata per gestire il ciclo di vita delle identità digitali in aziende e istituzioni. Questo sistema permette di creare identità affidabili per persone e per **NPE** (*Non-Person Entities*, es. server, API, dispositivi IoT).

Si divide in tre pilastri fondamentali:

4.3.1 1. Gestione delle Identità

Si occupa della creazione iniziale e del ciclo di vita dell'identità digitale. Inizia con il *provisioning* (raccolta dei dati e approvazione), la definizione degli attributi per identificare univocamente il soggetto, fino ad arrivare alla dismissione e revoca quando il soggetto abbandona l'organizzazione.

4.3.2 2. Gestione delle Credenziali

Una credenziale è l'oggetto crittografico che lega indissolubilmente l'identità a un token fisico o logico (smart card, certificato X.509). Il ciclo prevede:

1. **Approvazione:** Verifica della necessità della credenziale.
2. **Iscrizione:** L'utente dimostra la propria identità, fornendo dati biometrici o documenti ufficiali.
3. **Generazione e Distribuzione:** Creazione tecnica del token e consegna sicura.
4. **Manutenzione:** Procedure in caso di smarrimento, scadenza o revoca anticipata.

4.3.3 3. Gestione degli Accessi

Il processo operativo che utilizza le credenziali per autorizzare l'ingresso. Si appoggia su:

- **Gestione delle risorse:** Classifica le risorse e definisce le regole di protezione.
- **Gestione dei privilegi:** Assegna autorizzazioni agli utenti in base al loro profilo.
- **Gestione delle politiche (Policy):** Motore logico che valuta in tempo reale se concedere l'accesso in base a regole di contesto (tipico dell'ABAC).

Federazione delle Identità

La **federazione** permette a organizzazioni distinte di instaurare una fiducia reciproca per riconoscere le credenziali emesse dalle altre parti (es. accedere ai servizi della Pubblica Amministrazione usando l'identità SPID rilasciata da aziende private o accedere a servizi universitari tramite *Eduroam* / *IDEM*).

4.4 Trust Framework (OITF)

Nell'ambito dell'e-commerce, della PA digitale e delle transazioni online globali, condividere attributi di identità è complesso. Deve vigere il principio del **need-to-know**: se un servizio deve solo verificare che l'utente sia maggiorenne, non deve ricevere il nome, l'indirizzo o l'età esatta, ma solo un flag booleano.

Per standardizzare questa fiducia su scala globale, è nato l'**Open Identity Trust Framework (OITF)**. Esso fornisce un sistema aperto per lo scambio di identità e attributi, basato su standard come **OpenID**.

4.4.1 Componenti e Ruoli del Framework OITF

Il trust framework opera come un rigido programma di certificazione e responsabilità legale, distribuendo il carico su più attori:

- **Soggetti**: Gli utenti finali (clienti, cittadini).
- **Fornitori di Identità (IDP - Identity Provider)**: Organizzazioni che registrano, autenticano l'utente e generano i token di asserzione.
- **Relying Parties (RP - Service Provider)**: Applicazioni o siti web che offrono il servizio finale e si fidano delle affermazioni dell'IDP.
- **Fornitori di Attributi (AP)**: Entità specializzate che certificano specifici tratti dell'utente (es. l'Università certifica che l'utente è uno studente, la Banca certifica l'affidabilità creditizia).

L'OITF prevede inoltre figure di governance come **Periti e Revisori** (che certificano la conformità tecnica e legale degli IDP e dei RP) e **Risolutori di dispute** (arbitri legali in caso di frodi o violazioni delle policy condivise all'interno della rete di fiducia AXN - Attribute Exchange Network).

Capitolo 5

SQL Injection e Sicurezza dei Database

5.1 SQL Injection

Cos'è una SQL Injection?

Le **SQL Injection (SQLi)** sono una gravissima classe di vulnerabilità informatiche in cui un attaccante inserisce o manipola parti di codice SQL (iniettando comandi) all'interno degli input di un'applicazione web. L'obiettivo è ingannare il database di back-end facendogli eseguire istruzioni non previste dal programmatore, al fine di estrarre, alterare o distruggere informazioni preziose.

5.1.1 Come Funzionano le SQLi e Fonti di Attacco

L'attacco si concretizza quando un'applicazione web prende i dati forniti dall'utente e li concatena direttamente (senza alcuna validazione o sanificazione) all'interno di una stringa che compone una query SQL.

È possibile classificare gli attacchi in base al **fronte o vettore di attacco**:

- **Input utente standard:** L'attaccante inietta i comandi SQL nei normali campi dei moduli web (es. form di login, barre di ricerca, campi URL GET).
- **Cookie e Header HTTP:** I cookie sono spesso usati dalle applicazioni per recuperare informazioni sullo stato o sui permessi dell'utente. Se il server effettua query basate sul contenuto dei cookie senza validarle, l'attaccante può alterarli localmente per dirottare la query.
- **Input fisico (Out-of-band sources):** I comandi SQL possono essere iniettati al di fuori del contesto web classico, ad esempio stampando payload SQL su codici a barre o moduli cartacei, che, una volta digitalizzati tramite scanner e OCR e inviati al DBMS, causano l'iniezione.

5.2 Tipi di SQL Injection

Gli attacchi SQLi possono essere raggruppati in tre macro-categorie in base al **canale** utilizzato per estrarre i dati.

5.2.1 1. Attacchi In Banda (In-Band SQLi)

Un attacco in banda è il più comune e facile da sfruttare. Sfrutta **lo stesso canale di comunicazione** sia per iniettare il codice maligno sia per visualizzare i risultati. I dati estratti vengono mostrati direttamente nella pagina web dell'applicazione.

Tautologia

Inietta codice nel *WHERE* affinché la query sia valutata come **sempre vera**.

Query originale prevista dal programmatore:

```
1 SELECT * FROM users WHERE username = '$username';
```

L'attaccante inserisce nel campo di login: ' OR '1'='1

```
1 SELECT * FROM users WHERE username = '' OR '1'='1';
```

Poiché '1'='1' è sempre vero, il DB ignora il fallimento del controllo sull'username e restituisce indiscriminatamente **tutti gli utenti** registrati nella tabella.

Commenti di Fine Riga

L'attaccante inietta il suo codice utile e poi aggiunge il marcatore di commento (es. - o #). Tutti i controlli legittimi scritti dal programmatore dopo quell'input vengono letteralmente ignorati dal DB.

Query originale:

```
1 SELECT * FROM users WHERE username = '$username' AND password = '$pwd';
```

L'attaccante inserisce come username: admin'

```
1 SELECT * FROM users WHERE username = 'admin' -- ' AND password = '';
```

L'intera verifica della password (tutto ciò che segue - -) diventa un commento inutile. L'attaccante entra nell'account admin senza possederne la password.

Query Piggybacked (Aggiuntive)

L'attaccante sfrutta il carattere terminatore di istruzione (il punto e virgola ;) per chiudere la query legittima e **accodare una seconda query letale**.

Query originale:

```
1 SELECT * FROM users WHERE username = '$username';
```

L'attaccante inserisce: '; DROP TABLE users;

```
1 SELECT * FROM users WHERE username = ''; DROP TABLE users; --';
```

Il DB esegue la prima select (a vuoto) e poi, come seconda istruzione autonoma, esegue il comando DROP TABLE, cancellando l'intero database utenti.

5.2.2 2. Attacchi Inferenziali (Inferential SQLi)

In un attacco inferenziale non c'è un trasferimento diretto e visibile dei dati. L'attaccante deve **dedurre (inferire)** le informazioni formulando centinaia o migliaia di richieste specifiche e osservando come cambia il comportamento del server.

- **Error-based SQL Injection:** L'attaccante inietta sintassi errata o funzioni logiche impossibili. Il database va in errore e il server web (se non configurato correttamente) mostra la traccia dell'errore (Stack Trace) sullo schermo, rivelando all'attaccante il tipo di database (es. MySQL, Oracle), la versione e i nomi esatti delle tabelle o colonne nascoste.
- **Blind SQL Injection (Iniezione Cieca):** L'attaccante fa **domande booleane** (es. "L'ID 1 ha la password che inizia con 'A'?"). Se la pagina web carica normalmente, la risposta è "Vero". Se carica senza una specifica immagine o impiega un secondo in più a caricare (Time-based Blind SQLi), la risposta è "Falso". Procedendo carattere per carattere, l'attaccante estrae l'intero database.

5.2.3 3. Attacchi Fuori Banda (Out-of-Band SQLi)

L'attaccante inietta il payload, ma i risultati vengono esfiltrati attraverso un **canale di comunicazione differente**. Avviene quando il server web è ben difeso in uscita verso l'attaccante, ma il server database retrostante ha il permesso di fare richieste HTTP verso l'esterno. L'attaccante ordina al DB di inviare un file contenente le password verso un server controllato dall'hacker.

5.3 Contromisure per le SQL Injection

5.3.1 1. Programmazione Difensiva

È la contromisura assoluta e risolutiva, che deve essere implementata direttamente nel codice sorgente:

- **Validazione rigorosa dell'Input (Sanitization):** Controllare i tipi. Se il campo "età" deve essere un intero, il server deve respingere qualsiasi input contenente lettere o simboli speciali (come virgolette e apici) usando RegEx.
- **Inserimento Parametrizzato (Prepared Statements):** È la panacea tecnica per fermare le SQLi. Lo sviluppatore definisce la struttura della query e lascia "buchi" per i parametri. Il database compila la struttura in anticipo. Quando arriva l'input dell'utente, questo viene trattato strettamente come **valore testuale o numerico**, e mai fuso con il codice eseguibile. Anche se l'utente inietta `DROP TABLE`, il DB cercherà letteralmente un utente che si chiama "DROP TABLE".
- **Uso di ORM (SQL DOM):** Uso di framework (es. Hibernate, Entity Framework) che mappano le entità a oggetti e incapsulano la creazione automatica delle query, garantendo validazione e sanitizzazione native.

5.3.2 2. Rilevamento (Detection)

Agisce per individuare attacchi in corso analizzando il traffico di rete o i log (es. tramite un Web Application Firewall - WAF):

- **Signature-based (Sulle firme):** Il firewall confronta il traffico HTTP in entrata con un dizionario di pattern SQL noti (es. la stringa `UNION SELECT`). Veloce ma aggirabile da payload offuscati.
- **Anomaly-based (Sulle anomalie):** Usa il Machine Learning. Il sistema impara come appare una query HTTP "normale" durante una fase di addestramento. Qualsiasi richiesta che devia pesantemente dalla norma viene bloccata.
- **Code Analysis (Analisi dinamica e statica):** Scanner di vulnerabilità automatici che analizzano il codice dell'app o fanno finti attacchi in ambiente di test per scovare le vulnerabilità prima del rilascio.

5.3.3 3. Prevenzione a Tempo di Esecuzione (Runtime)

L'applicazione auto-monitora le query generate a runtime, confrontando l'albero sintattico (*parse tree*) della query che sta per essere eseguita con un modello astratto previsto dal programmatore. Se la logica della query muta, l'esecuzione viene abortita.

5.4 Controllo degli Accessi ai Database (Amministrazione)

Oltre a difendere l'applicazione, è necessario blindare il database stesso sfruttando i permessi granulari del DBMS (come MySQL, PostgreSQL, Oracle). L'amministrazione dei permessi può essere:

- **Centralizzata:** Solo pochi amministratori supremi (DBA) gestiscono tutti i permessi.
- **Basata sulla proprietà (Ownership):** Chi crea una tabella ne è il padrone assoluto e può concederne la lettura/scrittura ad altri.
- **Decentralizzata:** Il proprietario di un oggetto può permettere a terzi di gestire i permessi di quell'oggetto per conto suo.

Autorizzazioni in Cascata

La clausola `WITH GRANT OPTION` permette a `utente_hr` di girare i propri permessi ad altri colleghi. Tuttavia, se l'amministratore esegue il `REVOKE` su `utente_hr`, il sistema innesca una **revoca a cascata**, cancellando automaticamente tutti i permessi derivati che `utente_hr` aveva a sua volta concesso.

5.4.1 Ruoli nel Database (RBAC)

I DB moderni non assegnano permessi a singoli utenti, ma a **Ruoli**. Esistono ruoli a livello server (es. `sysadmin`, `diskadmin`), a livello di singolo database (es. `db_datareader`) e i **Ruoli Applicativi**. Questi ultimi sono assegnati all'applicazione web stessa e richiedono una password: l'utente accede all'app, e l'app assume un ruolo ristretto per parlare col DB, garantendo il principio del minimo privilegio.

5.5 Inferenza nei Database e Cifratura

L'Inferenza (Inference)

L'inferenza è un attacco logico e concettuale. Avviene quando un utente, pur facendo esclusivamente query legittime e perfettamente autorizzate, **incrocia e deduce matematicamente** informazioni sensibili a cui non dovrebbe avere accesso. Es. Un utente non può leggere lo stipendio di Alice. Ma ha il permesso di chiedere la "Somma degli stipendi dell'azienda" e poi, il giorno che Alice si licenzia, ricalcolare la "Somma". Sottraendo i due valori, deduce lo stipendio di Alice.

5.5.1 Contromisure contro l'Inferenza

- **In fase di progettazione (Design):** Modificare lo schema relazionale del DB per eliminare le dipendenze o introdurre perturbazioni casuali (rumore) nei dati statistici.
- **A tempo di esecuzione (Runtime):** Un modulo di sicurezza memorizza la cronologia delle query dell'utente. Se si accorge che l'utente sta cercando di "chiudere il cerchio" per un'inferenza, il DB rifiuta o altera l'ultima query.

5.5.2 Cifratura dei Database

Quando il controllo degli accessi e i firewall falliscono (es. un ladro ruba l'hard disk fisico del server), l'ultima linea di difesa è la **cifratura a riposo**.

- **I livelli:** Può essere applicata all'intero database (TDE - Transparent Data Encryption), alla singola tabella, colonna o persino al singolo record/cella.
- **Il problema della Rigidità:** Cifrare una colonna (es. i Cognomi) distrugge la capacità del DB di creare indici, ordinare alfabeticamente o fare ricerche sfocate (*LIKE '%Rossi%'*), impattando drasticamente sulle prestazioni applicative.
- **La gestione delle chiavi:** Dove nascondere la chiave di decifratura? Se è salvata nello stesso server del DB, chi ruba il disco ruba anche la chiave. Se è su un server separato, aggiunge latenza di rete e punti di fallimento strutturali (KMS - Key Management System).

Capitolo 6

Malware

6.1 Introduzione

Malware

Il malware (malicious software) è un programma inserito in un sistema con l'intento di comprometterne la confidenzialità, l'integrità o la disponibilità dei dati.

6.1.1 Classificazione e Propagazione

Il malware può essere classificato in due grandi categorie: in base ai **meccanismi di propagazione** e alle **azioni eseguite** (o payload) una volta raggiunto il bersaglio.

Meccanismi di propagazione

La prima categoria di propagazione del malware riguarda frammenti di *software parassita* che si attaccano a contenuti eseguibili esistenti. Il frammento può essere:

- Codice macchina che infetta un'applicazione, un programma di utilità o il codice utilizzato per l'avvio del sistema.
- Codice di scripting, utilizzato in file di dati come documenti Word, fogli di calcolo Excel o documenti PDF.

6.2 Tipi di Malware

Il malware comprende una vasta e variegata serie di software dannosi progettati per compromettere la sicurezza, la privacy o l'integrità dei sistemi informatici. Di seguito sono presentati i principali tipi di malware, classificati per caratteristiche e modalità di azione.

Advanced Persistent Threat (APT)

Un APT è una minaccia avanzata e persistente che utilizza diverse forme di malware per attaccare un obiettivo specifico in modo continuo e discreto per un lungo periodo. Spesso orchestrati da gruppi di hacker organizzati (o nazioni), mirano a ottenere l'accesso prolungato a una rete per rubare dati sensibili o sabotarne le operazioni.

Tipologie Comuni di Malware

Adware Software che visualizza o scarica pubblicità indesiderata (pop-up, reindirizzamenti). Oltre a compromettere l'esperienza utente, può raccogliere informazioni di navigazione ed essere difficile da rimuovere.

Exploit Kit Strumenti che automatizzano la creazione e la distribuzione di nuovo malware sfruttando vulnerabilità software note. Offrono meccanismi di propagazione sofisticati. Esempi noti: *Zeus*, *Blackhole*, *Sakura* e *Phoenix*.

Backdoor Meccanismi che consentono di aggirare i normali controlli di sicurezza, garantendo un accesso non autorizzato a un sistema o a un'applicazione.

Exploits Frammenti di codice progettati per sfruttare specifiche vulnerabilità nei software per eseguire codice non autorizzato.

Flooders (DoS Client) Strumenti utilizzati per generare un grande volume di traffico al fine di inondare e sovraccaricare un sistema (attacchi DoS).

Keyloggers Malware che monitorano e registrano ogni tasto premuto, fornendo agli attaccanti password o numeri di carta di credito.

Logic Bomb Malware programmati per rimanere inattivi fino al verificarsi di una condizione predefinita (es. una data). Una volta attivate, eseguono azioni dannose.

Macro Virus Virus che utilizzano codice o script incorporati in documenti (es. Word o Excel), attivandosi all'apertura del file.

Mobile Code Script o software trasferibili ed eseguibili su diverse piattaforme senza modifiche (es. JavaScript, ActiveX).

Rootkit Strumenti progettati per ottenere e mantenere il controllo root (amministratore) su un sistema senza essere rilevati.

Spyware Raccoglie informazioni dal computer dell'utente a sua insaputa (digitazione, dati a schermo, file di sistema) e le trasmette a terzi.

Trojan Programmi che appaiono utili ma nascondono funzioni dannose. Eludono la sicurezza sfruttando autorizzazioni legittime per poi installare altro malware.

Virus Malware che tenta di replicarsi inserendosi in altri file o programmi eseguibili.

Worm Programma che si replica autonomamente e si diffonde in rete sfruttando vulnerabilità software.

Zombie / Bot Programma che, una volta installato, può essere utilizzato remotamente per lanciare attacchi coordinati (spesso in una *botnet*).

6.3 Virus

Un virus informatico è un software che infetta altri programmi modificandoli con l'iniezione di codice che gli consente di replicarsi. Ogni volta che il computer infetto entra in contatto con codice non infetto, il virus si trasferisce diffondendosi da macchina a macchina. La diffusione può avvenire tramite supporti fisici (chiavette USB) o tramite reti, sfruttando documenti o servizi condivisi.

6.3.1 Struttura Generale e Fasi

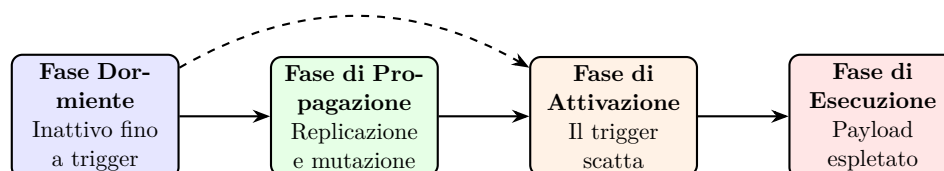
Struttura di un Virus

Un virus è costituito generalmente da tre parti:

1. **Meccanismo di infezione** (Vettore di infezione): il mezzo con cui il virus si propaga.
2. **Trigger** (Bomba logica): l'evento che determina l'attivazione o la consegna del payload.
3. **Payload**: ciò che il virus fa effettivamente oltre a diffondersi (es. danni al sistema o attività benigne ma fastidiose).

Una volta avviato, il virus cerca file eseguibili non infetti per replicarsi, dopodiché esegue il suo payload se le condizioni del trigger sono soddisfatte, per poi restituire il controllo al programma originale.

Spesso un virus comprime il file eseguibile infetto per mascherare l'aumento di dimensione dovuto all'iniezione di codice, rimanendo nascosto durante l'esecuzione del programma.



Un virus tipicamente attraversa quattro fasi:

1. **Fase dormiente**: il virus è inattivo. Verrà attivato da un evento (non tutti i virus hanno questa fase).
2. **Fase di propagazione**: il virus inserisce una copia (talvolta mutata per sfuggire al rilevamento) di se stesso in altri programmi o aree di sistema. I cloni inizieranno a loro volta a propagarsi.
3. **Fase di attivazione**: il virus viene attivato da un evento (es. conteggio di copie create) per svolgere la sua funzione.
4. **Fase di esecuzione**: viene espletata la funzione (il payload).

6.3.2 Classificazione dei Virus

1. In Base al Target

Boot sector infector Infetta il Master Boot Record (MBR) o il boot record, diffondendosi all'avvio.

File infector Infetta i file considerati eseguibili dal sistema operativo.

Macro virus Infetta file contenenti codice di scripting interpretato da un'applicazione (es. Office).

Virus multipartito Infetta i file in modi diversi contemporaneamente. Per rimuoverlo, bisogna bonificare tutti i possibili luoghi di infezione.

2. In Base alla Strategia di Camuffamento

Virus criptato Utilizza la crittografia per occultarsi. Una porzione del virus genera una chiave casuale usata per cifrare il resto del codice, memorizzando la chiave col virus. Ad ogni replicazione, la chiave cambia, evitando firme costanti.

Virus furtivo (Stealth) L'intero virus viene nascosto (es. tramite rootkit, compressione o mutazione) alterando le risposte del sistema per nascondere la propria presenza.

Virus polimorfo Crea copie funzionalmente equivalenti ma con pattern di bit diversi per ogni copia. Spesso usa una *mutation engine* (associata a crittografia).

Virus metamorfico Si ridefinisce completamente a ogni iterazione tramite trasformazioni complesse. Cambia non solo l'aspetto, ma talvolta anche il comportamento, rendendo il rilevamento estremamente arduo.

6.3.3 Macro Virus e di Scripting

Un macro virus si attacca ai documenti e sfrutta le capacità di macro dell'applicazione (es. Office, PDF) per eseguirsi e propagarsi.

Pericolosità dei Macro Virus

- **Indipendenza dalla piattaforma:** infettano contenuti trasversali; qualsiasi OS che supporti l'applicazione è vulnerabile.
- **Facilità di diffusione:** i documenti vengono condivisi molto più frequentemente rispetto agli eseguibili.
- **Controlli di accesso inefficaci:** poiché infettano documenti dell'utente, i controlli file system standard non possono prevenirli facilmente.
- Documenti come i PDF supportano componenti integrati (es. JavaScript), offrendo un'ampia superficie d'attacco.

6.4 I Worm

Un worm è un programma che cerca attivamente altre macchine da infettare tramite vulnerabilità software o funzionalità di rete e, una volta insediatosi, le usa per sferrare attacchi ad altre macchine in rete.

6.4.1 Meccanismi di Diffusione

- **E-mail / Messaggistica:** il worm si invia in allegato, eseguendosi all'apertura.
- **Condivisione file / USB:** si copia su supporti rimovibili, sfruttando l'esecuzione automatica o attendendo l'apertura manuale.
- **Esecuzione remota:** sfrutta vulnerabilità in servizi di rete.
- **Accesso remoto ai file / FTP:** si copia su altri sistemi sperando di essere eseguito.
- **Login remoto:** si autentica e copia/esegue se stesso via shell.

Come i virus, i worm presentano le fasi di dormienza, propagazione, attivazione ed esecuzione.

6.4.2 Ricerca del Target (Scanning)

Nella fase di propagazione, il worm scansiona la rete (*fingerprinting*) per trovare nuovi host, creando una vasta rete distribuita. Le strategie di scanning includono:

Casuale Ogni host analizza IP casuali con un seed diverso.

Hit-list L'attaccante precompila una lista di IP vulnerabili. Ogni macchina infetta riceve una porzione della lista da analizzare. Processo molto rapido.

Topologica Utilizza le informazioni (tabelle di routing, cache ARP, rubriche) dell'host vittima per trovare nuovi bersagli.

Sottorete locale Cerca target nella propria rete LAN, bypassando il firewall perimetrale.

Andamento di propagazione di un Worm

La propagazione avviene in tre fasi:

1. Crescita esponenziale: molti host vulnerabili, infezione rapida.
2. Crescita lineare: gli host sprecano tempo ad attaccare macchine già infette, il tasso rallenta.
3. Coda lunga: la maggior parte è infetta; si cercano a fatica i restanti.

6.5 Vulnerabilità Lato Client

Drive-by-Download Avviene quando un utente visita una pagina compromessa. Il codice sfrutta le vulnerabilità del browser per scaricare e installare malware **senza il consenso** dell'utente. A differenza dei worm, il malware attende passivamente la visita.

Watering Hole Variante mirata del drive-by-download. Gli attaccanti infettano un sito web *specifico* frequentemente visitato da un target aziendale o governativo.

Malvertising Gli attaccanti acquistano spazi pubblicitari legittimi (Ad Network) che contengono script malevoli. Colpiscono siti affidabili senza doverli hackerare direttamente. Spesso mirati in base a fascia oraria o profilazione.

Clickjacking L'interfaccia web viene manipolata. Attraverso **iframe** invisibili o trasparenti sovrapposti a bottoni apparentemente innocui, l'utente è ingannato a cliccare su comandi malevoli (es. autorizzare bonifici o permessi).

6.6 Trojan

Trojan

Un trojan è un programma in apparenza utile che contiene codice nascosto il quale, se invocato, svolge operazioni dannose, permettendo all'attaccante azioni indirette. Non si replicano autonomamente.

I trojan possono comportarsi secondo tre modelli:

1. **Esecuzione parallela:** Eseguono l'attività utile e il codice malevolo in parallelo.
2. **Modifica funzioni:** Eseguono l'originale ma ne alterano parti (es. una shell di login che invia password, o comando `ps` che nasconde processi).
3. **Sostituzione totale:** Un programma completamente fittizio.

6.7 Payload di un Malware

Il payload è l'azione specifica che il malware esegue a beneficio dell'attaccante, spesso innescato da bombe logiche.

6.7.1 Distruzione dei Dati

Ransomware

Crypta i dati dell'utente e richiede un riscatto (in criptovaluta) per la chiave di decrittazione.

- **WannaCry (2017):** Ha infettato migliaia di sistemi sfruttando una vulnerabilità Windows (EternalBlue), criptando file e chiedendo Bitcoin. Ha evidenziato l'importanza critica di backup offline e patch aggiornate.

6.7.2 Bombe Logiche

Codice latente che "esplode" solo se si verificano precise condizioni (data, accesso utente specifico, assenza di ping a un server), cancellando dati o bloccando il sistema.

6.8 Agenti di Attacco: Bot e Botnet

Bot e Botnet

Un **bot** (o zombie) è un malware che assume segretamente il controllo di un computer. L'insieme coordinato di centinaia o migliaia di bot forma una **botnet**.

6.8.1 Uso delle Botnet

Le botnet vengono impiegate per:

- **Attacchi DDoS:** sovraccaricare bersagli con traffico massiccio.
- **Spamming:** invio massiccio di email, eludendo filtri anti-spam.
- **Sniffing e Keylogging:** rubare credenziali o dati sensibili (in chiaro tramite sniffer, o prima della cifratura tramite keylogger).
- **Diffusione malware:** usare i bot per distribuire worm o virus.
- **Adware e Click Fraud:** generare clic automatici su pubblicità.

6.8.2 Controllo Remoto (Command and Control)

La differenza principale tra un worm e un bot è la presenza di un'infrastruttura di Command and Control (CnC).

- **Evoluzione:** In passato usavano canali IRC. Oggi usano HTTP/HTTPS o reti Peer-to-Peer per nascondere il traffico.
- **Protezione dei CnC:**
 - *DGA (Domain Generation Algorithms):* Generano centinaia di domini fasulli al giorno per trovare il server CnC attivo, rendendo il blocco manuale inutile.
 - *Fast-flux DNS:* Si alternano rapidamente gli IP associati a un dominio tramite proxy nella botnet stessa.

L'unica contromisura davvero efficace è l'identificazione e il **sequestro/disattivazione** dell'infrastruttura CnC (takedown).

6.9 Furto di Credenziali

6.9.1 Keylogger e Spyware

Keylogger Registra le digitazioni sulla tastiera. Poiché cattura i dati prima della cifratura, HTTPS è inutile in questo caso. I più evoluti filtrano i log per rubare solo le password.

Spyware e Trojan Bancari (Zeus) Malware avanzati (come Zeus) che monitorano l'attività browser, reindirizzano traffico o iniettano form web fasulli (*web injects*) per bypassare le contromisure grafiche (come le tastiere virtuali usate dalle banche).

6.9.2 Phishing

Ingegneria sociale via e-mail. L'attaccante invia mail mascherate da enti legittimi (es. banche) con senso d'urgenza, spingendo a cliccare su link fraudolenti per rubare le credenziali. Spesso inviato in massa via botnet.

Spear-Phishing

Variante altamente mirata (profilazione della vittima sui social) per colpire dirigenti o aziende specifiche, aumentando il tasso di successo (es. false autorizzazioni bancarie).

6.10 Stealthing (Backdoor e Rootkit)

Le tecniche di *stealthing* nascondono la presenza del malware nel sistema.

6.10.1 Backdoor

Accesso segreto (trapdoor) per aggirare l'autenticazione. Possono essere legittime (*maintenance hooks* per il debug del codice) o malevole (inserite da attaccanti). Per prevenirle: revisione attenta del codice, penetration testing e monitoraggio di rete per anomalie (es. porte in ascolto impreviste).

6.10.2 Rootkit

Rootkit

Insieme di strumenti progettati per ottenere e mantenere l'accesso root al sistema in modo invisibile. Sovvertono le API del sistema operativo (es. comandi come `ls` o il Task Manager) per non mostrare file, processi o connessioni legate all'attaccante.

Classificazione dei Rootkit

Persistente Sopravvive al riavvio infettando chiavi di registro o file d'avvio. Più facili da rilevare offline.

Residente in memoria Non lascia traccia sul disco; sparisce al riavvio ma elude gli antivirus.

User-mode Gira nello spazio utente, intercetta e altera le chiamate alle librerie (es. `libc`).

Kernel-mode Gira a livello kernel, modificando le strutture dati del kernel. Molto più potente e difficile da rilevare.

Virtual Machine Based Installa un hypervisor malevolo (VMM) e fa girare l'intero OS vittima in una macchina virtuale invisibile all'OS stesso.

Modalità esterna / Firmware Si nasconde nel BIOS, UEFI o firmware di periferiche, offrendo persistenza totale anche in caso di formattazione.

Capitolo 7

Attacchi Denial-of-Service

7.1 Introduzione

Denial-of-Service (DoS)

Un attacco Denial-of-Service (DoS) è un tentativo di compromettere la disponibilità di un servizio ostacolandone o bloccandone completamente la fornitura. L'attacco tenta di esaurire alcune risorse critiche associate al servizio (ad esempio, larghezza di banda o memoria), impedendogli di rispondere alle richieste degli utenti.

Distributed Denial-of-Service (DDoS)

Gli attacchi DDoS raggiungono un'efficacia nettamente superiore sfruttando molteplici sistemi informatici compromessi come fonti di attacco simultanee. Queste reti sono costituite da computer e dispositivi (noti come *bot* o *zombie*) infettati da malware, che ne permette il controllo a distanza da parte di un attaccante tramite una *botnet*.

7.2 TCP SYN Flood

Un attacco SYN flood è un tipo di DoS che mira a rendere un server indisponibile consumandone tutte le risorse di connessione. Inviando ripetutamente pacchetti di richieste di connessione iniziale (SYN), l'attaccante esaurisce le porte o la tabella di stato del server, obbligandolo a rispondere lentamente o a non rispondere affatto al traffico legittimo.

7.2.1 Funzionamento del TCP SYN Flood

Funzionamento Base di una Connessione TCP

Gli attacchi SYN flood sfruttano il processo di *three-way handshake* di una connessione TCP.

1. Il client invia un pacchetto **SYN** al server per avviare la connessione.
2. Il server risponde con un pacchetto **SYN/ACK** confermando la ricezione e accettando la comunicazione.
3. Il client restituisce un pacchetto **ACK** per confermare a sua volta.

Provocare un Denial-of-Service

Per provocare il DoS, l'attaccante sfrutta il fatto che dopo aver ricevuto il pacchetto **SYN** iniziale, il server alloca risorse per la connessione (che entra nello stato *semi-aperta* o *SYN-RECEIVED*), risponde con **SYN/ACK** e attende l'ultimo **ACK**.

7.2.2 Come si Verifica l'Attacco

1. L'attaccante invia un elevato volume di pacchetti **SYN** al server, spesso falsificando (*spoofing*) l'indirizzo IP di origine.
2. Il server risponde a ciascuna richiesta con **SYN/ACK** e alloca una porta/struttura dati, in attesa del completamento.
3. Poiché l'IP è falsificato (o l'attaccante semplicemente ignora il **SYN/ACK**), l'atteso pacchetto **ACK** non arriverà mai. Il server mantiene la connessione aperta fino al timeout. Saturate le risorse disponibili, il server smette di accettare nuove connessioni.

7.2.3 Tipologie di Attacco SYN Flood

Attacco diretto L'indirizzo IP dell'attaccante **non** è spoofato. È raro, poiché l'attaccante è vulnerabile e la mitigazione è banale (blocco dell'IP). Spesso usato solo se l'attaccante opera tramite botnet, dove l'IP esposto è quello del dispositivo infetto.

Attacco di spoofing L'attaccante falsifica l'indirizzo IP nei pacchetti **SYN**. Rendere più difficile l'identificazione. Risalire all'origine richiede complesse indagini con i provider di servizi Internet (ISP).

Attacco distribuito (DDoS) L'attacco è sferrato tramite una botnet e **ogni** bot falsifica ulteriormente gli IP sorgente. Rintracciare l'origine o mitigare l'attacco tramite semplici filtri IP diventa quasi impossibile.

7.2.4 Mitigazione degli Attacchi SYN Flood

Le moderne infrastrutture mitigate in cloud (es. Cloudflare) si interpongono tra il server vittima e l'attaccante. Gestiscono il processo di *three-way handshake* esternamente. La connessione viene passata al server reale **solo se e quando** il client invia il pacchetto **ACK** finale. Questa strategia sposta il carico del SYN flood sull'infrastruttura globale di mitigazione, risparmiando il server.

7.3 ICMP Flood

ICMP Flood / Ping Flood

Un ICMP flood è un attacco DoS volumetrico in cui l'attaccante sommerge un dispositivo target con pacchetti ICMP Echo-Request (Ping), saturandone la banda o la capacità di elaborazione.

7.3.1 Funzionamento dell'ICMP Flood

Il protocollo ICMP (Internet Control Message Protocol) è utilizzato dai dispositivi per comunicazioni diagnostiche (come i tool `ping` o `traceroute`). L'attacco mira a costringere il bersaglio a elaborare e rispondere a un volume insostenibile di richieste.

1. L'attaccante invia un flusso massiccio di pacchetti `ICMP Echo-Request` verso l'IP vittima.
2. Il server bersaglio, per specifiche di protocollo, è tenuto a inviare un pacchetto `ICMP Echo-Reply` per ciascuna richiesta ricevuta.
3. Questo doppio lavoro (ricezione massiccia ed emissione massiccia) satura sia le risorse CPU del bersaglio sia la larghezza di banda del suo collegamento di rete.

7.3.2 Mitigare l'ICMP Flood

La mitigazione più semplice consiste nel limitare (rate-limiting) o disabilitare del tutto la risposta ai pacchetti ICMP sul router di frontiera o sul firewall del dispositivo vittima, eliminando l'onere dell'elaborazione della risposta.

7.4 UDP Flood

Un UDP flood è un attacco DoS volumetrico in cui un gran numero di pacchetti UDP (User Datagram Protocol) viene inviato a porte casuali di un server bersaglio, con l'obiettivo di esaurirne la capacità elaborativa o saturare la tabella di stato del firewall.

7.4.1 Funzionamento dell'UDP Flood

UDP è un protocollo *connectionless*, non richiede handshake. Quando un server riceve un datagramma UDP, esegue due passaggi:

1. Controlla se c'è un'applicazione (servizio) in ascolto sulla porta di destinazione.
2. Se nessuna applicazione è in ascolto, il sistema operativo risponde inviando al mittente un pacchetto `ICMP Destination Unreachable`.

Il Paragone dell'UDP Flood

Si può paragonare a una receptionist d'albergo che riceve telefonate continue per camere diverse. Per ogni chiamata, deve controllare l'elenco (la tabella delle porte), verificare che l'ospite non ci sia o non risponda, e poi informare il chiamante (il pacchetto ICMP). Se i centralini si illuminano contemporaneamente di richieste fittizie, la receptionist è subito sopraffatta e non può gestire i veri clienti.

7.4.2 Mitigare l'UDP Flood

I sistemi operativi moderni limitano per default la velocità di generazione dei pacchetti ICMP. Tuttavia, se il volume di traffico UDP è enorme, il *firewall stateful* a monte del server andrà comunque in saturazione cercando di tracciare tutte queste "finte connessioni". La mitigazione efficace richiede filtri a monte (presso l'ISP o servizi anti-DDoS cloud) per assorbire il volume prima che raggiunga il target.

7.5 HTTP Flood

HTTP Flood (Livello 7)

È un attacco DoS applicativo (Livello 7 OSI) progettato per sommergere un server Web con richieste HTTP apparentemente legittime. È estremamente insidioso perché è difficile distinguere le richieste malevole da un improvviso picco di traffico reale (effetto *flash crowd*).

7.5.1 Tipologie di Attacco HTTP Flood

HTTP GET Botnet coordinate inviano continue richieste per scaricare risorse pesanti (immagini, PDF, o pagine dinamiche) costringendo il server web a consumare CPU e I/O disco per recuperarle e inviarle.

HTTP POST Spesso diretto verso moduli di login o di ricerca. Il server deve gestire i parametri e spesso interrogare un database di backend. L'elaborazione del DB è computazionalmente costosa, quindi con poche richieste POST ben assestate l'attaccante può paralizzare l'intero stack applicativo (esaurimento risorse DB).

7.5.2 Mitigare un HTTP Flood

Implementare una "challenge" (es. CAPTCHA, controlli JavaScript invisibili o analisi comportamentale) prima di consentire l'accesso al server backend, per verificare se il client è un browser umano o un bot.

7.6 DNS Amplification Attack

Gli attacchi di amplificazione sono una combinazione sofisticata di *reflection* (riflessione) e *amplification* (amplificazione), solitamente basati su protocolli UDP.

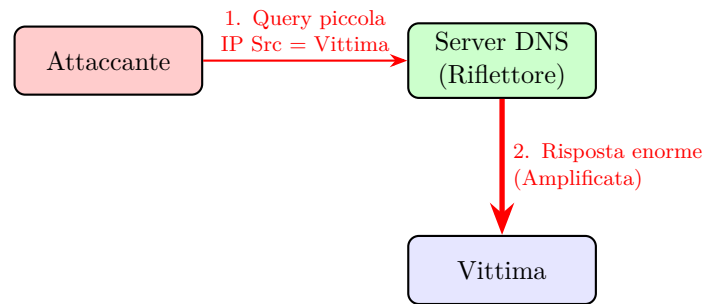
7.6.1 Concetti Chiave

Reflection Attack L'attaccante invia richieste a server legittimi terzi (riflettori) **falsificando l'IP sorgente** per farlo coincidere con quello della vittima. Il server terzo elaborerà la richiesta e invierà la risposta alla vittima.

Amplification Attack Sfrutta servizi in cui una richiesta piccola (*trigger packet*) genera una risposta molto più grande (fattore di amplificazione). L'attaccante moltiplica l'efficacia della sua banda.

7.6.2 Funzionamento del DNS Amplification Attack

Sfrutta i server DNS (Open Resolvers) per amplificare enormemente il traffico contro un bersaglio.



1. L'attaccante invia (tramite botnet) query UDP falsificando l'IP mittente con l'IP della vittima verso numerosi server DNS pubblici (Open Resolvers).
2. La richiesta chiede record molto estesi (es. parametro ANY, firme DNSSEC o ampi record TXT/IPv6).
3. I server DNS, comportandosi correttamente secondo il protocollo, inviano le risposte massive all'indirizzo IP che credono essere il richiedente (la vittima).
4. La rete della vittima viene invasa da un volume di traffico che è decine di volte superiore a quello generato originariamente dall'attaccante (fattore di amplificazione fino a 50x o più).

7.6.3 Mitigare il DNS Amplification Attack

- **Ingress Filtering (BCP 38):** I provider di rete dovrebbero scartare i pacchetti in uscita che hanno IP sorgente non appartenenti alla propria sottorete, impedendo di fatto l'IP spoofing.
- **Sicurezza dei Server DNS:** Disabilitare la ricorsione pubblica sui server DNS (*Open Resolvers*). Un DNS dovrebbe fornire ricorsione solo agli utenti della propria rete locale, non a tutta Internet.

Capitolo 8

Intrusion Detection System

8.1 Introduzione

Intrusion Detection System (IDS)

Gli Intrusion Detection System (IDS) sono strumenti di sicurezza progettati per monitorare e analizzare eventi all'interno di un sistema o di una rete, con l'obiettivo di identificare in tempo reale, o quasi, i tentativi di accesso non autorizzati.

8.2 Struttura di un IDS

Un IDS è tipicamente strutturato in tre componenti logici principali:

Sensori Raccolgono dati da varie parti del sistema come pacchetti di rete, file di log o chiamate di sistema. Queste informazioni vengono inoltrate agli analizzatori.

Analizzatori Ricevono i dati dai sensori e hanno il compito di determinare se si è verificata un'intrusione. Possono anche suggerire o intraprendere azioni di risposta.

Interfaccia utente Consente agli amministratori di visualizzare gli output del sistema, ricevere gli alert e controllare le configurazioni dell'IDS.

8.3 Configurazioni di IDS

Gli IDS si classificano principalmente in base alla fonte e al tipo di dati che analizzano:

Host-based IDS (HIDS) Monitora le caratteristiche di un singolo host e gli eventi che vi accadono (es. file di log, integrità dei file di sistema).

Network-based IDS (NIDS) Analizza il traffico di rete su segmenti o dispositivi specifici (es. router, switch), verificando i livelli di rete, trasporto e applicativi per identificare attività anomale.

IDS distribuiti o ibridi Combinano informazioni provenienti da più sensori, sia host-based che network-based, inviandole a un analizzatore centrale per migliorare il rilevamento correlando eventi complessi.

8.4 Principi di Base e Requisiti

Il principio fondamentale del rilevamento delle intrusioni si basa sull'assunto che il comportamento di un intruso differisca, in modo quantificabile, da quello di un utente legittimo.

Falsi Allarmi vs Tasso di Rilevamento

La progettazione di un IDS richiede un bilanciamento critico:

- Un'interpretazione troppo ampia (*lossy*) genera molti **Falsi Positivi** (comportamenti legittimi segnalati come intrusioni), desensibilizzando gli amministratori.
- Una definizione troppo rigida porta a **Falsi Negativi** (attacchi reali non rilevati).

L'obiettivo ideale è massimizzare il tasso di rilevamento mantenendo un basso tasso di falsi allarmi.

8.4.1 Requisiti di un IDS Efficace

Per essere realmente utile, un IDS dovrebbe soddisfare i seguenti requisiti:

- **Funzionamento continuo:** autonomia con minima supervisione umana.
- **Tolleranza ai guasti:** capacità di recuperare rapidamente da crash.
- **Resistenza alla sovversione:** monitoraggio di se stesso per rilevare tentativi di manomissione da parte degli attaccanti.
- **Basso impatto prestazionale:** overhead ridotto sui sistemi monitorati.
- **Configurabilità e Adattabilità:** capacità di adattarsi alle variazioni storiche del comportamento legittimo e alle policy di sicurezza.
- **Scalabilità:** capacità di monitorare da pochi a migliaia di host.

8.5 Analisi di un IDS (Metodi di Rilevamento)

Gli analizzatori IDS utilizzano diversi approcci per identificare le intrusioni.

8.5.1 Rilevazione delle Anomalie (Anomaly Detection)

Prevede la creazione di un profilo di comportamento "normale" degli utenti legittimi. I comportamenti osservati vengono poi confrontati con questo modello: deviazioni significative vengono segnalate come intrusioni.

Fasi del Processo:

1. **Fase di addestramento:** si costruisce un modello del comportamento legittimo analizzando un periodo di normale attività.
2. **Fase di rilevamento:** il traffico attuale viene confrontato con il modello e classificato.

Le tecniche spaziano dall'analisi statistica (univariata/multivariata), a sistemi esperti basati su conoscenza, fino ad algoritmi di *machine learning*.

Pro e Contro delle Anomalie

Può rilevare attacchi sconosciuti (zero-day), ma la creazione di un modello accurato è molto complessa e solitamente genera un alto tasso di falsi positivi quando il sistema o gli utenti cambiano abitudini lecite.

8.5.2 Rilevazione tramite Firma o Euristica (Misuse Detection)

Questo approccio cerca schemi specifici (*pattern* o firme) associati ad attacchi noti.

Basati su Firma: Confrontano i dati con un database di pattern di attacchi noti (simile agli antivirus classici). Sono efficienti e generano pochi falsi positivi, ma non rilevano minacce sconosciute e necessitano di continui aggiornamenti del database.

Basati su Regole Euristiche: Utilizzano insiemi di regole logiche per identificare tentativi di sfruttamento di vulnerabilità note. Un esempio storico è **SNORT**, un NIDS open-source che utilizza regole scritte in formato leggibile per analizzare i pacchetti.

8.6 Host-Based e Network-Based IDS

8.6.1 HIDS Distribuito

Tradizionalmente un HIDS opera isolato. Oggi si preferiscono HIDS distribuiti che coordinano le difese su più host. L'architettura può essere **Centralizzata** (un nodo raccoglie tutto, facile correlazione ma rischio di collo di bottiglia) o **Decentralizzata** (nodi cooperanti, più resilienti ma complessi da sincronizzare).

L'architettura tipica prevede:

1. **Agente host:** cattura e filtra i dati di audit locali e li formatta in record standard (HAR).
2. **Agente LAN Monitor:** monitora il traffico locale del segmento LAN.
3. **Gestore Centrale:** aggrega e correla i dati per trarre inferenze (es. lo stesso utente sta fallendo il login su 10 macchine diverse contemporaneamente).

8.6.2 Network-Based Intrusion Detection System (NIDS)

Un NIDS agisce come uno *sniffer* passivo sulla rete. Esamina i pacchetti in transito, assemblandone i flussi per cercare pattern malevoli a livello trasporto o applicativo. A differenza dell'HIDS, non consuma risorse sui server di produzione, ma può faticare ad analizzare traffico cifrato (es. HTTPS).

8.7 IDS Distribuito e Cooperativo (Ibrido)

Gli IDS ibridi o SIEM (Security Information and Event Management) integrano dati da NIDS, HIDS, Firewall e log applicativi.

8.7.1 Sfide Moderne

- **Minacce veloci e mutanti:** Worm e malware polimorfi o attacchi zero-day sfuggono alle firme tradizionali.
- **Assenza di confini:** Con il BYOD (Bring Your Own Device) e il cloud, il concetto di "perimetro" e "firewall aziendale" è svanito. Ogni dispositivo è una potenziale via di ingresso.

8.7.2 Rilevamento Cooperativo (Gossip)

Per affrontare minacce lente e distribuite, si usano **sistemi cooperativi p2p**. Ogni nodo (host) agisce come sensore locale. Invece di lanciare falsi allarmi per piccoli sospetti, i nodi comunicano tra loro (protocollo di *gossip*). Se la somma delle anomalie locali raggiunge una soglia globale, si innesca la difesa. L'approccio aumenta enormemente la copertura, riduce i falsi positivi e sfrutta i dati a livello applicativo dell'host.

8.8 Honeypot

Honeypot

Gli honeypot ("barattoli di miele") sono sistemi esca progettati per attirare gli attaccanti e distoglierli dai sistemi critici, raccogliendo al contempo preziose informazioni sul loro comportamento, i loro strumenti e le loro tecniche (Threat Intelligence).

Gli honeypot non ospitano dati reali. Qualsiasi interazione con l'honeytrap è da considerarsi **immediatamente malevola e sospetta**, rendendo l'analisi estremamente pulita (Zero falsi positivi da traffico legittimo).

8.8.1 Classificazione degli Honeypot

Honeypot a Bassa Interazione

Emulano solo alcuni servizi di rete limitati (es. porte FTP, SSH o Web fasulle aperte in ascolto).

- **Vantaggi:** Sicurissimi da usare (l'attaccante non può prendere il controllo del sistema host), molto leggeri, facili da installare.
- **Svantaggi:** Gli attaccanti esperti (o i bot avanzati) capiscono subito che si tratta di un'emulazione e la abbandonano, fornendo poche informazioni.

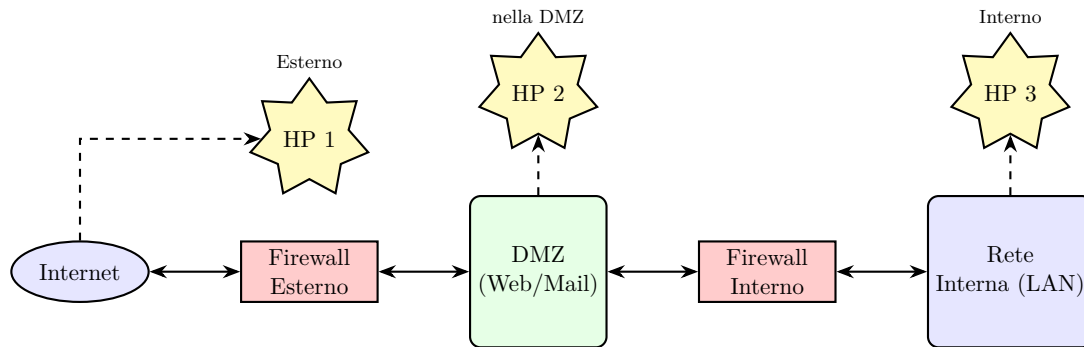
Honeypot ad Alta Interazione

Offrono un sistema operativo reale e completo (spesso virtualizzato), vulnerabile di proposito. L'attaccante ha accesso a una vera shell.

- **Vantaggi:** Ingannano gli hacker più esperti. Permettono di registrare ogni **keystroke**, catturare l'upload di malware e studiare zero-day exploit completi.

- **Svantaggi:** Molto complessi da mantenere. Se non adeguatamente confinati (isolamento di rete e virtualizzazione), l'attaccante potrebbe usare l'honeypot per attaccare macchine reali (*pivot*).

8.8.2 Posizionamento degli Honeypot nella Rete



All'Esterno del Firewall (HP 1) Rileva le scansioni indiscriminate su Internet e i worm. Non presenta alcun rischio per la rete interna, ma raccoglie molto "rumore" di fondo e non aiuta a capire cosa possa passare il firewall.

Nella DMZ (HP 2) Rileva attacchi mirati ai servizi esposti al pubblico. Deve essere protetto rigorosamente per non diventare un ponte verso il firewall interno.

Nella Rete Interna (HP 3) Cruciale per rilevare movimenti laterali di malware già penetrati o tentativi di attacco da parte di minacce interne (*insider threat*).

Capitolo 9

Firewall

9.1 Introduzione

Firewall

I firewall sono dispositivi o software di sicurezza basati sull'idea che il traffico di rete, specialmente quello proveniente da ambienti meno sicuri (come Internet), debba essere autenticato e ispezionato prima di passare a un ambiente più sicuro.

9.2 Requisiti Minimi e Politiche d'Accesso

1. **Canalizzazione obbligatoria:** Tutto il traffico (sia in entrata che in uscita) deve passare attraverso il firewall, che agisce come unico punto di accesso (choke point).
2. **Filtro autorizzativo:** Solo il traffico autorizzato, conforme alle *policy* di sicurezza definite dall'organizzazione, deve essere lasciato passare.
3. **Immunità:** Il firewall stesso deve risiedere su un sistema affidabile e sicuro, immune ai tentativi di penetrazione.

Le **politiche d'accesso** si basano su caratteristiche di filtraggio quali: indirizzi IP, protocolli (TCP/UDP), porte di rete, direzione del traffico, o addirittura comportamenti legati al tempo (orari consentiti) e frequenza (rilevamento di scansioni).

9.3 Filtri del Firewall

Un firewall può operare usando due approcci opposti alle policy predefinite:

Default discard (Deny All) Tutto ciò che non è espressamente permesso è bloccato. È la scelta più conservativa, raccomandata e preferita in ambito aziendale.

Default forward (Allow All) Tutto ciò che non è espressamente proibito è permesso. È un approccio più aperto ma decisamente meno sicuro.

9.3.1 Firewall a Filtraggio di Pacchetto (Stateless)

Applica regole a ciascun pacchetto IP in ingresso e in uscita isolatamente. Decide se inoltrarlo (*forward*) o scartarlo (*drop/reject*) leggendo:

- Indirizzi IP di origine e destinazione.
- Numeri di porta di livello trasporto (TCP/UDP).
- Campo protocollo nell'header IP (es. ICMP, TCP, UDP).
- Interfaccia di rete di arrivo o uscita.

Attacchi ai Packet Filter

I firewall stateless presentano diverse vulnerabilità strutturali:

- **IP Spoofing:** L'attaccante falsifica l'IP sorgente fingendo di essere un host interno. *Contromisura:* Scartare pacchetti che arrivano dall'interfaccia esterna ma recano un IP interno (Anti-spoofing).
- **Source Routing:** L'attaccante specifica il percorso forzato del pacchetto. *Contromisura:* Disabilitare e scartare pacchetti con opzione IP Source Routing.
- **Tiny Fragment Attack:** L'attaccante frammenta l'header TCP in pezzi microscopici, sperando che il firewall legga solo il primo frammento incompleto e lasci passare il resto. *Contromisura:* Imporre una lunghezza minima per il primo frammento.

9.3.2 Firewall a Filtraggio di Pacchetto con Stato (Stateful)

A differenza del filtro stateless, il firewall *stateful* tiene traccia del **contesto** delle connessioni TCP/UDP. Crea una tabella di stato dinamica in cui memorizza le connessioni in uscita legittime (es. un client interno che apre una connessione sulla porta 80). Quando arrivano i pacchetti di risposta sulla porta ad alto numero generata casualmente dal client (es. porta 51234), il firewall controlla la tabella: se il pacchetto appartiene a una connessione aperta, passa; altrimenti viene bloccato. Questo evita di dover tenere permanentemente aperte tutte le porte alte in ingresso, migliorando drasticamente la sicurezza.

9.4 Tipi di Firewall Avanzati

9.4.1 Gateway a Livello Applicativo (Proxy)

Un proxy applicativo funge da intermediario totale. L'utente si connette al proxy, si autentica, e il proxy apre una seconda e distinta connessione verso il server reale.

- **Vantaggi:** Altissima sicurezza (comprende i comandi del livello 7 OSI, es. GET di HTTP o PUT di FTP), logging eccellente.
- **Svantaggi:** Forte *overhead* prestazionale, poiché interrompe la connessione TCP ed elabora i dati per intero in *user-space*.

9.4.2 Proxy a Livello di Circuito

Gestisce connessioni senza analizzare il payload applicativo. L'intermediario accetta la connessione dal client TCP, ne apre un'altra verso il server, e semplicemente copia i segmenti TCP da una connessione all'altra. Ideale se l'amministratore si fida degli utenti interni (es. proxy SOCKS in uscita).

9.5 Classificazione per Locazione

Bastion Host Un sistema hardware/software pesantemente fortificato, esposto alla rete ostile. Esegue solo i servizi strettamente necessari (spesso solo funzioni di firewall o proxy) ed è pesantemente *hardened*.

Host-based Firewall Software installato su un singolo server (es. *iptables* su Linux, Windows Defender Firewall). Protegge il singolo sistema a prescindere dall'infrastruttura di rete.

Personal Firewall Software per PC o workstation domestiche (es. Little Snitch, Norton). Avvisano l'utente sulle connessioni e bloccano traffico non autorizzato a livello applicativo.

pfSense

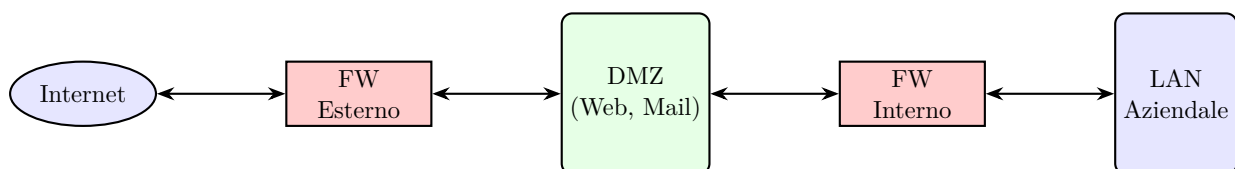
pfSense è una popolare distribuzione basata su FreeBSD dedicata all'implementazione di router e firewall avanzati. Date le sue capacità (VPN, proxy, stateful filtering) e robustezza, è spesso impiegato come Bastion Host nelle architetture di rete aziendali.

9.6 Architetture di Sicurezza di Rete (Topologie)

L'amministratore decide quanti firewall usare e dove posizionarli. La soluzione più comune per proteggere i servizi esposti è la DMZ.

9.6.1 La Rete DMZ (Demilitarized Zone)

La DMZ è un segmento isolato che contiene i servizi pubblici dell'azienda (Web Server, Mail, DNS). È separata sia da Internet che dalla LAN aziendale.



- **Firewall Esterno:** Controlla l'accesso alla DMZ limitandolo ai servizi essenziali.
- **Firewall Interno:** Molto più restrittivo. Impedisce ad un attaccante che abbia compromesso un server nella DMZ di fare movimenti laterali ed entrare nella LAN interna. Al contempo protegge la DMZ da minacce provenienti dalla LAN.

9.6.2 Topologie Comuni

Router di screening Un semplice router (stateless) tra LAN e Internet. Usato per SOHO (Small Office / Home Office).

Bastion singolo in-line Un solo firewall tra interno ed esterno.

Bastion singolo a T Un unico firewall con tre interfacce di rete (tre schede di rete): una per Internet, una per la LAN, una per la DMZ.

Doppio bastion in-line (Vedi schema sopra). La DMZ si trova tra due firewall hardware separati. Molto sicuro e usato da grandi aziende.

Firewall distribuiti Una combinazione di firewall autonomi hardware di perimetro coordinati con firewall host-based installati su ogni endpoint, tutti gestiti da un server centrale.

9.6.3 VPN e Firewall

Le VPN (Virtual Private Network) crittografano il traffico (es. protocollo IPSec) permettendo a sedi remote di comunicare in sicurezza su Internet.

VPN e Ispezione

Se un gateway IPSec è posto **dietro** il firewall, il traffico attraversa il firewall criptato. Il firewall non potrà ispezionarne il contenuto a livello 7, vanificando alcune difese. Per questo motivo, nei dispositivi moderni (es. Next-Generation Firewall) i tunnel VPN vengono spesso terminati direttamente **sul** firewall, che decripta il traffico, lo ispeziona e poi lo inoltra alla rete interna.

9.7 Intrusion Prevention System (IPS)

L’Intrusion Prevention System (IPS) è l’evoluzione reattiva dell’IDS. Non si limita a rilevare e avvisare, ma **agisce attivamente** (blocca pacchetti, chiude connessioni TCP, altera chiamate di sistema) per prevenire l’intrusione. L’IPS è spesso installato *in-line* (in linea col traffico) come un firewall di nuova generazione.

9.7.1 Host-Based IPS (HIPS)

Lavora sul singolo host. Identifica comportamenti malevoli tramite anomalie o firme. Può bloccare:

- Alterazioni non autorizzate di file di sistema e del registro (rootkit).
- Tentativi di escalation dei privilegi.
- Attacchi di Buffer Overflow e *directory traversal*.

Un HIPS moderno sfrutta tecnologie di **Sandbox**: isola codice sospetto o script all’interno di un ambiente sicuro per osservarne il comportamento prima di permetterne l’esecuzione sul sistema reale.

9.7.2 Network-Based IPS (NIPS)

È una versione *in-line* del NIDS. Deve essere capace di analizzare il traffico a velocità di rete elevatissime. Requisito fondamentale è la ricostruzione del payload: il NIPS riassume i frammenti TCP per leggere l'intero flusso applicativo e confrontarlo con le firme. Se il flusso è malevolo, interrompe la connessione (es. inviando TCP RST o semplicemente scartando (*dropping*) tutti i pacchetti futuri di quel flusso).

Usa pattern matching (byte specifici), stateful matching (ricerca firme nel flusso di stato), e anomalie statistiche o protocollari (deviazioni dagli standard RFC).

9.7.3 Sistema Immunitario Digitale

Un paradigma moderno e globale. Prevede una vasta rete di sensori IPS distribuiti che inviano dati e file sospetti a un centro di analisi globale in cloud. Se in un'azienda viene rilevato un comportamento mai visto (zero-day), il sistema immunitario lo analizza in cloud (tramite sandbox AI), estrae una firma e invia in tempo reale l'aggiornamento a tutti i firewall/IPS del mondo connessi a quel servizio. Questo offre una risposta collettiva e proattiva (es. Threat Intelligence di Cisco Talos o FortiGuard).

Capitolo 10

Buffer Overflow

10.1 Principi del Buffer Overflow

Buffer Overflow

Un buffer overflow (o buffer overrun/overwrite) è una condizione anomala in cui un programma, scrivendo dati in un'area di memoria preallocata (il buffer), supera i limiti di capienza del buffer stesso, andando a sovrascrivere locazioni di memoria adiacenti.

Questo fenomeno, originato da difetti di programmazione, può essere sfruttato da un attaccante per causare il crash dell'applicazione (Denial of Service) o, peggio, per alterare il flusso di esecuzione del programma e iniettare codice malevolo, ottenendo il controllo del sistema.

10.1.1 Cause e Conseguenze

La causa principale è la mancanza di controlli di *bounds checking* (verifica dei limiti) in linguaggi di basso livello come C o C++.

Le principali conseguenze includono:

- **Corruzione dei dati:** Variabili adiacenti vengono sovrascritte, alterando la logica del programma.
- **Alterazione del flusso di esecuzione:** Il programma cerca di eseguire istruzioni in un indirizzo alterato dall'attaccante.
- **Segmentation Fault:** L'accesso a zone di memoria non mappate o protette causa la terminazione anomala del processo.
- **Esecuzione di codice arbitrario:** L'attaccante inietta il proprio codice (shellcode) e devia il flusso per eseguirlo, ereditando i privilegi del processo vulnerabile.

10.1.2 Requisiti per Sfruttare un Overflow

Per capitalizzare la vulnerabilità, l'attaccante deve:

1. **Identificare il vettore di vulnerabilità:** Trovare un input controllabile dall'esterno non sanato correttamente. Spesso si usa il *fuzzing* (invio massivo di input casuali, es. stringhe molto lunghe) o la *static code analysis*.

2. **Comprendere il layout della memoria (Stack Frame):** L'attaccante deve conoscere o indovinare l'esatta distanza in byte tra l'inizio del buffer e l'indirizzo di ritorno (RET) per sovrascriverlo con estrema precisione.

10.2 Shellcode

Shellcode

Lo shellcode è un piccolo frammento di codice (generalmente scritto in linguaggio assembly e convertito in byte grezzi - *opcode*) che l'attaccante inietta nella memoria del target. Viene chiamato così perché il suo scopo tradizionale è aprire una shell (es. `/bin/sh` su Linux o `cmd.exe` su Windows) per consentire all'attaccante di inviare comandi.

10.2.1 Caratteristiche e Restrizioni

Lo shellcode è **altamente dipendente dall'architettura** (x86, ARM, x64) e dal sistema operativo attaccato, poiché invoca direttamente le chiamate di sistema (System Calls o *syscall*) bypassando le librerie.

La scrittura dello shellcode è complessa e deve rispettare rigide regole dettate dai vettori di iniezione:

Position Independent Code (PIC) Il codice non deve usare indirizzi assoluti in memoria, poiché la posizione del buffer cambia da esecuzione a esecuzione (specie se ci sono meccanismi di ASLR). Utilizza invece salti relativi.

Nessun byte NULL (0x00) Le funzioni C che manipolano le stringhe (`strcpy`, `sprintf`) si fermano appena incontrano un byte NULL, considerandolo la fine della stringa. Se lo shellcode contiene uno 0x00, verrebbe copiato solo in parte, corrompendosi. Gli sviluppatori di shellcode usano trucchi assembly (es. operazione logica `XOR EAX, EAX`) per azzerare i registri senza scrivere zeri.

10.3 Difese Contro i Buffer Overflow

Nel corso degli anni, le architetture e i sistemi operativi hanno implementato difese stratificate (a tempo di compilazione e a tempo di esecuzione).

10.3.1 Difese a Tempo di Compilazione

Scelta del Linguaggio e Librerie Sicure

Utilizzare linguaggi strongly typed o managed (Java, C#, Python) che eseguono automaticamente il bounds checking, eliminando alla radice il problema. Se si deve usare il C/C++, i programmatori devono usare versioni sicure delle librerie standard (es. `strncpy` al posto di `strcpy`, `snprintf` invece di `sprintf`) oppure framework esterni come *Libsafe*.

Meccanismi di Protezione dello Stack (Canaries)

Stack Canaries (StackGuard)

L'estensione StackGuard del compilatore GCC inserisce un piccolo valore casuale (chiamato **canarino** o *stack cookie*) tra le variabili locali e l'indirizzo di ritorno (RET) al momento della chiamata della funzione. Prima di eseguire l'istruzione RET, la CPU controlla l'integrità del canarino. Se un buffer overflow cerca di sovrascrivere il RET, sovrascriverà inevitabilmente anche il canarino. Il mismatch del valore causa l'immediata terminazione del programma, sventando l'attacco.

Altri sistemi, come **Stackshield** / **RAD (Return Address Defender)**, copiano il RET originale in un'area di memoria isolata e protetta (shadow stack) all'ingresso della funzione, confrontandolo all'uscita per verificarne l'integrità.

10.3.2 Difese a Tempo di Esecuzione

Randomizzazione dello Spazio degli Indirizzi (ASLR)

L'Address Space Layout Randomization (ASLR) è una difesa software del sistema operativo. Modifica in modo casuale gli indirizzi di base dello stack, dell'heap e delle librerie condivise (libc) ad ogni nuova esecuzione del programma. In questo modo, l'attaccante non può *hardcodare* l'indirizzo a cui far saltare l'esecuzione, rendendo l'attacco una scommessa statistica (spesso destinata a fallire causando un semplice crash).

Pagine di Guardia

Inserimento di pagine di memoria vuote e non mappate tra aree critiche di memoria. Qualsiasi tentativo di accesso (lettura/scrittura) sequenziale da parte di un buffer overflowing che oltrepassa il limite urterà la *guard page*, innescando una violazione di memoria (page fault) e il blocco del programma.

10.4 Altre Forme di Attacchi Overflow

Sostituzione dello Stack Frame Se un off-by-one error (errore di un solo byte) permette di sovrascrivere l'ultimo byte del Frame Pointer (EBP), l'attaccante non sovrascrive direttamente il RET. Invece, fa puntare l'EBP a uno *stack frame fittizio* creato precedentemente nell'area dati o nello stack, alterando la catena delle chiamate di funzione e dirottando l'esecuzione.

Heap Overflow L'heap è l'area di memoria per le allocazioni dinamiche (`malloc`, `new`). Qui non c'è un indirizzo di ritorno (RET) da sovrascrivere. L'attaccante mira a sovrascrivere i metadati di gestione della memoria (usati dalle funzioni `free()`) per corrompere i puntatori, oppure sovrascrive esplicitamente i *puntatori a funzione* o i *VTable pointers* del C++ presenti nelle strutture dati vicine, forzandole a puntare allo shellcode. **Difese:** Allocatori heap sicuri (che verificano l'integrità dei metadati), ASLR dell'heap e heap non eseguibile.

Capitolo 11

Sicurezza del Software

11.1 Problemi di Sicurezza del Software

La stragrande maggioranza delle vulnerabilità di sicurezza informatica deriva da pratiche di programmazione inadeguate. Enti di ricerca e liste autorevoli (come la OWASP Top 10) evidenziano come la maggior parte degli attacchi sfrutti un sottoinsieme limitato e ricorrente di errori architetturali e di programmazione.

Tra i più comuni troviamo: **Injection** (es. SQL Injection, OS Command Injection), **Cross-Site Scripting (XSS)**, **Buffer Overflow**, **gestione insicura delle risorse** e della memoria, e **difese logiche inadeguate** (es. autenticazione mancante, credenziali *hard-coded*, crittografia debole).

11.2 Sicurezza vs. Qualità del Software

Spesso i concetti di qualità e sicurezza del software vengono confusi, sebbene si concentrino su aspetti fondamentalmente diversi della gestione dei guasti.

Qualità e Affidabilità (*Software Reliability*)

Si occupa della prevenzione e gestione dei fallimenti **accidentali**, tipicamente causati da input imprevisti, malfunzionamenti hardware o *bug* stocastici. L'obiettivo è massimizzare la stabilità del sistema durante il suo utilizzo nominale e in condizioni operative standard.

Sicurezza (*Software Security*)

Si concentra sui fallimenti indotti **deliberatamente** da un attaccante (un attore ostile). L'attaccante seleziona e invia input anomali, mirati e specifici, al fine di innescare e sfruttare vulnerabilità nel codice. Tali input sono spesso statisticamente improbabili in un contesto di utilizzo standard.

11.3 Programmazione Difensiva

Per mitigare le minacce alla sicurezza, l'ingegneria del software adotta il paradigma della **programmazione difensiva**.

Principio della Programmazione Difensiva

La programmazione difensiva consiste nel progettare e implementare il software affinché mantenga la propria integrità e operatività anche sotto attacco. Il postulato fondamentale è: *non dare mai nulla per scontato*.

Il sistema deve essere in grado di rilevare stati anomali e, nel peggiore dei casi, attuare una **terminazione controllata** (*fail gracefully*), evitando in ogni circostanza di esporre dati sensibili, rivelare informazioni di *debug* all'utente finale o concedere privilegi non autorizzati. Ogni assunzione deve essere verificata a tempo di esecuzione e ogni possibile stato di errore deve essere gestito esplicitamente.

11.4 Gestione Sicura dell'Input

La corretta gestione dell'input rappresenta la prima linea di difesa per qualsiasi software. Per "input" si intende qualsiasi flusso di dati proveniente dall'esterno del perimetro di fiducia (*trust boundary*) del programma, come la tastiera, la rete, i form web, il file system o le variabili d'ambiente.

Criticità dell'Input

Le due criticità principali relative all'input, e vettori della maggior parte degli attacchi, sono la sua **dimensione** (che può causare overflow) e la sua **interpretazione** (che può causare injection).

11.4.1 Dimensione dell'Input e Buffer Overflow

Se un'applicazione non verifica rigorosamente che la dimensione dei dati in ingresso sia compatibile con la capacità del buffer di memoria preposto a contenerli, si verifica un **Buffer Overflow**. Questa vulnerabilità permette a un attaccante di sovrascrivere aree di memoria adiacenti, alterando il flusso di esecuzione del programma e arrivando, nei casi più gravi, all'esecuzione di codice arbitrario (*Arbitrary Code Execution*).

11.4.2 Interpretazione dell'Input e Attacchi di Injection

Un attacco di *Injection* si verifica quando un input non adeguatamente sanitizzato o validato viene interpretato dal sistema come una vera e propria istruzione o comando, alterando così la logica di esecuzione.

- **Command Injection:** L'input dell'utente viene concatenato a una *system call* per eseguire comandi sul sistema operativo. Inserendo metacaratteri della shell (es. `;` oppure `|`), l'attaccante può eseguire comandi arbitrari sul sistema operativo ospite.
- **SQL Injection (SQLi):** L'input viene impiegato per la costruzione dinamica di una query al database relazionale. Sfruttando metacaratteri SQL (come `'` o la sequenza di commento `--`), è possibile manipolare la sintassi originale della query per aggirare l'autenticazione, esfiltrare dati sensibili o corrompere le tabelle del database.
- **Code Injection:** L'input fornito viene valutato ed eseguito direttamente come codice sorgente dall'interprete del linguaggio (es. l'uso improprio di funzioni `eval()` o vulnerabilità di *Remote File Inclusion* in applicazioni PHP).

11.4.3 Tecniche di Validazione dell'Input

Whitelisting vs Blacklisting

- **Whitelisting (Raccomandato):** Consiste nell'accettare esclusivamente input che corrisponda a un formato, tipo o pattern (tramite *Regular Expression*) esplicitamente definito come sicuro. Tutto il traffico non conforme viene scartato per impostazione predefinita.
- **Blacklisting (Sconsigliato):** È il tentativo di bloccare stringhe o pattern noti per essere malevoli. È un approccio intrinsecamente debole, poiché l'attaccante troverà costantemente nuove codifiche, varianti o evasioni per eludere il filtro.

Inoltre, prima di qualsiasi validazione, l'input deve subire un processo di **Canonicalization**. L'input deve essere convertito nella sua forma canonica. Questo impedisce che tecniche di evasione basate su codifiche alternative (es. *URL encoding*) superino i controlli di sicurezza che altrimenti non le riconoscerebbero.

11.5 Scrittura di Codice Sicuro e Gestione della Memoria

11.5.1 Tipizzazione, Memory Leak e Race Condition

Altre classi di vulnerabilità legate alla gestione del codice e delle risorse includono:

- **Tipizzazione Forte (*Strong Typing*):** Linguaggi come Java o Rust impongono vincoli rigorosi sui tipi di dato a tempo di compilazione ed esecuzione. Questo mitiga intere classi di vulnerabilità tipiche dei linguaggi a tipizzazione debole e gestione manuale della memoria (come C/C++), dove le conversioni implicite e l'aritmetica dei puntatori espongono al grave rischio di corruzione della memoria.
- **Memory Leak:** La mancata deallocazione dinamica della memoria sull'*heap* causa l'esaurimento progressivo e inesorabile delle risorse di sistema. Questa condizione è facilmente sfruttabile per attacchi di *Denial-of-Service* (DoS).
- **Race Condition:** Si manifesta quando processi operanti in concorrenza accedono a risorse condivise senza un'adeguata sincronizzazione (es. in assenza di *mutex* o operazioni atomiche). A seconda del tempismo (*timing*) dei processi, questo può condurre alla corruzione dello stato interno, a *deadlock* o a vulnerabilità di tipo *Time-of-Check to Time-of-Use*.

11.6 Interazione con il Sistema Operativo e Privilegi

L'esecuzione di un programma è mediata dal Sistema Operativo, il quale introduce ulteriori e critici vettori di attacco se i processi non vengono gestiti con attenzione.

Il Principio del Minimo Privilegio (*Least Privilege*)

Un processo deve essere eseguito possedendo **unicamente** i privilegi strettamente necessari all'assolvimento della propria funzione specifica, e mantenerli per il minor tempo possibile. In questo modo, se il processo viene compromesso, i danni causati dall'attaccante saranno limitati ai privilegi del processo stesso.

Per implementare fattivamente questo principio si ricorre a tecniche avanzate:

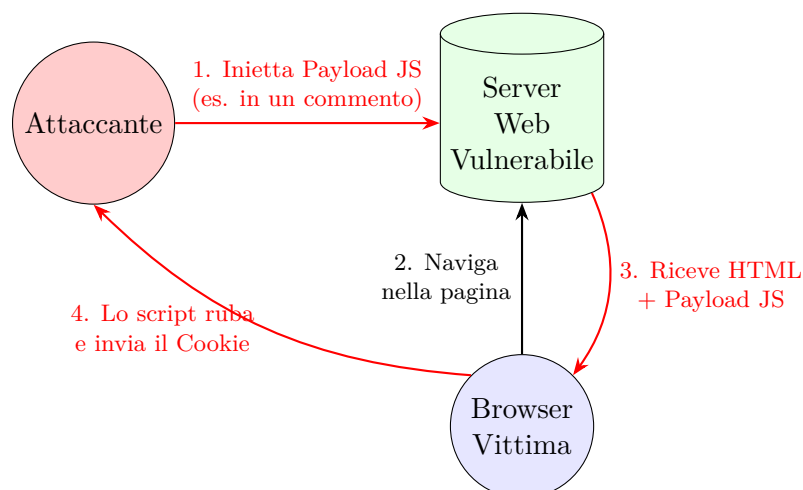
- **Privilege Dropping:** Demoni di rete che necessitano di privilegi di *root* solo per il *binding* iniziale (es. porsi in ascolto su porte riservate < 1024) devono rinunciare (de-escalare) ai propri permessi assumendo un utente con privilegi limitati (es. *www-data* o *nobody*) subito dopo l'avvio.
- **Sandboxing e Chroot:** L'esecuzione di processi potenzialmente non fidati o esposti a input utente in ambienti virtualmente isolati (es. *container*, *chroot jail*). Questo impedisce al processo di accedere all'intero file system principale e limita fortemente le chiamate di sistema critiche disponibili.

11.7 Gestione Sicura dell'Output e Cross-Site Scripting (XSS)

Nel contesto delle applicazioni web, non è sufficiente gestire in sicurezza l'input; la corretta gestione dell'output è altrettanto critica.

Cross-Site Scripting (XSS)

Il **Cross-Site Scripting** si verifica quando un'applicazione web riflette o memorizza un input malevolo fornito da un utente (tipicamente un *payload* in linguaggio JavaScript) e lo ripropone incapsulato nelle pagine HTML inviate ai browser di altre vittime innocue. Poiché il browser si fida dell'origine della pagina, eseguirà ciecamente lo script. Questo consente all'attaccante di esfiltrare *cookie* di sessione, fare richieste non autorizzate per conto dell'utente o manipolare visivamente il DOM (*Document Object Model*).



Difese per l'Output:

- **Sanitizzazione e Output Encoding:** Consiste nel trasformare tutti i caratteri speciali che hanno un significato per il parser HTML in entità inoffensive (es. < viene convertito in <; > in >,) prima della renderizzazione della pagina.
- **Dichiarazione del Character Encoding:** È necessario specificare esplicitamente la codifica dei caratteri (es. UTF-8) nell'header HTTP per impedire interpretazioni anomale del testo da parte del parser del browser, che potrebbero eludere le difese di encoding.

Capitolo 12

Sicurezza nei OS e Virtualizzazione

12.1 Introduzione alla Sicurezza dei Sistemi Operativi

Vulnerabilità Intrinseche e Deployment

È imperativo riconoscere che i Sistemi Operativi (OS) presentano vulnerabilità intrinseche, costantemente sondate da agenti malevoli (es. *worm* o *botnet* su Internet). Un sistema può essere compromesso già durante la delicata fase di installazione (*deployment*), prima ancora che vengano applicate le patch di sicurezza o implementate le policy aziendali.

12.2 Il Processo di Messa in Sicurezza (NIST SP 800-123)

Le Fasi del Deployment Sicuro

1. **Valutazione dei Rischi e Pianificazione:** Analisi preventiva del contesto operativo, delle minacce e dei requisiti di business.
2. **Hardening dell'OS base:** Messa in sicurezza del sistema operativo prima dell'esposizione in rete.
3. **Sicurezza delle Applicazioni:** Configurazione sicura dei servizi applicativi chiave.
4. **Protezione del Contenuto:** Salvaguardia dei dati critici tramite crittografia (Data at Rest / Data in Transit) e rigorosi controlli di accesso.
5. **Sicurezza di Rete:** Implementazione di firewall host-based e sistemi di rilevamento delle intrusioni.
6. **Manutenzione Continua:** Definizione di processi per il patching automatizzato, il logging e il backup nel tempo.

12.3 Hardening del Sistema Operativo

La messa in sicurezza dell'OS rappresenta il fondamento fiduciario (*Root of Trust*) per tutte le applicazioni sovrastanti.

Insecure by Default

Le configurazioni di *default* fornite dai produttori (sia per sistemi operativi che per dispositivi di rete) privilegiano storicamente l'usabilità immediata e la retrocompatibilità a scapito della sicurezza. Questo approccio *insecure by default* richiede un'azione proattiva da parte dell'amministratore.

Hardening

L'hardening (indurimento) è il processo di riduzione della superficie di attacco di un sistema tramite la disattivazione di servizi non necessari, l'applicazione di configurazioni restrittive e l'implementazione del principio del minimo privilegio.

L'hardening procede attraverso i seguenti principi:

- **Installazione Sicura:** Il deployment iniziale deve avvenire in una rete logicamente isolata (VLAN di *staging* o rete *air-gapped*) per prevenire infezioni pre-patching da parte di *worm* diffusi nella rete aziendale.
- **Principio di Minimalità:** Installare esclusivamente i pacchetti, i moduli e i driver strettamente necessari. La rimozione (o la mancata installazione) di un servizio riduce drasticamente i vettori di attacco esposti.
- **Sicurezza del Boot:** Proteggere il processo di avvio (es. impostando password per UEFI/BIOS e abilitando il *Secure Boot*) per prevenire l'alterazione dei file critici, il caricamento di *rootkit* o di *hypervisor* malevoli prima che l'OS principale venga caricato.

12.3.1 Manutenzione, Logging e Backup

Il **Logging** è un controllo reattivo fondamentale per la *Digital Forensics* e l'*Incident Response*. I log devono essere raccolti, protetti da manomissioni (es. inoltrandoli a un server *syslog* remoto), ruotati e analizzati (preferibilmente in modo automatizzato tramite soluzioni SIEM - *Security Information and Event Management*).

Parallelamente, rigorose policy di **Backup e Archiviazione** garantiscono la *Disaster Recovery*, proteggendo l'organizzazione da guasti fisici, errori umani o attacchi distruttivi.

12.4 Sicurezza in Ambiente Linux/Unix

I sistemi *Unix-like* basano storicamente la loro sicurezza su un modello di permessi granulare noto come Controllo degli Accessi Discrezionale (DAC).

- **Utenti, Gruppi e ACL:** I permessi classici (suddivisi in lettura, scrittura, esecuzione per *owner*, *group*, *others*) possono essere estesi tramite le *Access Control Lists* (ACL) per gestire scenari di condivisione più complessi. L'autenticazione è tipicamente demandata e centralizzata tramite il framework **PAM** (*Pluggable Authentication Modules*).

12.5 Sicurezza in Ambiente Windows

I sistemi operativi Microsoft Windows utilizzano un'architettura di sicurezza basata sui **Security Identifier (SID)**, stringhe univoche che identificano utenti, gruppi e computer, garantendo una gestione unificata (es. tramite *Active Directory* nei domini aziendali).

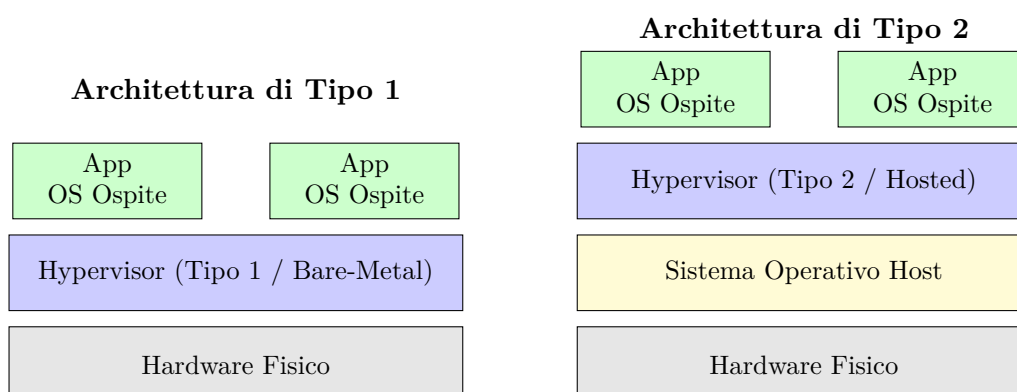
- **Mandatory Integrity Control (MIC):** implementa una forma derivata del modello di sicurezza di *Biba*. Assegna rigidi livelli di integrità (Basso, Medio, Alto, Sistema) ai processi e agli oggetti. Questo meccanismo impedisce a codice non attendibile o isolato (come un processo del browser a integrità bassa) di modificare file di sistema eseguiti a integrità alta.
- **User Account Control (UAC):** Rappresenta l'implementazione del principio del minimo privilegio applicato all'interfaccia utente. Gli utenti (anche se amministratori) operano di default con un token di accesso standard e restrittivo, richiedendo un'elevazione dei privilegi esplicita e consensuale per eseguire task amministrativi o modificare impostazioni di sistema.
- **Registro di Sistema e Crittografia:** L'hardening in Windows passa spesso dalla configurazione massiva e centralizzata di chiavi di registro, distribuita tramite *Group Policy Objects* (GPO). È inoltre fondamentale abilitare la *Data at Rest Encryption* nativa (es. **BitLocker** per la cifratura del disco intero o EFS per singoli file).

12.6 Sicurezza della Virtualizzazione

La virtualizzazione astrae le risorse hardware fisiche (CPU, RAM, storage, network) fornendole in modo isolato a molteplici Macchine Virtuali (VM). L'**Hypervisor** (o *Virtual Machine Monitor - VMM*) è il cuore di questa architettura, responsabile del partizionamento logico, dello *scheduling* e del rigido isolamento delle VM ospiti.

12.6.1 Tipologie di Hypervisor

1. **Tipo 1 (Nativo / Bare-Metal):** Installato ed eseguito direttamente sull'hardware nudo del server (es. Proxmox, Microsoft Hyper-V). Offre prestazioni nettamente superiori e una superficie di attacco minore, in quanto non dipende da un OS ospite sottostante.
2. **Tipo 2 (Hosted):** Eseguito come una normale applicazione all'interno di un OS preesistente (es. VirtualBox, VMware Workstation). È più flessibile per l'uso *client* su macchine di sviluppo, ma eredita in blocco tutte le vulnerabilità, i ritardi e l'overhead dell'OS *host*.



12.6.2 Minacce e Isolamento negli Ambienti Virtualizzati

L'introduzione dell'hypervisor aggiunge un nuovo e critico strato architetturale:

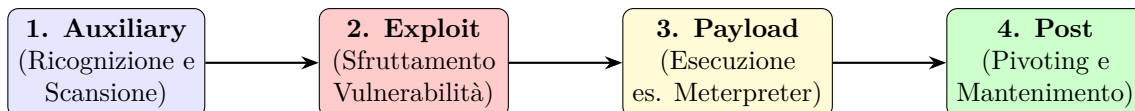
- **Traffico di Rete Virtuale:** Le VM comunicano frequentemente tra loro tramite *switch* virtuali interni all'hypervisor, rendendo il traffico di rete intra-host completamente invisibile ai tradizionali IPS e firewall hardware perimetrali. È necessario implementare **Firewall Virtuali** (operanti a livello dell'hypervisor o come *appliance* VM dedicate) e separare rigidamente il traffico di gestione (*management*), di infrastruttura (es. la rete dedicata alla migrazione live delle VM) e il traffico applicativo degli utenti.

VM Escape

Il **VM Escape** è una vulnerabilità critica e devastante in ambienti multi-tenant (come il Cloud). Si verifica quando un attaccante, partendo dal controllo completo di una VM compromessa, riesce a sfruttare un bug nell'hypervisor per eludere l'isolamento ed eseguire codice arbitrario direttamente sull'hypervisor host o all'interno delle altre VM co-ospitate sullo stesso hardware fisico.

12.7 Metasploit Framework: Penetration Testing e Assessment

Il **Metasploit Framework** è l'infrastruttura open-source *de facto* per lo sviluppo, il collaudo e l'esecuzione di codice *exploit*. È utilizzato costantemente dai professionisti della sicurezza (*White Hat* e *Red Team*) per validare sperimentalmente l'efficacia delle misure di *hardening* e condurre complessi *Penetration Test*.



12.7.1 Architettura e Moduli

Metasploit vanta un'architettura modulare ed estensibile che permette di disaccoppiare logicamente la fase di ingaggio (*exploit*) dal software esecutivo (*payload*). I moduli si dividono in:

Considerazioni Etiche e Legali (Dual-Use)

Metasploit, al pari di molti strumenti di sicurezza, è un software a duplice uso (*dual-use*). Nel contesto accademico e professionale aziendale, il suo utilizzo è strettamente normato e sorvegliato. L'esecuzione non autorizzata di moduli di exploit verso infrastrutture e server di cui non si possiede la proprietà, o per le quali non si è ottenuta un'autorizzazione scritta, formale ed esplicita (*Scope of Work / Rules of Engagement*), costituisce un **grave reato informatico** severamente punito dalle normative internazionali (es. accesso abusivo a sistema informatico).

Capitolo 13

Cloud Computing e IoT

13.1 Cloud Computing: Definizioni e Architettura

Cloud Computing (NIST SP 800-145)

Il Cloud computing è "un modello che abilita l'accesso di rete a risorse di calcolo configurabili (es. reti, server, storage, applicazioni e servizi).

Le 5 Caratteristiche Essenziali del Cloud

1. **Accesso alla Rete Esteso (*Broad network access*):** Risorse fruibili tramite meccanismi standard da piattaforme client eterogenee (mobile, workstation).
2. **Elasticità Rapida (*Rapid elasticity*):** Scalabilità dinamica (*scale-up* e *scale-down*) delle risorse in base al carico di lavoro.
3. **Servizio Misurato (*Measured service*):** Monitoraggio, controllo e fatturazione trasparente dell'utilizzo effettivo delle risorse (modello *pay-per-use*).
4. **Self-Service On-Demand:** Provisioning automatiche delle capacità computazionali da parte del cliente, senza interazione umana con il fornitore.
5. **Pooling delle Risorse (*Resource pooling*):** Modello *multi-tenant* in cui risorse fisiche e virtuali sono allocate e riallocate dinamicamente in base alla domanda.

13.1.1 Modelli di Servizio Cloud

Il NIST definisce tre modelli di servizio principali, che determinano la granularità del controllo e la ripartizione delle responsabilità operative e di sicurezza tra Cliente (*Cloud Service Consumer* - CSC) e Fornitore (*Cloud Service Provider* - CSP).

- **Software as a Service (SaaS):** Il cliente fruisce di un'applicazione completa erogata dal CSP (es. Microsoft 365, Salesforce). Il cliente demanda l'intera gestione dell'infrastruttura, dei sistemi e dell'applicativo, limitandosi a configurazioni utente base.
- **Platform as a Service (PaaS):** Il CSP fornisce l'infrastruttura sottostante, il sistema operativo e un ambiente di esecuzione pre-configurato. Il cliente ha il controllo esclusivo sullo sviluppo, testing e deployment delle proprie applicazioni (es. Heroku, AWS Elastic Beanstalk).

- **Infrastructure as a Service (IaaS):** Il CSP fornisce unicamente le risorse computazionali virtualizzate di base (CPU, RAM, Storage, Rete). Il cliente ha il controllo completo e l'onere della manutenzione dallo strato del Sistema Operativo in su (es. Amazon EC2, Google Compute Engine).

13.1.2 Modelli di Deployment

L'infrastruttura Cloud può essere implementata secondo quattro topologie primarie:

- **Cloud Pubblico:** Risorse hardware e software di proprietà di terzi e offerte via Internet a molteplici tenant. Vanta bassi costi operativi ma minori controlli.
- **Cloud Privato:** Infrastruttura dedicata a uso esclusivo di una singola organizzazione. Può essere *on-premise* o ospitata. Garantisce massima sicurezza ma alti costi di capitale.
- **Cloud Comunitario:** Condivisione dell'infrastruttura tra organizzazioni indipendenti con requisiti e policy normative affini (es. settore PA o sanitario).
- **Cloud Ibrido:** Architettura composita che orchestra due o più infrastrutture differenti garantendo flessibilità operativa (es. *cloud bursting*).

13.2 Sicurezza nel Cloud Computing

13.2.1 Multi-Tenancy e Isolamento

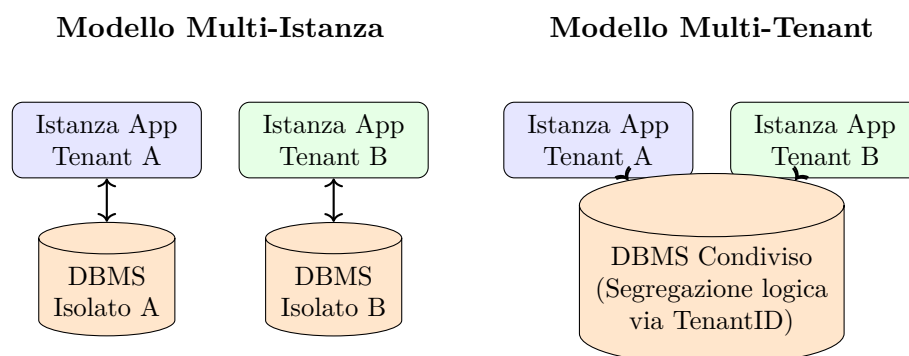
Rischi del Multi-Tenant

L'allocazione delle risorse tramite il modello *multi-tenant* espone i clienti al rischio di *cross-tenant attacks* (un utente malevolo che compromette l'isolamento logico per attaccare altre VM ospiti) o *side-channel attacks* sulla cache condivisa dell'hardware fisico, per dedurre operazioni crittografiche altrui.

13.2.2 Sicurezza dei Dati e Database nel Cloud

La perdita di controllo fisico sui server richiede un affidamento critico alla crittografia (*Data at Rest* e *Data in Transit*). La *Best Practice* impone che i dati vengano crittografati dal cliente **prima** di essere caricati nel Cloud (*client-side encryption*), mantenendo la gestione delle chiavi (*Key Management*) logicamente e fisicamente separata dal CSP.

In ambienti di database Cloud, si distinguono due architetture principali:



Il modello **Multi-Istanza** offre il massimo isolamento logico (un DBMS in una VM dedicata per singolo tenant), mentre il modello **Multi-Tenant** garantisce minor overhead ma comporta un maggiore rischio di fuga di dati in caso di *SQL Injection* (*data leakage*).

13.2.3 Security as a Service (SecaaS) e OpenStack Keystone

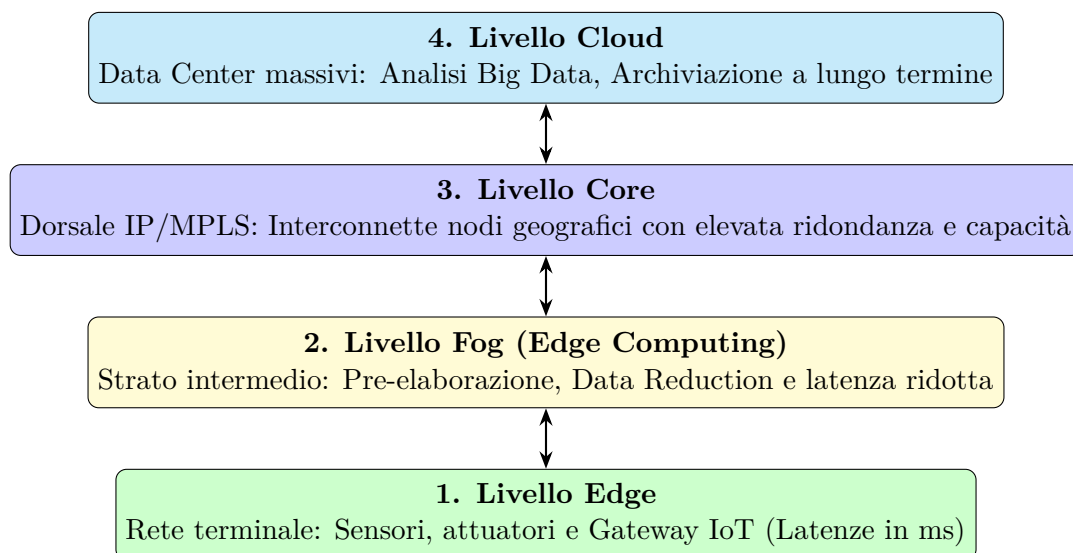
La *Cloud Security Alliance* (CSA) categorizza la fornitura di servizi di sicurezza in outsourcing (SecaaS), tra cui **IAM** (Identity Access Management) e **DLP** (Data Loss Prevention). Nei deployment IaaS open-source basati su *OpenStack*, l'autenticazione e la gestione delle *policy* sono demandate al modulo **Keystone**, che funge da *Identity Provider* centralizzato ed emette *token* di sessione per garantire un sicuro controllo degli accessi (RBAC).

13.3 Internet of Things (IoT) e Livelli di Rete

L'Internet of Things (IoT) rappresenta l'interconnessione globale di dispositivi *deeply embedded*, sensori e attuatori, in grado di comunicare autonomamente con le reti IP pubbliche o private. Un dispositivo IoT (*Smart Object*) è composto tipicamente da: **Sensori**, **Attuatori**, un **Micro-controllore** (MCU) per l'elaborazione locale, e un **Ricetrasmittitore** per la comunicazione wireless. La tecnologia **RFID** funge spesso da abilitatore per l'identificazione passiva.

13.3.1 Architettura di Rete IoT

La complessità topologica dei sistemi IoT richiede un'architettura suddivisa in quattro livelli operativi:



Fog Computing

Il Fog Computing decentralizza logicamente il Cloud. Porta le capacità di elaborazione, stoccaggio e "distillazione" del dato (*data reduction*) fisicamente più vicino ai sensori e ai gateway dell'IoT, garantendo latenze bassissime, risparmiando banda verso la dorsale e filtrando dati rumorosi in locale.

13.4 Sicurezza nell'Ambiente IoT

13.4.1 Requisiti e Framework Operativi

Il **Cisco IoT Security Framework** organizza la sicurezza attorno a requisiti architetturali critici.

Esposizione Fisica e Anti-Tampering

I dispositivi IoT sono spesso schierati in ambienti ostili o non presidiati. Per questo, l'**Anti-Tampering** (il rilevamento e la reazione a manomissioni fisiche invasive sul sensore) diventa un pilastro fondamentale, unitamente a rigidi controlli **RBAC** e alla cifratura persistente dei dati e dei pacchetti IP lungo l'intera dorsale.

13.4.2 Sicurezza Open-Source: Il modulo MiniSec

In contesti estremamente vincolati dal punto di vista computazionale ed energetico, come le *Wireless Sensor Networks* (WSN) governate dal sistema operativo **TinyOS**, l'overhead dei protocolli crittografici tradizionali (come IPsec) è insostenibile. Si impiegano quindi protocolli ottimizzati come **MiniSec**.

MiniSec assicura riservatezza, autenticazione del mittente e protezione dai *Replay Attack* pur minimizzando il consumo energetico. Alla base del suo funzionamento vi sono:

- Il cifrario a blocchi **Skipjack** (progettato storicamente dalla NSA per operare su architetture hardware con un bassissimo *memory footprint*).
- La modalità di cifratura autenticata **OCB (Offset Codebook)**, che si rivela estremamente efficiente perché genera simultaneamente sia il testo cifrato (*ciphertext*) sia il tag di autenticazione dei messaggi in un singolo passaggio elaborativo.

Le due modalità di MiniSec

MiniSec è progettato per adattarsi ai pattern di comunicazione delle reti di sensori:

- **MiniSec-U (Unicast)**: Ottimizzato per la comunicazione punto-punto. Impiega contatori sincronizzati tra nodo mittente e ricevente per scartare pacchetti duplicati o vecchi (mitigando i *Replay Attack*).
- **MiniSec-B (Broadcast)**: Ottimizzato per trasmissioni a più nodi. Per mitigare i *Replay Attack* senza poter sincronizzare un singolo contatore per tutti, divide il tempo in "epoche" e sfrutta la struttura dati probabilistica dei **Bloom Filter** per tracciare i pacchetti ricevuti all'interno della medesima epoca, riducendo drasticamente il consumo di memoria RAM sui minuscoli nodi sensore.

Capitolo 14

Sicurezza IT e Prevenzione dei Rischi

14.1 Definizione e Ciclo di Gestione

La Gestione della Sicurezza IT è il processo formale, strutturato e continuativo atto a sviluppare e mantenere livelli di sicurezza proporzionati per gli *asset* informativi di un'organizzazione.

14.1.1 Il Modello Plan-Do-Check-Act (PDCA)

Il processo di gestione non è statico e *una tantum*, bensì iterativo. Standard internazionali come l'ISO/IEC 27005 (relativo all'Information Security Management System - ISMS) modellano questo processo sul celebre ciclo di Deming, noto come **PDCA**.

- **Plan (Pianificare):** Stabilire le *policy*, definire il perimetro del sistema, eseguire il *Risk Assessment* e sviluppare un Piano di Trattamento del Rischio formale.
- **Do (Eseguire):** Implementare i controlli tecnici e organizzativi definiti nel piano.
- **Check (Verificare):** Monitorare continuativamente l'efficacia dei controlli, misurare la *compliance* interna ed esterna e gestire proattivamente gli incidenti.
- **Act (Agire):** Migliorare il sistema di gestione sulla base dei risultati degli audit, degli incidenti occorsi o dei mutamenti del panorama delle minacce (nuove tecnologie, nuovi attaccanti).

14.2 Contesto e Policy di Sicurezza

L'innescio del processo risiede nel rigoroso allineamento della sicurezza IT con gli obiettivi di *business* dell'organizzazione. La *governance* della sicurezza si declina in tre livelli gerarchici:

1. **Obiettivi di Sicurezza:** I risultati misurabili attesi in termini di protezione (es. "Garantire il 99.9% di uptime per il database clienti o conformità GDPR").
2. **Strategie:** Le metodologie architetturali e operative per raggiungere gli obiettivi definiti.
3. **Policy di Sicurezza:** Il documento programmatico (approvato e sponsorizzato dal *Senior Management*) che definisce l'ambito, i requisiti legali, le responsabilità del personale, la classificazione delle informazioni e l'approccio alla gestione del rischio.

14.3 Il Processo di Risk Assessment

L'importanza del Risk Assessment

La Valutazione del Rischio (*Risk Assessment*) è il motore logico e finanziario dell'ISMS. Senza di essa, l'allocazione delle risorse difensive risulterebbe casuale: l'organizzazione rischierebbe di sotto-proteggere asset critici o di sprecare *budget* per minacce irrilevanti.

14.3.1 Approcci alla Valutazione del Rischio

Approccio	Descrizione e Trade-off
Baseline	Applicazione di configurazioni standard e <i>best practice</i> (es. CIS Benchmarks) a tutti i sistemi indiscriminatamente. <i>Pro</i> : Economico e veloce. <i>Contro</i> : Ignora il profilo di rischio specifico dell'azienda; può risultare insufficiente per i dati critici o inutilmente restrittivo per i sistemi minori.
Informale	Valutazione pragmatica non strutturata basata sull'esperienza (<i>expert judgment</i>). <i>Pro</i> : Estremamente agile. <i>Contro</i> : Soggettivo, incostante e difficile da giustificare in fase di audit normativo.
Dettagliato	Analisi formale, quantitativa o qualitativa rigorosa, di ogni singolo asset, vulnerabilità e minaccia. <i>Pro</i> : Massima <i>assurance</i> e granularità. <i>Contro</i> : Estremamente oneroso in termini di tempo e costi; elevato rischio di obsolescenza rapida del documento.
Combinato	(Raccomandato). Applicazione dell'approccio <i>Baseline</i> per tutti i sistemi di base, seguita da un'analisi informale rapida per individuare i sistemi critici, i quali verranno poi sottoposti ad Analisi Dettagliata. Ottimizza il rapporto costi/benefici.

Tabella 14.1: Confronto degli approcci alla Valutazione del Rischio.

14.4 Analisi Dettagliata del Rischio

14.4.1 1. Contesto e Identificazione degli Asset

Si definisce primariamente l'**Appetito per il Rischio** (*Risk Appetite*) del *management* (il livello massimo di rischio che l'azienda è disposta ad accettare senza azioni correttive) e i vincoli normativi (es. GDPR per l'Europa, HIPAA per la sanità USA). Successivamente, si esegue l'inventario e la classificazione degli **Asset** (inclusi hardware, software, basi di dati, procedure e personale chiave), pesandoli in base all'impatto che la loro compromissione avrebbe sulla continuità aziendale.

14.4.2 2. Identificazione di Minacce e Vulnerabilità

Minacce e Vulnerabilità

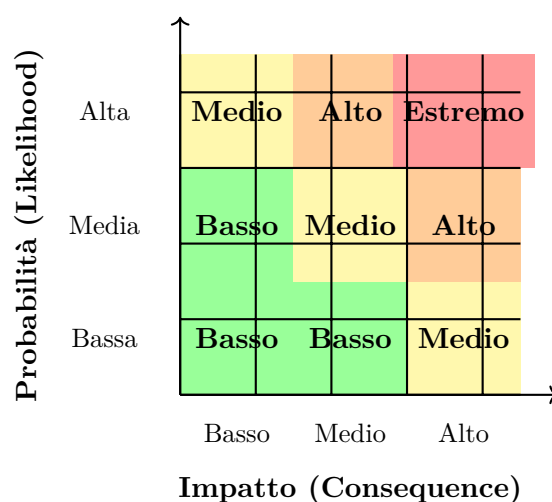
- **Minaccia (*Threat*)**: Un agente o evento potenziale (naturale, umano accidentale, umano deliberato) capace di causare danno all'organizzazione.
- **Vulnerabilità (*Vulnerability*)**: Una debolezza architetturale, difetto software o lacuna procedurale intrinseca all'asset protetto.

14.4.3 3. Analisi e Calcolo del Rischio

Il rischio quantitativo o qualitativo è funzione della probabilità che un evento nefasto si verifichi e dell'entità del danno che ne deriverebbe. Si procede valutando l'efficacia dei **Controlli Esistenti**, per poi stimare due vettori:

1. **Probabilità (*Likelihood*)**: Stimata dall'analista (es. in una scala da 1 - *Raro* a 5 - *Quasi Certo*).
2. **Impatto (*Consequence*)**: Stimato dai proprietari degli asset (*Asset Owner*) e dal management (es. da 1 - *Insignificante* a 5 - *Catastrofico*).

La combinazione di queste due metriche genera una **Matrice del Rischio** che restituisce il Livello di Rischio finale (Basso, Medio, Alto, Estremo).



Tutti i risultati confluiscono in un documento formale e costantemente aggiornato chiamato **Registro dei Rischi** (*Risk Register*).

Le Strategie di Trattamento del Rischio

1. **Riduzione della Probabilità (*Mitigation*)**: Implementazione di controlli preventivi (es. firewall, processi di *patching* regolari, MFA) per rendere l'attacco o l'errore più difficile e improbabile.
2. **Riduzione dell'Impatto**: Implementazione di controlli correttivi o di resilienza (es. backup archiviati *offline*) che limitano il danno a evento avvenuto.
3. **Trasferimento del Rischio (*Transfer*)**: Spostamento dell'onere finanziario o operativo su terzi (es. stipula di polizze assicurative *cyber* o *outsourcing* di servizi critici a provider Cloud qualificati con SLA stringenti).
4. **Evitamento del Rischio (*Avoidance*)**: Dismissione completa del sistema, del processo di business o della tecnologia che genera il rischio, qualora i costi di mitigazione superino di gran lunga i benefici economici.
5. **Accettazione del Rischio (*Acceptance*)**: Riconoscimento e approvazione formale del rischio residuo da parte del *Senior Management* senza ulteriori azioni di mitigazione. Viene tipicamente applicato quando il rischio è già al di sotto dell'appetito aziendale o quando il costo dei controlli necessari eccede il valore reale dell'asset da proteggere.

Capitolo 15

Controlli di Sicurezza

15.1 Classificazione dei Controlli di Sicurezza

Controllo di Sicurezza

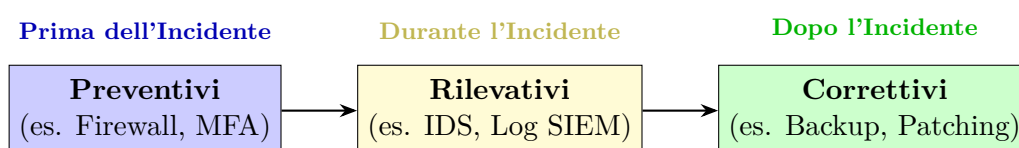
Un **Controllo di Sicurezza** (o contromisura) è un'azione, un dispositivo o una procedura implementata per mitigare un rischio, riducendo la probabilità che si verifichi una violazione della sicurezza o minimizzandone l'impatto qualora dovesse avvenire.

Categorie Strutturali dei Controlli

1. **Controlli di Gestione (*Management Controls*):** Sono di natura direttiva, strategica e amministrativa. Riguardano l'emanazione di *policy* aziendali, la pianificazione, l'esecuzione del *Risk Assessment* e l'autorizzazione formale all'esercizio dei sistemi.
2. **Controlli Operativi (*Operational Controls*):** Eseguiti prevalentemente dal personale o tramite procedure aziendali quotidiane. Includono la formazione del personale (*Security Awareness*), la sicurezza fisica degli edifici, la gestione sicura del personale e la risposta procedurale agli incidenti.
3. **Controlli Tecnici (*Technical Controls*):** Implementati a livello *hardware* e *software* direttamente dal sistema informatico. Includono il Controllo Logico degli Accessi (es. Active Directory), la crittografia, i protocolli di autenticazione e i meccanismi automatizzati di *audit* e *logging*.

15.1.1 Classificazione Funzionale

Dal punto di vista puramente funzionale o temporale, i controlli agiscono in diverse fasi del ciclo di vita di un attacco:



- **Preventivi:** Mirano a bloccare la minaccia alla fonte prima che possa causare danni (es. *firewall*, controllo accessi fisico).
- **Rilevativi:** Non impediscono l'attacco, ma segnalano un'intrusione o un'anomalia in corso (es. Sistemi di Rilevamento delle Intrusioni - IDS, telecamere di sicurezza).
- **Correttivi o di Ripristino:** Intervengono per contenere i danni e ripristinare lo stato sicuro post-incidente (es. procedure di *Disaster Recovery*, ripristino di *backup*, applicazione di *patch* d'emergenza).

15.2 Selezione e Analisi Costi-Benefici

La selezione dei controlli avviene sempre a valle della fase di *Risk Assessment*. L'organizzazione adotta generalmente un approccio ibrido per ottimizzare le risorse: implementa una **Baseline** di controlli standard e trasversali per tutti i sistemi (es. password sicure per tutti), e aggiunge controlli stringenti **Basati sul Rischio** esclusivamente per gli *asset* e i database ritenuti critici per il *business*.

Analisi Costi-Benefici (CBA)

La selezione finale dei controlli è una decisione di *management* governata da una rigorosa **Analisi Costi-Benefici**. Un controllo tecnico o operativo è ritenuto fattibile e conveniente solo se il suo costo (in termini puramente finanziari di acquisizione e di onere operativo/manutentivo) è strettamente inferiore al valore della riduzione del rischio (il danno atteso evitato) che esso garantisce.

I controlli selezionati vengono poi formalizzati in un documento vincolante, l'**IT Security Plan**. Questo piano agisce da *roadmap*, assegnando le risorse finanziarie, le responsabilità al personale e le scadenze temporali precise per la loro implementazione.

15.3 Monitoraggio e Gestione dei Cambiamenti

Le fasi di verifica e mantenimento sono cruciali per prevenire l'obsolescenza difensiva del perimetro di sicurezza, dal momento che il panorama delle minacce è in costante evoluzione.

Deriva delle Configurazioni (*Configuration Drift*)

Senza controlli continui, i sistemi tendono naturalmente a perdere la loro postura di sicurezza nel tempo. Modifiche ad-hoc, *patch* temporanee o disattivazioni non documentate di *firewall* causano la **Configuration Drift**.

Per contrastare questo fenomeno, si implementano:

- **Security Compliance e Audit:** Verifiche periodiche, manuali o automatizzate, per accertare che i controlli previsti siano effettivamente presenti, attivi e utilizzati correttamente, garantendo che le configurazioni siano conformi alle *policy* aziendali o agli standard normativi.
- **Change Management (Gestione dei Cambiamenti):** Un processo formale per la valutazione rigorosa dell'impatto sulla sicurezza di qualsiasi modifica architetturale, aggiornamento *software* o inserimento di nuovi dispositivi prima della loro messa in produzione.

Capitolo 16

Sicurezza Fisica e delle Infrastrutture

16.1 Panoramica

La sicurezza fisica per i sistemi informativi ha due scopi principali:

1. Prevenire danni all'infrastruttura fisica (hardware, edifici, energia elettrica).
2. Prevenire l'accesso e l'uso improprio di tale infrastruttura, al fine di proteggere le informazioni in essa contenute.

16.2 Minacce Principali

Le minacce fisiche possono essere raggruppate in tre macro-categorie generali:

- **Ambientali e Naturali:** Includono eventi catastrofici estremi (terremoti, inondazioni) e condizioni ambientali locali inadeguate, come temperature fuori norma, umidità sbilanciata, incendi, danni da acqua, polvere e infestazioni.
- **Tecniche:** Relativamente all'infrastruttura di supporto. Comprendono anomalie dell'alimentazione elettrica (sottotensioni, blackout, picchi di sovratensione) e le interferenze elettromagnetiche (EMI) che disturbano l'elettronica.
- **Causate dall'Uomo:** Spesso le più imprevedibili, includono l'accesso fisico non autorizzato alle strutture, il furto di hardware e dati, il vandalismo e il sabotaggio intenzionale.

16.3 Misure di Mitigazione e Prevenzione

Controlli e Prevenzione

- **Controlli Ambientali:** Sistemi HVAC per la regolazione del clima, rilevatori di fumo, sensori di allagamento e sistemi automatici per l'estinzione degli incendi.
- **Sicurezza Elettrica:** Gruppi di continuità (UPS) per brevi interruzioni e filtri di rete, affiancati da generatori a combustibile per sopperire a blackout prolungati.
- **Controllo degli Accessi:** Recinzioni, portinerie, serrature biometriche, badge tracciati, allarmi e videosorveglianza continua per impedire intrusioni fisiche.

Capitolo 17

Algoritmi di Crittografia

17.1 Principi Fondamentali e Strutture

Principio di Kerckhoffs

La sicurezza di un crittosistema deve dipendere esclusivamente dalla segretezza della **chiave**. Il sistema deve rimanere sicuro anche se l'attaccante conosce l'algoritmo.

Molti cifrari simmetrici classici (come il DES) si basano sulla **Rete di Feistel**. Il blocco di dati viene diviso in due metà (sinistra e destra). A ogni round, la metà destra viene processata da una funzione non lineare F e il risultato viene unito in XOR con la metà sinistra.

Vantaggio della Rete di Feistel

Garantisce la perfetta reversibilità del processo senza dover implementare due algoritmi distinti per cifratura e decifratura: in fase di decifratura, si applicano semplicemente le sottochiavi in ordine inverso.

Un'altra proprietà per un algoritmo (sia DES che AES) è l'**Effetto Valanga (Avalanche Effect)**: la variazione di un singolo bit nel testo in chiaro o nella chiave deve innescare un cambiamento in circa la metà dei bit del ciphertext finale, rendendo inefficace la crittoanalisi.

17.2 Interno dell'AES (Advanced Encryption Standard)

L'AES utilizza una **rete a sostituzione e permutazione (SPN)** che elabora l'intero blocco di 128 bit in parallelo. Ogni round dell'AES applica quattro trasformazioni distinte:

Substitute Bytes (SubBytes): Sostituzione byte per byte non lineare tramite una S-box, progettata per resistere ad attacchi crittanalitici.

Shift Rows (ShiftRows): Permutazione ciclica sulle righe della matrice di stato per garantire un'ottima *diffusione*.

Mix Columns (MixColumns): Trasformazione lineare sulle colonne, assicurando che l'output dipenda da tutti i bit di input.

Add Round Key (AddRoundKey): Operazione di XOR bit a bit tra la matrice di stato e la sottochiave di round.

17.3 Modalità Operative dei Cifrari a Blocchi

Per elaborare file interi (più lunghi del singolo blocco), si utilizzano modalità operative specifiche. Oltre alla già citata ECB (vulnerabile perché blocchi uguali producono ciphertext uguali), si impiegano:

- **Cipher Block Chaining (CBC):** L'input dell'algoritmo è lo XOR tra il plaintext corrente e il blocco ciphertext precedente. Nasconde i pattern ripetuti. Utilizza un Vettore di Inizializzazione (IV) per il primissimo blocco.
- **Cipher Feedback (CFB):** Trasforma di fatto un cifrario a blocchi in un cifrario a flusso.
- **Counter (CTR):** Utilizza un contatore incrementale cifrato e unito in XOR al plaintext. Offre l'enorme vantaggio del **parallelismo** e dell'accesso casuale ai blocchi.

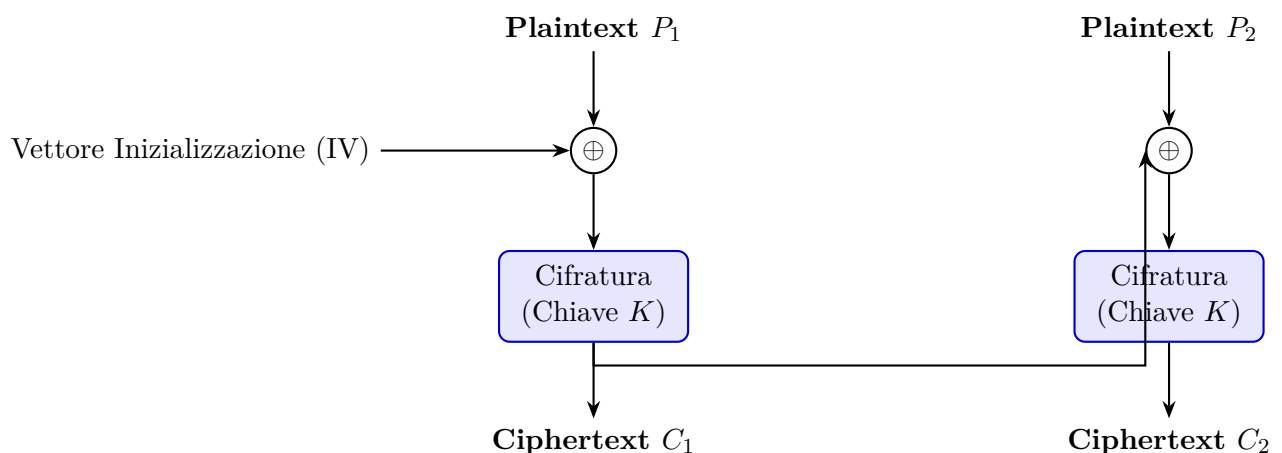
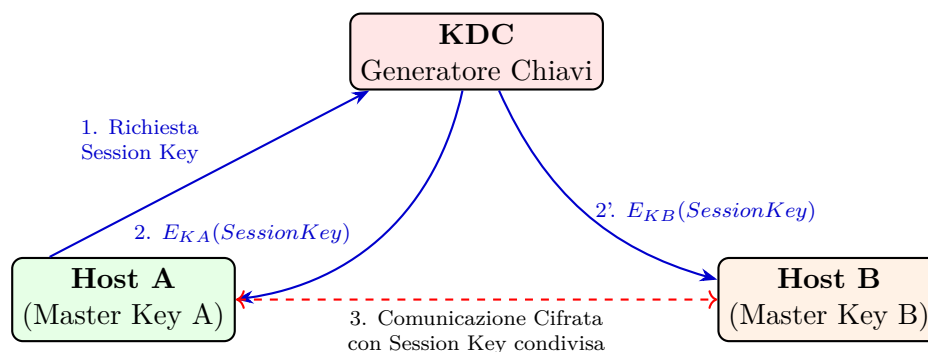


Figura 17.1: Schema concettuale della modalità Cipher Block Chaining (CBC)

17.4 Distribuzione delle Chiavi (KDC)

Per la crittografia simmetrica su reti scalabili (dove scambiarsi fisicamente le chiavi è impossibile), si utilizza un'architettura basata su un **Key Distribution Center (KDC)**. Ogni host condivide in modo sicuro e permanente una singola **Master Key** esclusivamente con il KDC.

Quando l'Host A vuole comunicare con l'Host B, il KDC genera una chiave temporanea **Session Key**, e la invia sia ad A (cifrata con la master key di A) sia a B (cifrata con la master key di B). I due host possono quindi comunicare usando questa chiave di sessione temporanea.



Capitolo 18

Crittografia a Chiave Pubblica e Autenticazione del Messaggio

18.1 Funzioni di Hash Sicure

La funzione di hash unidirezionale, o *funzione di hash sicura*, è un elemento crittografico fondamentale non solo per l'autenticazione dei messaggi, ma anche per le firme digitali. Essa consente di ottenere un'impronta digitale (*digest*) di lunghezza fissa a partire da un input di lunghezza arbitraria, garantendo efficienza computazionale e compattezza del risultato.

Proprietà di una Funzione di Hash Sicura

Le funzioni di hash moderne sono progettate per soddisfare proprietà di sicurezza ben precise:

- **Resistenza alla preimmagine (One-way):** Dato l'hash h , è computazionalmente impraticabile risalire al messaggio originale M .
- **Resistenza alla seconda preimmagine:** Dato un messaggio M_1 , è impraticabile trovare un $M_2 \neq M_1$ tale che $H(M_1) = H(M_2)$.
- **Resistenza alle collisioni:** È impraticabile trovare una qualsiasi coppia di messaggi distinti (M_1, M_2) che producano lo stesso valore di hash $H(M_1) = H(M_2)$.

18.1.1 Funzioni di Hash Semplici

Tutte le funzioni di hash operano secondo principi generali comuni. L'input (messaggio, file, ecc.) viene visto come una sequenza di blocchi di n bit. L'input viene elaborato un blocco alla volta in modo iterativo per produrre un valore di hash di n bit.

Una delle funzioni di hash più semplici è l'operazione di XOR (exclusive-OR) bit a bit di ogni blocco:

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

dove:

- C_i è l' i -esimo bit del codice hash.
- m è il numero di blocchi da n bit.
- b_{ij} è l' i -esimo bit nel j -esimo blocco.

Questa operazione (controllo di ridondanza longitudinale) calcola una semplice parità. È ragionevolmente efficace per rilevare errori accidentali, ma inefficace con dati formattati in modo prevedibile.

Inadeguatezza del RXOR per la Sicurezza

Una semplice miglioria è la procedura RXOR (rotazione di 1 bit del valore di hash per ogni blocco processato). Sebbene fornisca una buona misura di integrità contro l'alterazione casuale, è **inutile per la sicurezza crittografica**: un attaccante può calcolare facilmente un blocco finale ad hoc per generare qualsiasi hash desiderato.

18.1.2 La Funzione di Hash Sicura SHA

Negli ultimi anni, la famiglia di funzioni di hash più utilizzata è stata la Secure Hash Algorithm (SHA).

SHA-1: Pubblicata nel 1995, produce un hash di 160 bit. Oggi è deprecata a causa di vulnerabilità note alle collisioni.

SHA-2: Pubblicata nel 2002, è una famiglia con diverse lunghezze di output (SHA-224, SHA-256, SHA-384, SHA-512). Sono attualmente considerate altamente sicure e rappresentano lo standard industriale.

SHA-3: Pubblicata nel 2015 dal NIST, ha una struttura interna *sponge construction* completamente diversa da SHA-2, offrendo un'alternativa sicura nel caso venissero scoperte vulnerabilità strutturali nella famiglia SHA-2.

18.1.3 Elaborazione di SHA-512

L'algoritmo SHA-512 prende in input un messaggio di lunghezza inferiore a 2^{128} bit e produce un digest di 512 bit, elaborando blocchi sequenziali da 1024 bit.

Il processo si articola in 5 fasi:

1. **Padding:** Il messaggio viene riempito affinché la sua lunghezza sia congruente a 896 modulo 1024 (inserendo un bit '1' seguito dai necessari '0').
2. **Aggiunta della lunghezza:** Un blocco di 128 bit, indicante la lunghezza originale del messaggio, viene accodato. Il messaggio è ora un multiplo esatto di 1024 bit.
3. **Inizializzazione del buffer:** Un buffer di 512 bit (otto registri a 64 bit) viene inizializzato con costanti derivate dalle radici quadrate dei primi 8 numeri primi.
4. **Elaborazione in blocchi:** Il cuore è una funzione di compressione a 80 round. Per ogni blocco, la funzione prende il buffer del passo precedente (H_{i-1}) e produce il nuovo (H_i). L'output dell'80-esimo round viene sommato modulo 2^{64} al buffer di input.
5. **Output:** Dopo l'elaborazione di tutti i blocchi, il contenuto finale del buffer rappresenta il message digest di 512 bit.

18.2 HMAC

Per usare una funzione di hash per l'autenticazione dei messaggi è necessaria una chiave segreta. L'approccio standardizzato è l'**HMAC** (Hash-based Message Authentication Code).

Vantaggi dell'HMAC

Si preferiscono i MAC basati su hash perché le funzioni di hash software sono generalmente più veloci dell'implementazione in software di cifrari simmetrici a blocchi, e le librerie crittografiche sono ampiamente diffuse e ottimizzate.

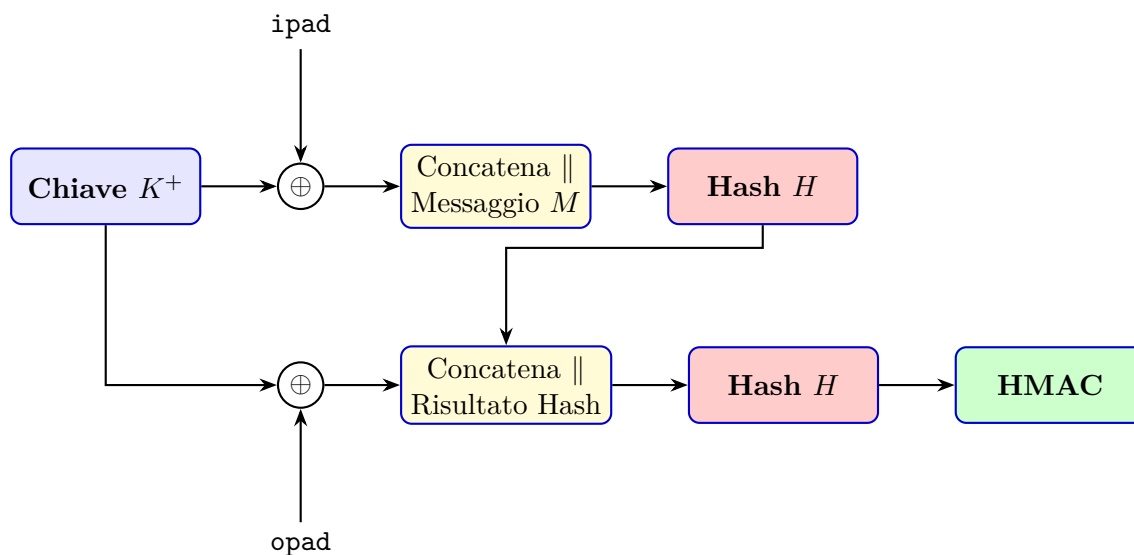
L'algoritmo HMAC combina una chiave segreta con il messaggio tramite due successivi passaggi della funzione di hash:

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

dove:

- H è la funzione di hash (es. SHA-256 o SHA-512).
- M è il messaggio in chiaro.
- K è la chiave segreta condivisa.
- K^+ è la chiave K allungata con zeri fino alla dimensione del blocco interno della funzione H .
- ipad e opad sono stringhe costanti esadecimali predefinite.
- $\parallel$ rappresenta la concatenazione.

La sicurezza di HMAC è formalmente dimostrata ed è strettamente legata alla robustezza della funzione di hash H sottostante. Senza la conoscenza della chiave K , non è possibile lanciare attacchi offline né generare MAC validi.



Schema dell'algoritmo HMAC

18.3 Cifratura Autenticata (AE)

La cifratura autenticata (*Authenticated Encryption*) è una tecnica che garantisce contemporaneamente sia la confidenzialità (cifratura) sia l'integrità/autenticità (MAC) dei dati all'interno di un unico schema ottimizzato.

Un esempio efficiente di tale tecnica è la modalità **Offset Codebook (OCB)**. OCB trasforma un cifrario a blocchi (come AES) in uno schema AE:

$$C[i] = E_K(M[i] \oplus Z[i]) \oplus Z[i]$$

dove E_K è la cifratura con chiave K , e $Z[i]$ è un offset unico e dipendente dalla posizione generato da un *nonce*. L'uso degli offset garantisce che blocchi plaintext identici generino ciphertext diversi, nascondendo i pattern. OCB produce contestualmente un tag di autenticazione complessivo cifrando un checksum crittografico di tutti i blocchi plaintext.

18.4 L'Algoritmo di Cifratura a Chiave Pubblica RSA

Sviluppato nel 1977 da Rivest, Shamir e Adleman, l'RSA è il cifrario asimmetrico a chiave pubblica più diffuso al mondo. È fondamentalmente un cifrario a blocchi in cui sia il plaintext che il ciphertext sono numeri interi compresi tra 0 e $n - 1$.

18.4.1 Descrizione dell'Algoritmo

Le operazioni matematiche di cifratura (C) e decifratura (M) sono:

$$C = M^e \pmod{n}$$

$$M = C^d \pmod{n} = (M^e)^d \pmod{n} = M^{ed} \pmod{n}$$

Le chiavi sono definite come:

- **Chiave Pubblica (pubblica a tutti):** $PU = \{e, n\}$
- **Chiave Privata (segreta):** $PR = \{d, n\}$

18.4.2 Generazione delle Chiavi

1. Scegliere due numeri primi molto grandi e distinti, p e q .
2. Calcolare $n = p \times q$.
3. Calcolare la funzione di Eulero $\phi(n) = (p - 1)(q - 1)$.
4. Scegliere l'esponente pubblico e tale che $1 < e < \phi(n)$ e $\gcd(\phi(n), e) = 1$.
5. Calcolare l'esponente privato d tale che $d \equiv e^{-1} \pmod{\phi(n)}$, ovvero $e \cdot d \equiv 1 \pmod{\phi(n)}$.

Esempio di Calcolo RSA

Siano scelti i piccoli numeri primi $p = 17$ e $q = 11$:

- Si calcola $n = 17 \times 11 = 187$ e $\phi(n) = 16 \times 10 = 160$.
- Si sceglie $e = 7$ (coprimo con 160), e si calcola $d = 23$ (poiché $23 \times 7 = 161 \equiv 1 \pmod{160}$).

Per cifrare il messaggio $M = 88$: $C = 88^7 \pmod{187} = 11$.

Per decifrare il testo $C = 11$: $M = 11^{23} \pmod{187} = 88$.

18.4.3 La Sicurezza di RSA

Gli attacchi contro RSA si dividono in varie categorie:

- **Forza Bruta:** Provare tutte le chiavi private. Mitigato usando chiavi molto lunghe.
- **Attacchi Matematici (Fattorizzazione):** Si basano sulla difficoltà computazionale di fattorizzare n nei suoi fattori p e q , essenziale per calcolare $\phi(n)$ e ricavare d . Attualmente si raccomandano chiavi n comprese tra i 2048 e i 4096 bit.
- **Timing Attacks:** Permettono all'attaccante di dedurre frammenti della chiave misurando con precisione il tempo impiegato per l'esponenziazione modulare. Contromisure includono *blinding* (mascherare il ciphertext prima della computazione).

18.5 Diffie-Hellman e Altri Algoritmi Asimmetrici

Il protocollo Diffie-Hellman è stato storicamente il primo algoritmo a chiave pubblica ed è dedicato specificamente allo **scambio sicuro delle chiavi** per la crittografia simmetrica su canali insicuri, e non alla cifratura di messaggi generici.

Logaritmo Discreto

La sicurezza di Diffie-Hellman si basa sulla difficoltà computazionale del calcolo dei logaritmi discreti in un campo finito: trovare l'esponente i tale che $b \equiv a^i \pmod{p}$ risulta infattibile per grandi valori.

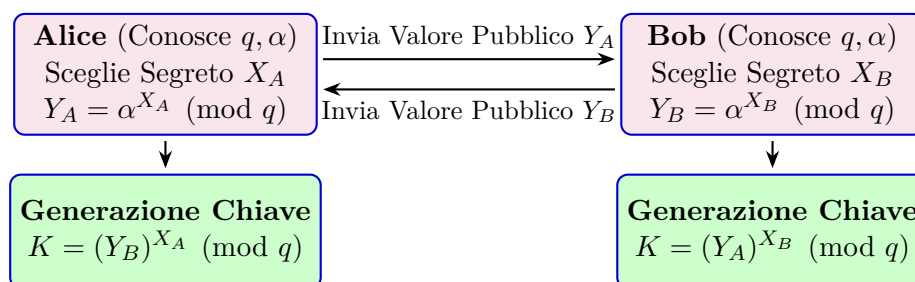
18.5.1 L'Algoritmo Diffie-Hellman

Siano q un numero primo grande e α una sua radice primitiva, parametri pubblici noti ad entrambi.

1. L'utente Alice sceglie un numero casuale segreto $X_A < q$ e calcola $Y_A = \alpha^{X_A} \pmod{q}$.
2. L'utente Bob sceglie un numero casuale segreto $X_B < q$ e calcola $Y_B = \alpha^{X_B} \pmod{q}$.
3. Alice e Bob si scambiano pubblicamente i risultati Y_A e Y_B sulla rete.
4. Alice calcola la chiave condivisa: $K = (Y_B)^{X_A} \pmod{q}$.
5. Bob calcola la chiave condivisa: $K = (Y_A)^{X_B} \pmod{q}$.

Entrambi ottengono la medesima chiave segreta condivisa K poiché:

$$(Y_B)^{X_A} = (\alpha^{X_B})^{X_A} = \alpha^{X_B X_A} = (\alpha^{X_A})^{X_B} = (Y_A)^{X_B} \pmod{q}$$



Scambio di chiavi Diffie-Hellman

Capitolo 19

Sicurezza su Internet

19.1 Secure E-mail e S/MIME

S/MIME (Secure/Multipurpose Internet Mail Extension) è un'estensione di sicurezza allo standard di formattazione e-mail MIME. È stato progettato per fornire servizi crittografici avanzati ai messaggi di posta elettronica, tra cui autenticazione, integrità e riservatezza.

Base Crittografica di S/MIME

S/MIME si basa fortemente sull'uso della crittografia a chiave pubblica asimmetrica e sull'infrastruttura a chiave pubblica (PKI), impiegando certificati digitali X.509 per la gestione sicura delle chiavi e per l'identificazione certa dei mittenti.

19.1.1 S/MIME e Funzionalità

La funzionalità S/MIME è oggi integrata in modo nativo nella maggior parte dei software di posta elettronica e aggiunge nuovi tipi di contenuto MIME per fornire la capacità di firmare e/o cifrare i messaggi, supportando quattro nuove funzioni specifiche:

Dati Busta (Enveloped data): Consiste in un contenuto cifrato di qualsiasi tipo e nelle chiavi simmetriche di cifratura del contenuto, a loro volta cifrate per uno o più destinatari. In questo modo solo i destinatari previsti, che sono in possesso della chiave privata corretta, possono decifrare la chiave e accedere al contenuto del messaggio.

Dati Firmati (Signed data): Una firma digitale creata calcolando il message digest (hash) del contenuto e cifrandolo con la chiave privata del mittente. Il contenuto e la firma vengono codificati in base64. Un messaggio di questo tipo può essere visualizzato solo da un destinatario con capacità S/MIME e garantisce autenticità e integrità.

Dati Firmati in Chiaro (Clear-signed data): Simile ai dati firmati, ma solo la firma digitale viene codificata in base64, mentre il testo rimane in chiaro. Permette ai destinatari privi di client con capacità S/MIME di leggere regolarmente il contenuto del messaggio (pur non potendo verificarne la firma), favorendo la compatibilità.

Dati Firmati e Busta (Signed and enveloped data): Entità annidate in cui dati cifrati possono essere firmati, e dati firmati possono essere cifrati. Questa potente combinazione consente di ottenere simultaneamente riservatezza, autenticità e integrità in un unico messaggio strutturato.

19.1.2 Funzionamento Generale

Fase di Firma (Dati Firmati/Firmati in Chiaro):

1. Viene calcolato un *message hash* del contenuto (tipicamente utilizzando algoritmi robusti come SHA-256).
2. L'hash viene cifrato con la chiave privata del mittente (es. tramite RSA o DSA) per creare la firma digitale vera e propria.
3. La firma viene allegata al messaggio e codificata in base64 (una codifica che mappa i dati binari in caratteri ASCII stampabili sicuri) per proteggerla da alterazioni accidentali in transito.

Fase di Cifratura (Dati Busta):

1. S/MIME genera una chiave di sessione segreta pseudocasuale esclusivamente per il singolo messaggio.
2. Il messaggio viene cifrato con questa chiave di sessione usando un algoritmo simmetrico rapido (es. AES).
3. La chiave di sessione viene quindi cifrata con la chiave pubblica del destinatario (usando l'algoritmo RSA) e trasmessa come "busta" insieme al messaggio cifrato.
4. Il destinatario userà la propria chiave privata per decifrare prima la chiave di sessione e, di conseguenza, il messaggio originale.

19.2 DomainKeys Identified Mail (DKIM)

DKIM è una specifica per la firma crittografica dei messaggi e-mail che permette a un intero dominio mittente (es. *@azienda.com*) di assumersi la responsabilità formale di un messaggio. I destinatari verificano la firma interrogando i record DNS pubblici del dominio mittente.

19.2.1 Architettura della Posta su Internet

Per comprendere DKIM è necessario definire i componenti:

- **MUA (Message User Agent):** Il client di posta dell'utente finale (es. Outlook, Thunderbird).
- **MHS (Message Handling Service):** Composto da:
 - *MSA (Mail Submission Agent):* Accetta il messaggio iniziale dal MUA.
 - *MTA (Message Transfer Agent):* Il server che inoltra il messaggio nella rete.
 - *MDA (Mail Delivery Agent):* Il servizio finale che consegna il messaggio al Message Store.
- **MS (Message Store):** Archivio messaggi in attesa di recupero (via protocolli POP/IMAP).
- **ADMD (Administrative Management Domain):** Provider di posta (ISP o organizzazione aziendale).
- **DNS:** Servizio usato da DKIM per pubblicare le chiavi pubbliche del dominio.

19.2.2 Strategia di DKIM

Differenza tra S/MIME e DKIM

A differenza di S/MIME, che è incentrato sull'utente finale e protegge l'intero contenuto, DKIM è uno strumento implementato a **livello di server** (trasparente per l'utente) ed è fondamentale per combattere minacce come *phishing* e *spoofing* dell'indirizzo di posta.

DKIM opera nel seguente modo:

1. Il messaggio in uscita viene firmato dall'MTA con la chiave privata segreta del dominio amministrativo (ADMD), coprendo parti del contenuto e le intestazioni chiave. Non viene mai usata la chiave personale dell'utente.
2. L'MDA o l'MTA del destinatario recupera la chiave pubblica del dominio mittente eseguendo una query al DNS (record TXT).
3. Se la verifica ha successo, il server conferma che il messaggio proviene effettivamente dal dominio dichiarato e non è stato alterato in transito, migliorando la *reputazione spam* del messaggio.

19.3 Secure Sockets Layer e Transport Layer Security

TLS, il successore standardizzato di SSL, fornisce un servizio di sicurezza *end-to-end* affidabile basandosi sul protocollo di trasporto TCP.

19.3.1 Architettura TLS

TLS è composto da un'architettura a due livelli principali, progettati in modo modulare.

Connessione vs Sessione in TLS

- **Connessione TLS:** Una relazione transitoria e temporanea peer-to-peer per il trasporto sicuro dei dati. Ogni connessione è legata a una specifica sessione.
- **Sessione TLS:** Un'associazione logica tra client e server creata dall'Handshake Protocol. Definisce un insieme di parametri crittografici che possono essere condivisi tra molteplici connessioni simultanee o successive, evitando così le costose operazioni asimmetriche di rinegoziazione.

19.3.2 Protocolli TLS in Dettaglio

Record Protocol: Prende i dati dalle applicazioni, li frammenta, li comprime (opzionale), calcola un MAC (per garantirne l'integrità), li cifra (per la confidenzialità) e aggiunge un'intestazione per l'invio.

Change Cipher Spec Protocol: Un singolo e breve messaggio per segnalare alla controparte il passaggio definitivo all'uso della nuova suite di cifratura appena negoziata.

Alert Protocol: Invia avvisi strutturati (che possono essere errori fatali che chiudono la connessione, o semplici notifiche).

Handshake Protocol: Il cuore di TLS. Autentica le parti, negozia gli algoritmi (cifratura simmetrica, hashing MAC e metodo di scambio chiavi) e stabilisce le chiavi segrete condivise. Si divide in 4 fasi principali (scambio Hello, invio certificati/chiavi dal server, invio certificati/chiavi dal client, e verifica finale con messaggi Finished).

Heartbeat Protocol: Aggiunta recente per monitorare la disponibilità della controparte e prevenire la chiusura silente della connessione da parte dei firewall o dei NAT.

19.4 HTTPS

HTTPS (HTTP over SSL/TLS) combina il protocollo web HTTP e TLS per garantire comunicazioni sicure tra browser e server web (generalmente sulla porta 443).

HTTPS si occupa di cifrare l'intera comunicazione applicativa, tra cui:

- L'URL esatto del documento o della risorsa richiesta
- Il contenuto del documento trasferito e i dati dei moduli compilati dall'utente
- I cookie di sessione e tutte le intestazioni HTTP.

19.5 Sicurezza di IPv4 e IPv6 (IPsec)

IPsec (Internet Protocol Security) è una potente suite di protocolli che fornisce sicurezza direttamente a livello di rete (livello IP), garantendo autenticazione e cifratura del tutto trasparenti per le applicazioni o gli utenti. È lo standard per reti VPN aziendali, accessi remoti ed extranet. Se implementato direttamente su firewall o router di confine, protegge l'intero traffico perimetrale.

19.5.1 Associazioni di Sicurezza (SA)

Una **SA (Security Association)** è una relazione logica **unidirezionale** (simplex) tra mittente e destinatario che definisce esattamente quali servizi crittografici di sicurezza applicare al traffico che vi scorre. Per una comunicazione bidirezionale servono quindi due SA. Una SA è identificata univocamente da tre parametri: Indice dei Parametri di Sicurezza (SPI), l'indirizzo IP di destinazione e l'identificatore del protocollo di sicurezza (tipicamente ESP).

19.5.2 Encapsulating Security Payload (ESP)

ESP è il protocollo cardine di IPsec che fornisce confidenzialità (cifratura), integrità e autenticazione dell'origine dei dati (avendo di fatto sostituito l'obsoleto protocollo AH - Authentication Header).

ESP supporta due distinte modalità operative:

Modalità Transport: Protegge solo il *payload* del pacchetto IP (es. il singolo segmento TCP/UDP), ma lascia l'intestazione IP originale in chiaro. È usata prevalentemente per comunicazioni sicure end-to-end tra due singoli host.

Modalità Tunnel: Protegge l'intero **pacchetto IP** originale, cifrandolo e incapsulandolo all'interno di un **nuovo pacchetto IP** dotato di una nuova intestazione esterna. È usata tipicamente per le VPN (da gateway a gateway), nascondendo completamente l'architettura della rete interna e il traffico ai router pubblici intermedi.

Capitolo 20

Autenticazione su Internet

20.1 Kerberos

Kerberos è uno standard Internet e lo standard *de facto* per l'autenticazione remota in reti locali aziendali, sviluppato originariamente al Massachusetts Institute of Technology (MIT).

Il Modello Kerberos

È un servizio di autenticazione basato su una **terza parte fidata** (*trusted third party*). Client e server non si autenticano scambiandosi direttamente password, ma si affidano a Kerberos per mediare la loro autenticazione reciproca, richiedendo all'utente di provare la propria identità per ogni servizio invocato in modo sicuro.

20.1.1 Il Protocollo Kerberos

Il protocollo Kerberos coinvolge tre entità principali: il client, i server applicativi e il server centrale Kerberos (composto a sua volta da due servizi). È progettato per contrastare varie minacce di rete (in particolare gli attacchi di impersonificazione e *replay*) centralizzando l'autenticazione in modo sicuro tramite crittografia simmetrica (originariamente DES, ora tipicamente AES).

L'**Authentication Server (AS)** conosce tutte le password (hash) di tutti i client registrati. Non si limita a validare l'identità, ma fornisce al client un *ticket* crittografico per richiedere servizi in modo sicuro, senza mai far viaggiare la password in chiaro.

Funzionamento del Protocollo

Il flusso di autenticazione si articola in tre fasi e sei passaggi ben definiti:

1. **Richiesta di un Ticket-Granting Ticket (TGT):** L'utente si collega a una workstation (client) e invia una richiesta iniziale all'AS contenente semplicemente il proprio ID utente in chiaro. L'AS verifica l'ID, crea un Ticket-Granting Ticket (TGT) e una chiave di sessione temporanea. Entrambi vengono **cifrati con una chiave derivata dalla password dell'utente** e inviati indietro al client.
2. **Ottenimento del TGT da parte del Client:** Il client richiede la password all'utente, calcola la chiave derivata e decifra il messaggio dell'AS, recuperando il TGT e la chiave di sessione. Il TGT è una credenziale riutilizzabile (cifrata con una chiave segreta condi-

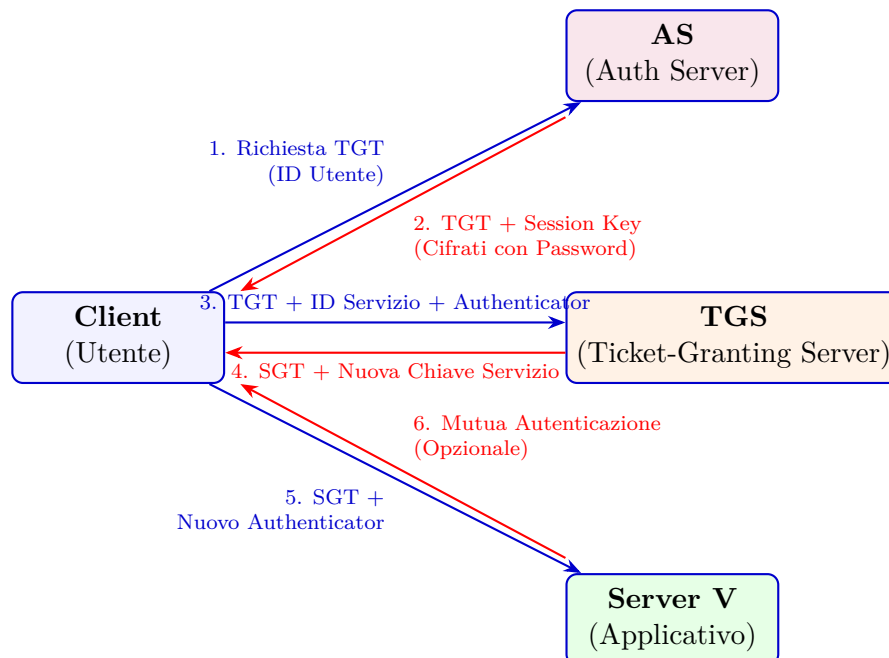
visa esclusivamente tra AS e il Ticket-Granting Server - TGS) contenente l'ID utente, un timestamp, una scadenza temporale e una copia della chiave di sessione.

3. **Richiesta di un Service-Granting Ticket (SGT):** Per accedere a uno specifico server applicativo (es. Server V), il client invia al TGS una richiesta contenente il TGT ottenuto precedentemente e un *Authenticator*.

L'Authenticator di Kerberos

Un **Authenticator** è una struttura dati crittografica *monouso* (one-time) e a vita molto breve (pochi minuti), contenente l'ID del client e il timestamp attuale, cifrata con la chiave di sessione. Poiché scade velocemente, previene gli attacchi di *replay* in cui un attaccante tenta di riutilizzare un ticket intercettato.

4. **Concessione del Service-Granting Ticket:** Il TGS, essendo la stessa entità di Kerberos, decifra il TGT (in cui trova la chiave di sessione) e usa quest'ultima per verificare l'Authenticator. Se valido, crea un Service-Granting Ticket (SGT) specifico per il Server V e lo invia al client insieme a una *nuova chiave di sessione specifica per il servizio*, il tutto cifrato con la chiave di sessione originale. Il SGT è cifrato con una chiave segreta condivisa solo tra TGS e il Server V.
5. **Richiesta del Servizio al Server:** Il client inoltra al Server V il SGT appena ricevuto e un nuovo Authenticator. Il Server V decifra il ticket con la propria chiave privata, verifica l'Authenticator e, se le credenziali corrispondono, concede l'accesso al servizio.
6. **Autenticazione Mutua (Opzionale ma consigliata):** Il server risponde con un messaggio cifrato (es. il timestamp dell'Authenticator incrementato di 1) per dimostrare al client la propria autenticità, impedendo il *server spoofing*.



Flusso di autenticazione del Protocollo Kerberos

20.1.2 Realm Kerberos e Kerberi Multipli

Un ambiente Kerberos completo e isolato (composto da un server Kerberos principale, vari client e server applicativi) è definito **realm**, e coincide generalmente con un dominio amministrativo di rete.

Per consentire l'autenticazione *inter-realm* (cioè un utente di un realm che accede a risorse di un altro), i server Kerberos di due realm diversi devono condividere una chiave segreta stabilita amministrativamente e fidarsi reciprocamente, permettendo così al client di ottenere un TGT locale per richiedere un ticket a un TGS remoto.

20.1.3 Versioni di Kerberos

- **Versione 4:** La prima ampiamente utilizzata, ma presentava vincoli di sicurezza (basata esclusivamente sul cifrario DES).
- **Versione 5:** Introdotta nel 1993 (e integrata nativamente in *Microsoft Active Directory*). Introduce migliorie sostanziali, tra cui il supporto per algoritmi crittografici multipli moderni (come AES), l'*Authentication Forwarding* (delega delle credenziali a un server intermedio per agire per conto del client) e una maggiore scalabilità per l'autenticazione inter-realm (cross-realm trust).

20.2 X.509

Lo standard X.509 (definito dall'ITU-T) è il formato universalmente accettato per i certificati a chiave pubblica, utilizzati trasversalmente in tecnologie come IPsec, TLS/HTTPS, S/MIME, VPN e firme elettroniche.

Cos'è un Certificato X.509?

Un certificato lega crittograficamente una **chiave pubblica** all'**identità** del suo proprietario (soggetto). Per garantirne la validità, il certificato è firmato digitalmente da una terza parte universalmente fidata, chiamata **Certificate Authority (CA)**.

20.2.1 Formato del Certificato

Un certificato X.509 (versione 3) include elementi fondamentali strutturati in modo rigoroso:

- **Nome del Soggetto:** L'entità (persona, server web o organizzazione) proprietaria della chiave.
- **Chiave Pubblica del Soggetto:** Il valore effettivo della chiave e l'algoritmo (es. RSA 2048-bit).
- **Periodo di validità:** Date di "Not Before" e "Not After".
- **Nome dell'Emittente:** L'identità della CA che ha rilasciato il certificato.
- **Firma Digitale della CA:** L'hash crittografico di tutto il certificato, firmato con la chiave privata dell'emittente, che garantisce l'inviolabilità dei dati.

La **versione 3** dello standard ha introdotto estensioni flessibili e critiche, come i *Basic Constraints*, i quali specificano se un certificato appartiene a una CA (ed è quindi abilitato a firmare a sua volta altri certificati) oppure appartiene a un utente finale (end-entity).

20.2.2 Revoca dei Certificati

Un certificato può essere revocato prima della sua naturale scadenza temporale (ad esempio in caso di furto o compromissione della chiave privata del soggetto). I meccanismi principali sono due:

Certificate Revocation List (CRL): Liste pubblicate periodicamente e firmate dalla CA, contenenti i numeri di serie dei certificati non più validi. La loro gestione su larga scala è onerosa e poco reattiva.

Online Certificate Status Protocol (OCSP): Un'alternativa moderna e più leggera che permette a un'applicazione (es. un browser web) di interrogare direttamente e dinamicamente i server della CA sullo stato in tempo reale di un singolo certificato specifico.

20.3 Public-Key Infrastructure (PKI)

Una PKI (Public-Key Infrastructure) è l'insieme complesso di hardware, software, persone, policy aziendali e procedure amministrative necessarie per creare, gestire, distribuire, utilizzare, conservare e revocare in sicurezza i certificati digitali su vasta scala.

20.3.1 Modello di Trust e Problematiche Attuali

La verifica autonoma di un certificato richiede di fidarsi incondizionatamente della CA emittente, creando una **"catena di fiducia" (Trust Chain)** che parte dal certificato del server, passa per CA intermedie, fino a risalire a una *Root CA* (CA Radice) fondamentale.

I sistemi operativi e i browser web attuali includono un *trust store* (archivio di certificati fidati) contenente decine o centinaia di Root CA preinstallate. Questo modello ha dimostrato diverse criticità pratiche:

Debolezze della PKI Pubblica

- **Fiducia orizzontale incondizionata:** Il sistema presume che *tutte* le CA nel trust store siano ugualmente affidabili e competenti. La compromissione di una singola CA debole (come nel disastroso incidente di DigiNotar del 2011) permette di generare certificati falsi ma tecnicamente validi per qualsiasi dominio del mondo.
- **Decisioni dell'utente:** Gli utenti finali spesso non hanno le competenze tecniche per valutare adeguatamente gli avvisi di sicurezza generati dai browser in caso di certificati non validi, portando a bypassarli.
- **Trust store frammentati:** Browser diversi (es. Firefox vs Chrome) e Sistemi Operativi (es. Windows vs macOS) gestiscono e aggiornano trust store del tutto differenti, portando a visioni di sicurezza non uniformi.

20.3.2 Public Key Infrastructure X.509 (PKIX)

Per cercare di standardizzare queste dinamiche, il gruppo di lavoro **PKIX** della IETF ha definito un modello architetturale formale per l'implementazione di una PKI su Internet. Il modello identifica rigorosamente gli elementi chiave del sistema (*End entity*, *CA*, *Registration Authority* o *RA*, e *Repository* CRL) e le relative funzioni di gestione (registrazione degli utenti, certificazione, recupero chiavi e revoca).